

# 1

# Repository

「final\_v2」からの卒業を告げた保管庫

## ひとことというと

プロジェクトの全ての変更履歴をまとめて記録・管理する「保管庫」。

1

## なぜ生まれた？

誰が・いつ・何を変更したか分からなくなり、「final\_v2」「final\_v2\_修正後」のようなファイルが増殖していた。



2

## それまでどうしていた？

フォルダごとコピーして、日付やバージョン名をファイル名に付けて管理していた。

|         | ファイルの手動保存       | Repository   |
|---------|-----------------|--------------|
| 変更履歴の管理 | ファイル名やコメントで手動管理 | コミットとして自動記録  |
| 誰が変更したか | 分からない・本人に聞くしかない | 常に記録され追跡できる  |
| 過去に戻す   | バックアップを探して復元    | いつでも過去の状態を復元 |
| 共有方法    | メールやチャットで送付     | クローンしてそのまま共有 |

3

## Repository で何ができるようになった？



変更履歴を丸ごと記録



いつでも過去の状態に戻せる



チームで一元管理・共有

## ひとことメモ

リポジトリには「ローカル」と「リモート」があり、GitHub はリモートリポジトリを置いておく場所、というだけの位置づけ。

## 関連キーワード

### 関連キーワード



#### Clone

リポジトリを丸ごと手元にコピーすること。

P.2



#### Commit

変更を記録として保存する単位。

P.5

### 一緒に覚えない言葉



#### GitHub

リモートリポジトリを置いておけるWebサービス。



#### Working Tree

今実際に作業しているファイル群のこと。

## 2

# Clone

まるごと持ち帰れる、プロジェクトのコピー機

### ひとことというと

リモートリポジトリの中身を、変更履歴つきで手元にまるごとコピーしてくること。

## 関連キーワード

関連キーワード



### Repository

変更履歴をまとめて記録する保管庫。

P.1



### Pull

クローン後、最新の変更を取り込む操作。

P.7

一緒に覚えたい言葉



### Origin

cloneした際に自動で付くリモートの標準名。



### SSH

安全に通信するための鍵認証の仕組み。

1

## なぜ生まれた？

他のPCやメンバーの環境に、同じプロジェクトを履歴ごと持ってくる方法が必要だった。



2

## それまでどうしていた？

リポジトリの中身をZIPでダウンロードして展開していた(履歴は付いてこない)。

|          | ZIPダウンロード     | Clone             |
|----------|---------------|-------------------|
| 手に入るもの   | 最新ファイルのみ      | 全履歴つきのコピー         |
| 更新の追従    | そのつどダウンロードし直す | pullで差分だけ取得       |
| 変更の反映    | できない・別送するしかない | push・pullで同期できる   |
| 初期セットアップ | 展開してまた一からやり直し | git clone のひと言で完了 |

3

## Clone で何ができるようになった？



履歴ごと丸ごと持ってくる



どのPCでも同じ環境を再現



続きの作業をすぐ始められる

### ひとことメモ

cloneするのは最初の1回だけ。2回目以降は pull で差分だけを取ってくる。

# 3

# Fork

許可なしで、自分だけの改造版を作る技

## ひとことという

他人のリポジトリを、自分のGitHubアカウント配下に複製して自由に改造できるようにする機能。

### 1 なぜ生まれた？

編集権限のないリポジトリに直接手を加えることはできないが、自分なりに改造したり、OSSに貢献したい場面が増えていた。



### 2 それまでどうしていた？

コードをダウンロードして、別プロジェクトとしてゼロから作り直していた。

|         | コードの複製        | Fork               |
|---------|---------------|--------------------|
| 元との関係   | 切れる(ただのコピー)   | 履歴・つながりを保ったまま複製    |
| 貢献のしやすさ | 差分を伝える手段がない   | Pull Requestで提案できる |
| 編集権限    | 不要(そもそも別物になる) | 不要(フォーク先は自分のもの)    |
| 元の更新の追従 | 手動で見比べるしかない   | 元リポジトリの変更を取り込める    |

### 3 Fork で何ができるようになった？



OSSへの貢献ができる



自分の実験場として自由に改造



元に迷惑をかけずに試せる

## ひとことメモ

Fork してから Pull Request を送るまでが、OSS 開発の基本パターン。

## 関連キーワード

関連キーワード



### OSS

ソースコードが公開されたソフトウェア。

P.15



### コントリビュート

プロジェクトへ改善を還元すること。

P.16

一緒に覚えたい言葉



### Pull Request

変更を元のリポジトリに取り込んでもらう提案。



### Upstream

フォーク元の本家リポジトリを指す言葉。

4

# Branch

本流を汚さずに未来を試す並行世界

ひとことというと

本流(main)から分かれて、他に影響を与えずに作業できる「並行世界」を作る仕組み。

1 なぜ生まれた？

新機能の開発中は本番のコードを触れなくなり、複数人が同時に違う機能を作ると変更同士がぶつかっていた。



2 それまでどうしていた？

フォルダをコピーして「app\_new」のような名前で別々に作業していた。

|        | フォルダコピー        | Branch       |
|--------|----------------|--------------|
| 並行作業   | フォルダを増やして管理    | ブランチを分けるだけ   |
| 本流への影響 | コピー作業中は本流を触れない | mainは常に安全なまま |
| 統合方法   | 手作業で見比べてコピー    | mergeで自動的に統合 |
| 切り替え   | フォルダを開き直す      | checkoutで一瞬  |

3 Branch で何ができるようになった？



複数人が同時並行で開発できる



実験して失敗してもmainは無傷



完成したら merge で合流できる

ひとことメモ

「ブランチを切る」という言い方をよくする。main ブランチが本流にあたる。

## 関連キーワード

関連キーワード



Commit

ブランチ上に積み重ねる変更の記録。

P.5



Merge

分かれたブランチをひとつに統合する操作。

P.8

一緒に覚えたい言葉



main / master

プロジェクトの本流となるブランチの名前。



Checkout

作業対象のブランチを切り替える操作。

5

# Commit

「セーブ」に理由を添える1コマ

## ひとことという

変更内容を、メッセージ付きで記録の1コマとして保存すること。

1

## なぜ生まれた？

どこまで手を加えたか、何のための変更だったかが後から分からなくなっていた。



2

## それまでどうしていた？

作業のキリのいいところで、ファイルをまるごと別名保存していた。

|       | 別名保存        | Commit        |
|-------|-------------|---------------|
| 記録の単位 | ファイル全体をまるごと | 変更した差分だけ      |
| 変更理由  | ファイル名や記憶に頼る | コミットメッセージに残す  |
| 巻き戻し  | 前のファイルを探し出す | 特定のコミットにすぐ戻れる |
| 差分確認  | 目視で見比べる     | diffで一目瞭然     |

3

## Commit で何ができるようになった？



変更履歴が全部残る



いつでもその時点に戻れる



「なぜ直したか」が記録に残る

## ひとことメモ

「小さく・こまめに」コミットするのが基本。1 コミット 1 テーマが読みやすい。

## 関連キーワード

関連キーワード



### Branch

コミットを積み重ねる並行世界。

P.4



### Push

コミットをリモートに送る操作。

P.6

一緒に覚えたい言葉



### Staging Area

コミットする変更を仮に置いておく場所。



### Diff

変更前後の差分を表示する仕組み。

6

# Push

手元の作業をチームに届けるひと押し

## ひとことというと

ローカルで積み上げたコミットを、リモートリポジトリに送って反映させること。

### 1 なぜ生まれた？

手元で進めた作業の内容を、チームやサーバーに反映する手段が必要だった。



### 2 それまでどうしていた？

修正したファイルをメールやチャットで送ったり、FTPでアップロードしていた。

|         | ファイル送付         | Push          |
|---------|----------------|---------------|
| 共有方法    | メール・チャットで送付    | コマンド一つで送信     |
| 反映されるもの | ファイルの最新版だけ     | 変更履歴ごと        |
| 衝突      | 気づかず上書きされがち    | 衝突があれば警告される   |
| 手間      | そのつどファイルを探して送る | git push のひと言 |

### 3 Push で何ができるようになった？



コマンド一つでチームに共有



変更履歴ごとサーバーに反映



デプロイの起点にもなる

## ひとことメモ

pushする前に一度 pull して最新を取り込んでおくと、衝突に慌てずに済む。

## 関連キーワード

関連キーワード



### Commit

pushする変更の記録の単位。

P.5



### Pull

逆にリモートの変更を取り込む操作。

P.7

一緒に覚えたい言葉



### CI/CD

pushをきっかけに自動テスト・デプロイする仕組み。



### Force Push

履歴を強制的に上書きするpush。取り扱い注意。

# 7

# Pull

みんなの最新を、自分の手元に引き寄せる

## ひとことというと

リモートリポジトリの最新の変更を、自分の手元に取り込むこと。

1

## なぜ生まれた？

チームメンバーが加えた変更を、自分の環境にも反映させる必要があった。



2

## それまでどうしていた？

最新ファイルを送ってもらい、自分の変更と手作業で見比べながら統合していた。

|      | 手動マージ       | Pull              |
|------|-------------|-------------------|
| 取得方法 | 送ってもらう・都度依頼 | git pull で自動取得    |
| 統合作業 | 目視で見比べて手作業  | fetch+merge で自動統合 |
| 最新性  | 聞かないと分からない  | いつでも最新を取得できる      |
| 衝突対応 | 気づきにくい      | コンフリクトとして明示される    |

3

## Pull で何ができるようになった？



常にチームの最新に追いつける



自分の変更と自動で統合される



衝突があればその場で気づける

## ひとことメモ

pull は「fetch+merge」の合わせ技。中身を分けて考えると仕組みが理解しやすい。

## 関連キーワード

関連キーワード



### Push

自分の変更をリモートに送る操作。

P.6



### Merge

pull の中で行われている統合処理。

P.8

一緒に覚えたい言葉



### Fetch

リモートの変更を取得だけしてmergeしない操作。



### Pull Request

コードレビューを経て取り込む提案の仕組み。

# 8

# Merge

分かれた道を、また1本につなぐ

## ひとことというと

分かれていたブランチの変更を、ひとつに統合すること。

## 1 なぜ生まれた？

ブランチで分かれて進めた作業を、最終的に1つにまとめる必要があった。



## 2 それまでどうしていた？

2つのファイルを並べて見比べ、必要な部分を手でコピペして1つにまとめていた。

|      | 手動統合       | Merge         |
|------|------------|---------------|
| 統合方法 | 目視+手作業でコピペ | コマンドで自動統合     |
| 衝突検出 | 気づかず上書きしがち | コンフリクトとして明示   |
| 履歴   | 統合の過程は残らない | マージコミットとして記録  |
| 対象範囲 | ファイル単位     | プロジェクト全体を一括統合 |

## 3 Merge で何ができるようになった？



ブランチの変更を安全に統合



衝突箇所だけ集中して確認できる



統合の記録が履歴に残る

## ひとことメモ

衝突(コンフリクト)が起きても慌てなくて大丈夫。該当箇所だけ手で直せばいい。

## 関連キーワード

関連キーワード



### Branch

mergeで統合される側の並行世界。

P.4



### Rebase

mergeとは別の、もう一つの統合方法。

P.9

一緒に覚えたい言葉



### Conflict

自動で統合できず人の判断が必要になる状態。

P.11



### Fast-forward

分岐がない場合に、そのままつなげる統合方法。

## 9

# Rebase

枝分かれの歴史を、まっすぐに描き直す

## ひとことという

ブランチの土台を最新の main に乗せ替えて、履歴を一直線に整えること。

### 1 なぜ生まれた？

merge を繰り返すと履歴が枝分かれだらけになり、変更の流れが後から追いにくくなっていた。



### 2 それまでどうしていた？

merge でブランチを統合していた（統合の跡が履歴に残り、次第に複雑になる）。

|         | Merge         | Rebase           |
|---------|---------------|------------------|
| 履歴の見た目  | 枝分かれして複雑      | 一直線できれい          |
| 元のコミット  | そのまま残る        | 新しいコミットとして積み直す   |
| 統合方法    | 最後にまとめて合流     | 土台から作り直して合流      |
| 向いている場面 | チームの統合記録を残したい | 自分のブランチをきれいに整えたい |

### 3 Rebase で何ができるようになった？



履歴が一直線で見やすくなる



レビューしやすいコミットに整理



最新の main を取り込んで作業できる

## ひとことメモ

共有済みのブランチを rebase すると事故のもと。自分のローカルブランチだけに使うのが基本。

## 関連キーワード

### 関連キーワード



#### Merge

rebase と並ぶ、もう一つの統合方法。

P.8



#### Branch

土台を乗せ替える対象そのもの。

P.4

### 一緒に覚えたい言葉



#### HEAD

今いるコミットの位置を指す目印。



#### Squash

複数のコミットを1つにまとめる操作。

# Cherry-pick

欲しい1つだけを、狙って摘み取る

## ひとことという

他のブランチにある特定の1コミットだけを、狙って自分のブランチに取り込むこと。

## 関連キーワード

関連キーワード



### Commit

cherry-pickで摘み取る対象の単位。

P.5



### Branch

摘み取った先の並行世界。

P.4

一緒に覚えてほしい言葉



### Hotfix

本番の緊急バグを直すための修正のこと。



### SHA

コミットを一意に識別するハッシュ値。

## 1 なぜ生まれた?

別ブランチの修正のうち1つだけを今のブランチにも反映したいのに、mergeだと全部ついてきてしまっていた。



## 2 それまでどうしていた?

該当の修正をファイルを開いて目視で探し、コピペして手動で反映していた。

|        | 手動コピペ         | Cherry-pick      |
|--------|---------------|------------------|
| 取り込む範囲 | ブランチ全体をmerge  | 狙った1コミットだけ       |
| 探す手間   | ファイルを見比べて探す   | コミット履歴から指定するだけ   |
| 履歴     | 反映元が分からなくなる   | 新しいコミットとして記録される  |
| 用途     | 全機能をまとめて統合したい | 緊急のバグ修正だけ先に反映したい |

## 3 Cherry-pick で何ができるようになった?



必要な修正だけピンポイントで反映



緊急のバグ修正をすぐ展開



ブランチ全体を統合せずに済む

### ひとことメモ

バグ修正をmain以外の複数ブランチ(リリースブランチなど)に配りたいときによく使う。

# 11

# Conflict

同時に手を加えた瞬間に鳴る警報

## ひとことという

同じ箇所を複数人が別々に変更していて、Gitが自動で統合できなくなった状態。

## 1 なぜ生まれた？

複数人が同じファイルの同じ箇所を同時に編集すると、片方の変更が知らないうちに消えてしまうことがあった。



## 2 それまでどうしていた？

後から保存した人のファイルで上書きされ、先の変更は静かに消えていた。

|         | 無自覚な上書き    | Conflict         |
|---------|------------|------------------|
| 検出      | 気づかれない     | 明示的に警告される        |
| 消える変更   | 静かに消えてしまう  | 両方残った上で選べる       |
| 解決方法    | 誰かに聞いて復元する | 該当箇所を見て手で選ぶ      |
| 発生タイミング | 上書き保存の瞬間   | merge・pull時に判明する |

## 3 Conflict で何ができるようになった？



変更が知らずに消えなくなる



衝突箇所だけピンポイントで確認



どちらを残すか自分で選べる

## ひとことメモ

コンフリクトは「事故」ではなく「事前に気づけてラッキー」くらいの気持ちで直すもの。

## 関連キーワード

### 関連キーワード



### Merge

コンフリクトが表面化しやすい統合操作。

P.8



### Pull

コンフリクトが判明するもう一つの場面。

P.7

### 一緒に覚えない言葉



### マージツール

衝突箇所を見比べながら解決を助けるツール。



### Diff

変更前後の違いを比較する表示形式。

# 12

# Revert

過去の1コマだけを、そっと打ち消す

## ひとことというと

過去のコミットの変更を打ち消す、新しいコミットを作って元に戻すこと。

## 関連キーワード

関連キーワード



### Reset

revertとよく比較される巻き戻し操作。

P.13



### Commit

revertが打ち消す対象の単位。

P.5

一緒に覚えてたい言葉



### ロールバック

システムを以前の状態に戻す一般的な言葉。

## 1 なぜ生まれた？

間違った変更を公開ブランチに反映してしまった後、安全に取り消したい場面があった。



## 2 それまでどうしていた？

何を変えたか思い出しながら、手で逆の修正を書いて元に戻していた。

|        | 手動巻き戻し           | Revert          |
|--------|------------------|-----------------|
| 元に戻す方法 | 記憶を頼りに手で書き直す     | 対象コミットを指定するだけ   |
| 履歴     | 間違いも修正も跡が残らない    | 取り消した記録として残る    |
| 安全性    | 共有後の履歴改変が必要になりがち | 履歴を書き換えずに安全に戻せる |
| 対象範囲   | ファイル単位で手動対応      | コミット単位で正確に指定    |

## 3 Revert で何ができるようになった？



コマンド一つで間違いを打ち消せる



履歴を書き換えずに安全に戻せる



何を取り消したかが記録に残る

## ひとことメモ

公開済み(push済み)のコミットを取り消すときは、resetではなくrevertを使うのが安全。

# 13

# Reset

今いる場所を、指定の地点まで巻き戻す

## ひとことという

コミットの位置を過去に戻し、それ以降の変更をなかったことにする(または手元に戻す)こと。

## 1 なぜ生まれた？

直前のコミットを間違えたり、手元の作業をやり直したいのに  
古いファイルが見つからないことがあった。



## 2 それまでどうしていた？

バックアップフォルダから昔のバージョンのファイルを探し出して上書きしていた。

|        | 旧ファイル探し       | Reset          |
|--------|---------------|----------------|
| 戻し方    | 昔のファイルを探して上書き | コミットを指定して戻すだけ  |
| 戻る範囲   | フォルダ単位でバラバラ   | リポジトリ全体を正確に指定  |
| 変更の扱い  | 上書きすると消えてしまう  | オプションで手元に残せる   |
| 共有後の利用 | 特に区別なく使ってしまう  | 共有済みの履歴では避けるべき |

## 3 Reset で何ができるようになった？



直前のコミットをやり直せる



手元の変更を段階的に戻せる



昔のファイルを探し回らずに済む

## ひとことメモ

pushして共有済みの履歴をresetすると他の人と食い違ってしまう。そこはrevertの出番。

## 関連キーワード

関連キーワード



### Revert

resetより安全な、共有履歴向けの巻き戻し。

P.12



### Commit

resetで戻す先を指定する単位。

P.5

一緒に覚えてたい言葉



### HEAD

今いるコミットの位置を指す目印。



### Working Tree

今実際に作業しているファイル群のこと。

# 14

# Tag

この瞬間に、名前という付箋を貼る

## ひとことというと

特定のコミットに「v1.0.0」のような名前を付けて、目印として残すこと。

## 関連キーワード

関連キーワード



### Commit

タグを貼り付ける対象そのもの。

P.5



### Repository

タグごと履歴を保管する場所。

P.1

一緒に覚えたい言葉



### セマンティックバージョンング

v1.2.3のようにバージョンを意味づけて付けるルール。



### リリースノート

そのバージョンで何が変わったかをまとめた文書。

## 1 なぜ生まれた？

どのコミットが実際にリリースしたバージョンなのか、後から分からなくなっていた。



## 2 それまでどうしていた？

リリースのたびにプロジェクト全体をコピーして「release\_v1.0」フォルダとして保管していた。

|        | フォルダ複製保管        | Tag                |
|--------|-----------------|--------------------|
| 目印の付け方 | フォルダを丸ごとコピーして保管 | 該当コミットに名前を付けるだけ    |
| 容量     | バージョンごとに増え続ける   | 履歴の中の目印だけなので軽い     |
| 探しやすさ  | フォルダを漁って探す      | タグ名で一発検索           |
| 用途     | 手元の目印だけ         | リリースノートやCIの起点にも使える |

## 3 Tag で何ができるようになった？



リリースしたバージョンが一目瞭然



タグを指定してその時点を取得



リリース作業を自動化する起点に

## ひとことメモ

バージョン番号 (v1.0.0 など) をタグに使うルールをセマンティックバージョンングという。

## 15

## OSS

世界中の手で育てる、公開されたコード

## ひとことというと

ソースコードを公開し、誰でも自由に閲覧・改造・再配布してよいソフトウェアのこと。

## 1 なぜ生まれた？

高価なソフトウェアや、中身がブラックボックスなソフトウェアに頼らざるを得ない状況があった。



## 2 それまでどうしていた？

企業が作った非公開のソフトウェアを購入し、決められた機能をそのまま使っていた。

|        | 非公開ソフトウェア      | OSS           |
|--------|----------------|---------------|
| ソースコード | 非公開            | 公開されていて誰でも読める |
| 改造     | できない           | 自由に改造・再配布できる  |
| コスト    | 購入・ライセンス費用がかかる | 多くが無料で使える     |
| 開発体制   | 1社が管理          | 世界中の開発者が共同で改善 |

## 3 OSS で何ができるようになった？



世界中の開発者の知見を活用できる



中身を読んで学習・改造できる



コントリビュートで恩返しできる

## ひとことメモ

OSS にも著作権とライセンスがある。「公開＝何をしてもいい」わけではない点に注意。

## 関連キーワード

関連キーワード



## Fork

OSSに手を加えるための最初の一步。

P.3



## コントリビュート

OSSに改善を還元する行為そのもの。

P.16

一緒に覚えたい言葉



## ライセンス

そのソフトウェアの利用・改変・再配布のルール。



## メンテナー

OSSプロジェクトの管理・運営を担う人。

# コントリビュート

使うだけでなく、育てる側にまわる一歩

## ひとことという

OSSなどのプロジェクトに対して、コードやドキュメントの改善を提案・還元すること。

## 関連キーワード

関連キーワード



### Fork

コントリビュートの第一歩となる複製。

P.3



### OSS

コントリビュートの対象になるプロジェクト。

P.15

一緒に覚えたい言葉



### Pull Request

変更を提案してレビューしてもらう仕組み。



### Issue

バグ報告や要望を管理する場所。

## 1 なぜ生まれた？

OSSの恩恵を受けるだけで、見つけたバグや改善点がプロジェクト全体には反映されない状態が続いていた。



## 2 それまでどうしていた？

不具合を見つけても、手元だけで直して自分のプロジェクトの中で使っていた。

|        | 受け取るだけ         | コントリビュート              |
|--------|----------------|-----------------------|
| 改善の反映先 | 自分の手元だけ        | 元のプロジェクト全体            |
| 方法     | 自己流にファイルを書き換える | ForkしてPull Requestを送る |
| 恩恵     | 自分だけが得る        | 同じ問題で困る全員が得る          |
| 評価     | 特に残らない         | GitHub上に活動履歴として残る     |

## 3 コントリビュートで何ができるようになった？



世界中のユーザーと一緒に育てられる



実務経験として実績が形に残る



レビューを通じて技術力が伸びる

## ひとことメモ

いきなりコードでなくても、誤字修正やドキュメント翻訳から始めるのも立派なコントリビュート。

# 21

# Client

「お客さん」だと思っていたら、ブラウザのことだった

## ひとことという

サーバーに要求(リクエスト)を送る側。ブラウザやスマホアプリなど、自分が直接触っている画面のこと。

1

## なぜ生まれた？

アプリケーションを「要求する側」と「処理する側」に分けずに設計すると、機能が増えるほど役割の境界があいまいになり、どこで何を処理しているかを追いにくなっていた。



2

## それまでどうしていた？

システムを役割分担のない一枚岩として扱い、要求する部分と応答する部分を明確に区別していなかった。

|     | 区別しない頃        | クライアント/サーバーで理解      |
|-----|---------------|---------------------|
| 見え方 | システム全体がひとかたまり | 要求する側＝クライアントと分かる    |
| 実体  | 漠然としたイメージ     | ブラウザ・アプリ・CLIなど具体的な形 |
| 役割  | 切り分けられない      | 表示・入力・要求の送信を担当      |

3

## Client で何ができるようになった？



画面の表示や入力を担当できる



サーバーに要求を送れる



サーバーとの役割分担を説明できる

## ひとことメモ

クライアントは「使う人」ではなく「要求するプログラム」。人間ではなくブラウザやアプリを指すと知ってから、先輩との会話がかみ合うようになった。

## 関連キーワード

対になる技術



### Server

クライアントからの要求を受け取り、処理して返す側。

P.22

やり取りの中身



### Request

クライアントがサーバーに送る依頼のデータ。

P.23



### Response

サーバーがリクエストに対して返す結果。

P.24

# 22

# Server

「なんかサーバー落ちてるらしい」の"サーバー"の正体

## ひとことというと

クライアントからの要求(リクエスト)を受け取り、処理して結果を返す側のプログラムや機械。

### 1 なぜ生まれた？

1台のマシンの中で要求を受け付ける役割と処理する役割が混在していると、負荷が集中した際にどこがボトルネックかを切り分けにくかった。



### 2 それまでどうしていた？

「重い処理をしてくれる場所」という程度の理解にとどまり、担うべき役割が明確に定義されていなかった。

|       | ふわっとした理解   | サーバーの役割で理解             |
|-------|------------|------------------------|
| 正体    | ブラックボックス   | 要求を処理して返すプログラム         |
| 形     | 1台の機械だと思込む | クラウド上の仮想サーバーのことも多い     |
| 落ちる原因 | わからず終い     | プロセス停止・リソース枯渇など具体的に絞れる |

### 3 Server で何ができるようになった？



リクエストを受け取って処理できる



複数のクライアントに同時対応できる



クラウド上でも同じ役割を果たせる

## ひとことメモ

サーバーは「場所」ではなく「役割」。  
同じマシンでも、動かすプログラム次第でクライアントにもサーバーにもなれると知って霧が晴れた。

## 関連キーワード

対になる技術



### Client

サーバーに要求を送る側。ブラウザやアプリなど。

P.21

関連キーワード



### Request

クライアントがサーバーに送る依頼のデータ。

P.23

# 23

# Request

「APIを叩く」の"叩く"の中身、それがリクエスト

## ひとことという

クライアントがサーバーに送る「これをやってほしい」という依頼のデータ。

## 1 なぜ生まれた？

通信の内容(宛先・方法・付随データ)を明示的に構造化しないと、送っている情報の全体像を後から追跡できなかった。



## 2 それまでどうしていた？

通信は「送れば何となく届くもの」として扱われ、中身の構造が特に定義されていなかった。

|      | ブラックボックス操作 | リクエストとして理解           |
|------|------------|----------------------|
| 意識   | 画面の変化だけ見る  | 送っている中身(宛先・内容)を意識する  |
| 中身   | わからない      | メソッド・URL・ヘッダー・ボディで構成 |
| デバッグ | 勘に頼る       | リクエスト内容を確認して原因を特定    |

## 3 Request で何ができるようになった？



通信の中身を確認できる



デバッグの手がかりにできる



API仕様書を読めるようになる

## ひとことメモ

リクエストは「お願いの手紙」。  
誰宛に(URL)、何をしてほしいか(メソッド)、詳細情報(ヘッダー・ボディ)を書いて送っている。

## 関連キーワード

対になる技術



### Response

リクエストに対してサーバーが返す結果。

P.24

構成要素



### Header

リクエストやレスポンスに添える付加情報。

P.34

# 24

# Response

エラー画面の裏側で返ってきていた「返事」

## ひとことというと

サーバーがリクエストに対して返す結果。ステータスコードと中身(ボディ)がセットになっている。

## 1 なぜ生まれた？

処理結果を単なるデータの塊として返すだけでは、成功したのか失敗したのかをクライアント側が機械的に判断できなかった。



## 2 それまでどうしていた？

結果の良し悪しを画面の見た目だけで判断しており、機械が読み取れる形式の情報は用意されていなかった。

|      | 見た目だけで判断   | レスポンスの中身で判断            |
|------|------------|------------------------|
| 判断材料 | 画面の見た目     | ステータスコード+ボディの中身        |
| 原因特定 | 勘          | 200番台/400番台/500番台で切り分け |
| 対応   | とりあえず再読み込み | 中身を見て原因を絞り込んでから対応      |

## 3 Response で何ができるようになった？



結果を正確に受け取る



ステータスコードで切り分けられる



エラーの原因を特定しやすくなる

## ひとことメモ

レスポンスは「返事」。  
中身(ボディ)だけでなく、頭についている数字(ステータスコード)にも情報が詰まっている。

## 関連キーワード

対になる技術



### Request

クライアントがサーバーに送る依頼のデータ。

P.23

関連キーワード



### HTTP

リクエスト・レスポンスをやり取りする共通ルール。

P.25

# 25

# HTTP

URLの前につける謎の文字、の正体

## ひとことというと

クライアントとサーバーがリクエスト・レスポンスをやり取りするための共通ルール(プロトコル)。

### 1 なぜ生まれた？

クライアントとサーバーが異なるメーカー・異なる実装で作られていても通信できるようにするには、共通のやり取りのルールが必要だった。



### 2 それまでどうしていた？

通信のたびに実装ごとの独自の手順でやり取りしており、互換性のない方式が乱立していた。

|        | ルールを意識しない    | HTTPというルールで理解      |
|--------|--------------|--------------------|
| 認識     | 通信は自動でつながるもの | 共通ルールに従ってやり取りしている  |
| やり取り   | なんとなく届く      | メソッド・ヘッダーで構造化されている |
| トラブル対応 | わからず終い       | 仕様に沿って原因を切り分けられる   |

### 3 HTTP で何ができるようになった？



異なる機器同士でも通信できる



構造化されたやり取りができる



仕様書を読んで理解を深められる

## ひとことメモ

HTTPは「会話のルールブック」。  
ブラウザもサーバーもこのルールに従うから、作ったメーカーが違っても会話が成立する。

## 関連キーワード

発展形



### HTTPS

HTTPに暗号化(TLS)を追加した安全な通信。

P.26

構成要素



### Request

クライアントがサーバーに送る依頼のデータ。

P.23



### Response

サーバーがリクエストに対して返す結果。

P.24

# 26

# HTTPS

「Sがついてるだけ」じゃなかった、鍵マークの意味

## ひとことという

HTTPの通信をTLSで暗号化したもの。途中で盗み見・改ざんされにくくする仕組み。

### 1 なぜ生まれた？

通信内容が経路の途中で盗み見・改ざんされる恐れがあり、パスワードやカード番号を安全に送る手段が必要だった。



### 2 それまでどうしていた？

HTTPのまま平文でやり取りしており、通信経路上で内容を読み取られたり書き換えられたりする恐れがあった。

|        | HTTP(暗号化なし)  | HTTPS(TLSで暗号化)  |
|--------|--------------|-----------------|
| 通信内容   | 素通し・覗き見できる   | 暗号化されて中身が読めない   |
| 改ざん    | 途中で書き換えられる恐れ | 改ざんされると検知できる    |
| 信頼できるか | 見た目では判断できない  | 証明書で相手が本物か確認できる |

### 3 HTTPS で何ができるようになった？



通信内容を暗号化できる



相手が本物か証明書で確認できる



改ざんを検知できる

## ひとことメモ

HTTPSは「封筒に入れて送るHTTP」。  
中身が見えなくなるだけでなく、宛先が本物かどうか確認できるようになる。

## 関連キーワード

元になった技術



### HTTP

暗号化されていない通信プロトコル。

P.25

仕組みを支える技術



### TLS

HTTPSの暗号化・認証を担う仕組み。

P.37



### SSL証明書

通信相手が本物であることを証明する電子証明書。

P.38

# 27

# DNS

覚えられない数字を、覚えられる名前に変える仕組み

## ひとことという

ドメイン名(www.example.comなど)をIPアドレスに変換する「インターネットの電話帳」。

### 1 なぜ生まれた?

サーバーの住所であるIPアドレスは数字の羅列で覚えにくく、人間が扱いやすい名前でアクセスできる仕組みが必要だった。



### 2 それまでどうしていた?

接続先をIPアドレスの数字で直接指定するしかなく、覚えるにも共有するにも不便だった。

|         | 名前解決を知らない頃    | DNSとして理解             |
|---------|---------------|----------------------|
| アクセス方法  | ドメイン名を打つだけ    | 裏でIPアドレスに変換されている     |
| 障害時の見え方 | なぜか繋がらない、で終わる | 名前解決の失敗として切り分けられる    |
| 管理      | 意識しない         | Aレコード・CNAMEなど設定を理解する |

### 3 DNS で何ができるようになった?



名前からIPアドレスを引ける



覚えやすい名前でアクセスできる



繋がらない原因を切り分けられる

## ひとことメモ

DNSは「インターネットの電話帳」。  
人間が覚えやすい名前と、機械が使う住所(IPアドレス)を結びつけてくれる。

## 関連キーワード

### 関連キーワード



#### Domain

DNSで管理される、覚えやすい名前そのもの。

P.28



#### IP Address

DNSが変換した先にある、機械の住所。

P.29

# Domain

「ドメイン取った？」の"ドメイン"は住所の名前だった

## ひとことというと

インターネット上の住所(IPアドレス)に付けられた、人間が読める名前。

### 1 なぜ生まれた？

IPアドレスが変わってもアクセス先を一定に保ちたい、複数のサービスを覚えやすい名前で見分けたいという要求があった。



### 2 それまでどうしていた？

サービスへのアクセスをIPアドレスの数字に依存しており、サーバーを移設するたびにアクセス方法が変わってしまっていた。

|    | 名前を意識しない頃   | ドメインとして理解                   |
|----|-------------|-----------------------------|
| 認識 | リンクを辿るだけの存在 | 取得・更新して管理する資産               |
| 構造 | ただの文字列      | www.example.com<br>のような階層構造 |
| 費用 | 無料だと思っていた   | 年単位でレジストラに<br>更新料を払う        |

### 3 Domain で何ができるようになった？



覚えやすい名前が発信できる



DNSに登録してIPアドレスと紐付けられる



自分の名前として管理・更新できる

## ひとことメモ

ドメインは「インターネット上の店名・住所」。  
取得しただけでは終わらず、更新し続けないと他人に取られてしまう。

## 関連キーワード

関連キーワード



### DNS

ドメイン名をIPアドレスに変換する仕組み。

P.27



### SSL証明書

ドメインが本物であることを証明する電子証明書。

P.38

# IP Address

ドメインの奥にある、本当の"住所"

## ひとことという

インターネットに繋がる機器 1 台 1 台に割り振られる、数字だけの住所。

1

## なぜ生まれた？

インターネットに繋がる無数の機器の中から通信相手を一意に特定する、共通の住所の仕組みが必要だった。



2

## それまでどうしていた？

機器同士の接続先を特定する統一的な方法がなく、ネットワークごとに独自の識別方法に頼っていた。

|        | 住所を意識しない頃 | IPアドレスとして理解         |
|--------|-----------|---------------------|
| 接続の仕組み | なんとなく繋がる  | 正確な住所を指定して接続している    |
| 種類     | 区別なし      | グローバルIP/プライベートIPの違い |
| トラブル対応 | 原因不明      | pingやIPで疎通確認ができる    |

3

## IP Address で何ができるようになった？



機器を一意に特定できる



グローバル/プライベートを区別できる



pingなどで疎通確認ができる

## ひとことメモ

IPアドレスは「機械だけがわかる本当の住所」。  
ドメイン名はその住所につけた、人間用のあだ名にすぎない。

## 関連キーワード

### 関連キーワード



#### DNS

ドメイン名をIPアドレスに変換する仕組み。

P.27



#### Port

同じIPアドレスの中で、アプリを区別する番号。

P.30



#### NAT

プライベートIPとグローバルIPを変換する仕組み。

P.41

# Port

`:3000`って何?の答え

## ひとことというと

同じIPアドレス(住所)の中で、どのアプリ・サービス宛かを区別する番号。

### 1 なぜ生まれた?

1 台のサーバーでWebサーバーやメールサーバーなど複数のサービスを同時に動かすには、IPアドレスだけでは宛先を区別できなかった。



### 2 それまでどうしていた?

1 台のマシンには1つのサービスしか動かせない前提で運用されており、複数サービスを同時提供する方法が確立されていなかった。

|        | ポートを意識しない頃     | ポートとして理解          |
|--------|----------------|-------------------|
| 宛先の指定  | IPアドレスだけで届くと思う | IPアドレス+ポート番号で届く   |
| 複数サービス | 1 台 1 サービスだと思う | 同じ機械で複数サービスを同時待受  |
| トラブル対応 | 原因不明           | ポート番号の衝突・未開放に気づける |

### 3 Port で何ができるようになった?



同じ機械で複数サービスを区別できる



アプリ宛にピンポイントで届けられる



衝突や未開放のトラブルに気づける

## ひとことメモ

ポートは「建物(IPアドレス)の中の部屋番号」。同じ建物でも、部屋番号が違えば全く別のサービスに届く。

## 関連キーワード

関連キーワード



### IP Address

機器そのものの住所。ポートはその中の分け。

P.29



### TCP

ポート番号を使って通信を確立するプロトコル。

P.35

# 31

# Cookie

ログインしっぱなしにできる、あの仕組み

## ひとことという

ブラウザに保存される小さなデータ。サーバーがユーザーを識別するために使う。

### 1 なぜ生まれた？

HTTPは1回のやり取りごとに関係がリセットされる仕組みのため、ログイン状態などをリクエストをまたいで維持する方法が必要だった。



### 2 それまでどうしていた？

リクエストごとに関係がリセットされ、ページを移動するたびにログイン情報などの状態が失われていた。

|       | 状態を覚える仕組みがない頃 | Cookieで状態を覚える     |
|-------|---------------|-------------------|
| ページ移動 | 毎回ログアウトされる    | ログイン状態を維持できる      |
| 保存場所  | なし            | ブラウザ側に小さなデータとして保存 |
| できること | 都度ログインが必要     | カート内容やログインを継続できる  |

### 3 Cookie で何ができるようになった？



ログイン状態を維持できる



一定期間データを保持できる



サーバーがユーザーを識別できる

## ひとことメモ

CookieはHTTPの「一期一会」を克服する仕組み。ブラウザに小さなメモを持たせておくことで、サーバーが「前回と同じ人」だと気づける。

## 関連キーワード

関連キーワード



### Session

Cookieを使って実現される、ログイン状態などの管理。

P.32



### JWT

署名付きトークンでログイン情報を持ち運ぶ方式。

P.33

32

# Session

「セッション切れました」のセッションとは何か

ひとことという

ログインから終了までの、ユーザーとサーバーの一連のやり取りをひとまとまりとして管理する仕組み。

1 なぜ生まれた？

Cookieだけでは重要な情報をブラウザ側に平文で持たせることになり、改ざんや漏えいのリスクなくユーザーの状態を管理する方法が必要だった。



2 それまでどうしていた？

ユーザーの状態を示す情報をすべてブラウザ側のCookieに直接持たせており、内容を書き換えられるリスクがあった。

|           | Cookieだけで完結すると思う頃 | セッションとして理解        |
|-----------|-------------------|-------------------|
| 情報の置き場所   | ブラウザ(Cookie)だけ    | サーバー側にも状態を保持      |
| Cookieの役割 | 情報そのものを持つ         | セッションIDという鍵だけを持つ  |
| 期限切れ      | 原因不明で困る           | 有効期限やログアウトで切れると理解 |

3 Session で何ができるようになった？



ログインから終了までを管理できる



サーバー側で状態を安全に保持できる



期限切れの挙動を理解できる

ひとことメモ

セッションは「サーバー側のロッカー」、Cookieは「そのロッカーの鍵」。鍵(セッションID)だけをブラウザに持たせ、中身はサーバー側で管理する。

関連キーワード

関連キーワード



Cookie

セッションIDなどを保存する、ブラウザ側の小さなデータ。

P.31



JWT

サーバー側に状態を持たない、別の認証情報の持ち方。

P.33

# JWT

## セッションを使わないログイン、の裏側

### ひとことというと

ユーザー情報を署名付きのトークンに詰め込んで持ち運ぶ、サーバー側に状態を持たない認証方式。

1

### なぜ生まれた？

サーバーを複数台に増やして負荷分散すると、どのサーバーがリクエストを受けてもログイン状態を判定できる仕組みが必要になった。



2

### それまでどうしていた？

ログイン状態をサーバー側のセッションストアで管理しており、複数サーバー間で情報を共有する仕組みが必要だった。

|          | セッション方式         | JWT                   |
|----------|-----------------|-----------------------|
| 状態の保持場所  | サーバー側(セッションストア) | トークン自体に情報が入っている       |
| 複数サーバー対応 | セッション共有の仕組みが必要  | トークンさえあればどのサーバーでも検証可能 |
| 改ざん対策    | サーバーが正本を持つので安心  | 署名で改ざんを検知する仕組みが必須     |

3

### JWT で何ができるようになった？



サーバー側に状態を持たず認証できる



複数サーバー間で認証を共有しやすい



署名で改ざんを検知できる

### ひとことメモ

JWTは「サインが入った身分証」を毎回持ち歩くイメージ。中身は誰でも読めるが、署名があるから偽造や改ざんはすぐバレる。

## 関連キーワード

### 関連キーワード



#### Session

サーバー側で状態を管理する、もう一つの方式。

P.32



#### Cookie

JWTをブラウザに保存する際の入れ物としても使われる。

P.31



#### Header

JWTは多くの場合 Authorizationヘッダーに載せて送る。

P.34

# 34

# Header

本文じゃない、荷物の"送り状"の部分

## ひとことというと

リクエストやレスポンスの本体(ボディ)とは別に付け加える、付加情報の集まり。

### 1 なぜ生まれた？

本文(データそのもの)とは別に、認証情報やデータ形式などの付加情報を通信のたびにやり取りする仕組みが必要だった。



### 2 それまでどうしていた？

やり取りされる情報は本文のみを想定しており、認証情報や形式指定などの付加情報を載せる場所が定義されていなかった。

|          | 本文だけを意識する頃   | ヘッダーも意識する                   |
|----------|--------------|-----------------------------|
| やり取りする情報 | 本文(ボディ)だけ    | 本文+付加情報(ヘッダー)               |
| 認証情報     | どこに入れるか分からない | Authorizationヘッダーに載せる       |
| トラブル対応   | 原因が本文しか見えない  | Content-Typeなどヘッダーからも原因を探れる |

### 3 Header で何ができるようになった？



本文とは別に情報を付け加えられる



認証トークンなどを安全に運べる



通信のトラブルを切り分けやすくなる

## ひとことメモ

ヘッダーは「荷物の送り状」。中身(本文)そのものではなく、宛先や種類・認証情報といった付加情報が書かれている。

## 関連キーワード

関連キーワード



### Request

ヘッダーが付け加えられる、依頼データそのもの。

P.23



### JWT

Authorizationヘッダーに載せて送る認証トークン。

P.33

# 35

# TCP

「届いたか確認する」通信、その名前

## ひとことというと

データが確実に届いたかを確認しながら通信するプロトコル。信頼性を重視する。

1

## なぜ生まれた？

ネットワーク上でデータが途中で失われたり順序が入れ替わったりする中で、送った内容が確実に、正しい順番で届くことを保証する仕組みが必要だった。



2

## それまでどうしていた？

データを送りっぱなしにしており、途中で欠落したり順序が入れ替わったりしても検知・再送する手段がなかった。

|          | 確認する仕組みを知らない頃 | TCPとして理解           |
|----------|---------------|--------------------|
| 届いたかの確認  | 考えたことがない      | 受信確認(ACK)をしながら通信する |
| 順序       | 気にしていない       | 送った順番通りに届くよう保証する   |
| 速度と引き換えに | 特に意識しない       | 確認作業がある分、UDPより多少遅い |

3

## TCP で何ができるようになった？



データの到達を確認しながら送れる



順序を保証して届けられる



信頼性が必要な通信に向いている

## ひとことメモ

TCPは「配達完了の確認を取る宅配便」。  
多少時間がかかっても、確実に・順番通りに届けることを優先する。

## 関連キーワード

対になる技術



### UDP

確認を取らず、とにかく速さを優先するプロトコル。

P.36

関連キーワード



### Port

TCPが通信相手のアプリを特定するために使う番号。

P.30

# UDP

「確認より速さ」を選んだ通信

## ひとことというと

データが届いたかの確認をせず、とにかく速く送ることを優先するプロトコル。

## 関連キーワード

対になる技術



### TCP

確認を取りながら確実に届けるプロトコル。

P.35

## 1 なぜ生まれた？

動画配信やリアルタイム通信では、確認応答による遅延よりも速度が優先され、多少のデータ欠落を許容してでも高速に送る仕組みが求められた。



## 2 それまでどうしていた？

TCPのように確認応答を取りながら通信しており、リアルタイム性が求められる用途では確認のオーバーヘッドが遅延として無視できなかった。

|         | TCPの発想しか知らない頃   | UDPとして理解                |
|---------|-----------------|-------------------------|
| 届いたかの確認 | 必ず確認するものだと思う    | 確認せず送りっぱなしにできる          |
| 速度      | 確認作業の分、多少遅れると思う | 確認がない分、圧倒的に速い           |
| 向いている場面 | 特に区別しない         | 動画配信・ゲームなど多少の欠落を許容できる場面 |

## 3 UDP で何ができるようになった？



確認を省いて高速に送れる



リアルタイム性が求められる通信に向く



多少のデータ欠落を許容できる

### ひとことメモ

UDPは「確認は取らないから代わりに速い宅配便」。多少荷物が届かなくても、止まらず配り続けることを優先する。

## 37

## TLS

## HTTPSの"S"の中身を作っている技術

## ひとことというと

通信を暗号化し、相手が本物であることも確認する仕組み。HTTPSの安全性を支えている。

## 1 なぜ生まれた？

旧来のSSLに脆弱性が発見され、暗号方式を強化しつつ互換性を保った新しい暗号化の仕組みが必要になった。



## 2 それまでどうしていた？

SSLという名称の暗号化方式が使われていたが、既知の脆弱性が積み重なり安全性を維持できなくなっていた。

|        | SSL(旧世代)           | TLS(後継)          |
|--------|--------------------|------------------|
| 名前     | SSL 2.0/3.0        | TLS 1.0~1.3として進化 |
| 安全性    | 脆弱性が見つかり非推奨に       | 暗号方式を強化して安全性を維持  |
| 現状の呼び方 | 慣習で"SSL証明書"と呼ばれ続ける | 実体はTLSで通信している    |

## 3 TLS で何ができるようになった？



通信内容を暗号化できる



通信相手が本物か確認できる



古いバージョンを無効化して安全性を保てる

## ひとことメモ

「SSL証明書」という呼び方は残っているが、実際に通信を守っているのは後継技術のTLS。名前だけが昔のまま定着している。

## 関連キーワード

## 関連キーワード



## HTTPS

TLSによって暗号化されたHTTP通信。

P.26



## SSL証明書

TLS通信で相手の身元を証明する電子証明書。

P.38



## Let's Encrypt

無料でSSL証明書を発行してくれる認証局。

P.39

# SSL証明書

「証明書の期限切れ」で赤くなる画面の正体

## ひとことというと

サイトの運営者が本物であることを証明する電子証明書。TLS通信で使われる。

### 1 なぜ生まれた？

通信を暗号化するだけでは、接続先のサーバーが本物かどうかまでは保証できず、なりすましを見抜く手段が必要だった。



### 2 それまでどうしていた？

接続先が本物かどうかを、URLの見た目やドメイン名の印象だけで判断するしかなかった。

|      | 見ただ目で判断する頃   | 証明書で判断                |
|------|--------------|-----------------------|
| 本人確認 | URLの見た目だけで判断 | 認証局が発行した証明書で確認        |
| 期限   | 意識しない        | 有効期限があり、切れると警告が出る     |
| 取得方法 | 存在を知らない      | 認証局に申請、または自動発行サービスを使う |

### 3 SSL証明書 何ができるようになった？



サイト運営者の身元を証明できる



なりすましサイトと区別できる



TLSによる暗号化通信の土台になる

## ひとことメモ

SSL証明書は「サイトの身分証明書」。期限が切れると「本人確認ができない状態」になるので、ブラウザが警告を出す。

## 関連キーワード

関連キーワード



### TLS

この証明書を使って暗号化通信を行う仕組み。

P.37



### Let's Encrypt

この証明書を無料・自動で発行してくれるサービス。

P.39

# Let's Encrypt

「証明書、無料でいいの?」と最初は疑った

## ひとことというと

SSL 証明書を無料・自動で発行してくれる認証局。個人開発のサイトも簡単にHTTPS化できる。

1

## なぜ生まれた?

SSL 証明書の取得に費用と手続きがかかるため、個人サイトや小規模プロジェクトでは常時HTTPS化が後回しにされがちだった。



2

## それまでどうしていた?

SSL 証明書は有料の認証局に申請し、書類確認などの手続きを経て取得するものだった。

|       | 有料・手続き重視の時代  | Let's Encrypt         |
|-------|--------------|-----------------------|
| 費用    | 有料が当たり前      | 無料で発行できる              |
| 発行手続き | 書類・審査に時間がかかる | ドメイン所有の自動確認のみで発行      |
| 更新    | 手動で更新申請      | 自動更新の仕組みを組める(有効期限は短め) |

3

## Let's Encrypt で何ができるようになった?



無料でSSL証明書を発行できる



発行から設定まで自動化できる



個人サイトでも気軽にHTTPS化できる

## ひとことメモ

Let's Encryptのおかげで、「常時HTTPS」が個人サイトでも当たり前になった。無料の裏には非営利団体による運営がある。

## 関連キーワード

関連キーワード



### SSL証明書

Let's Encryptが発行してくれる証明書そのもの。

P.38



### HTTPS

この証明書によって実現される暗号化通信。

P.26

# VPN

「社内ネットワークに繋いで」と言われて焦った日

## ひとことという

インターネット上に暗号化された専用の通信路(トンネル)を作り、離れた場所からでも安全に社内ネットワークなどに接続する仕組み。

## 関連キーワード

関連キーワード



### NAT

プライベートIPとグローバルIPを変換する仕組み。

P.41



### Proxy(フォワードプロキシ)

クライアントの代わりに通信を中継する仕組み。

P.43

## 1 なぜ生まれた?

拠点や自宅など、離れた場所から社内ネットワークへ安全に接続する手段が求められたが、専用線の敷設には多額の費用がかかった。



## 2 それまでどうしていた?

社外から社内ネットワークへアクセスするには、専用線を敷設するか、直接出社するしかなかった。

|      | 専用線・出社前提     | VPN                  |
|------|--------------|----------------------|
| 接続方法 | 専用線の敷設や出社が必要 | インターネット経由で仮想的な専用線を作る |
| コスト  | 専用線は高額       | ソフトウェアで低コストに実現       |
| 安全性  | 物理的に隔離されている  | 暗号化されたトンネルで安全性を確保    |

## 3 VPN で何ができるようになった?



外部から社内ネットワークに安全に接続できる



通信を暗号化してトンネルを作れる



リモートワークでも同じ環境で作業できる

## ひとことメモ

VPNは「インターネットの中に作る秘密の通路」。公共の道路(インターネット)の中に、覗かれないトンネルを掘るイメージ。

# 41

# NAT

自宅のPCが全部同じ"グローバルIP"に見える理由

## ひとことという

家庭や社内など、限られた場所だけで使うプライベートIPと、インターネット上のグローバルIPを変換する仕組み。

### 1 なぜ生まれた？

インターネットに接続する機器の数に対して、割り当て可能なグローバルIPアドレスの数が不足していた。



### 2 それまでどうしていた？

接続する機器1台ごとに個別のグローバルIPアドレスを割り当てる前提で運用されていた。

|          | 1台1グローバルIP<br>だと思っ頃 | NATとして理解                    |
|----------|---------------------|-----------------------------|
| IPの割り当て  | 機器ごとに別のグローバルIPが必要   | 家庭内はプライベートIP、外向きは共通のグローバルIP |
| IPアドレス不足 | 特に意識しない             | 限られたグローバルIPを節約できる           |
| 見え方      | 各機器が別々に見えると思う       | 外からは同じグローバルIPの中の1台に見える      |

### 3 NAT で何ができるようになった？



プライベートIPとグローバルIPを変換できる



限られたグローバルIPを節約できる



内部のネットワーク構成を外部から隠せる

## ひとことメモ

NATは「マンションの代表電話番号」。  
部屋(プライベートIP)はバラバラでも、外から見える番号(グローバルIP)は1つにまとめられる。

## 関連キーワード

関連キーワード



### IP Address

NATが変換する対象、プライベート/グローバルの住所。

P.29



### サブネット (Subnet)

プライベートIPを整理して区切る仕組み。

P.42

# サブネット (Subnet)

IPアドレスを"部署ごと"に区切る発想

## ひとことというと

1つのネットワークを、用途や部署などの単位で複数の小さなネットワークに分割する仕組み。

### 1 なぜ生まれた？

ネットワーク全体を1つの区画のまま運用すると、障害の影響範囲が広がりやすく、用途ごとのアクセス制御も難しかった。



### 2 それまでどうしていた？

ネットワークを1つの大きな区画としてまとめて管理しており、用途や部署ごとに細かく制御する手段がなかった。

|         | 1つの大きなネットワーク | サブネットで分割            |
|---------|--------------|---------------------|
| 管理単位    | 全部まとめて1つ     | 用途ごと(公開用・非公開用など)に分割 |
| 障害の影響範囲 | 全体に波及しやすい    | 範囲を区切って影響を限定できる     |
| セキュリティ  | 一律の設定しかできない  | 区画ごとに細かくアクセス制御できる   |

### 3 サブネット(Subnet) で何ができるようになった？



ネットワークを用途ごとに分割できる



区画ごとにアクセス制御できる



障害やトラブルの影響範囲を限定できる

## ひとことメモ

サブネットは「大きなオフィスのフロア分け」。全員を1フロアに詰め込まず、部署ごとに区切ることで管理しやすくする。

## 関連キーワード

関連キーワード



### IP Address

サブネットが分割の対象とする住所そのもの。

P.29



### NAT

プライベートIPとグローバルIPを変換する仕組み。

P.41

# 43

# Proxy(フォワードプロキシ)

「社内からは直接アクセスできません」の理由

## ひとことという

クライアントの代わりにインターネットへアクセスし、結果を取り次いでくれる中継役のサーバー。

### 1 なぜ生まれた？

社内の全端末から個別に外部へ直接アクセスすると、アクセス制御やログ管理を端末ごとに行う必要があり、統制が取れなかった。



### 2 それまでどうしていた？

クライアントがそれぞれ直接インターネット上のサーバーに接続しており、アクセス制御やキャッシュを一括で行う仕組みがなかった。

|      | 直接接続しか知らない頃     | プロキシを経由すると理解         |
|------|-----------------|----------------------|
| 接続経路 | クライアントから直接サーバーへ | クライアント→プロキシ→サーバーと中継  |
| メリット | 特に意識しない         | アクセス制御・キャッシュ・匿名化ができる |
| 見え方  | 個々の機器がそのまま外に見える | 外からはプロキシだけが見える       |

### 3 Proxy(フォワードプロキシ) で何ができるようになった？



クライアントの代わりに通信を中継できる



アクセス制限やログ管理をまとめて行える



内部のクライアント構成を外部から隠せる

## ひとことメモ

フォワードプロキシは「会社の代表窓口」。  
社員(クライアント)は直接取引先に行かず、窓口(プロキシ)を通して外部とやり取りする。

## 関連キーワード

関連キーワード



### VPN

暗号化されたトンネルで社内ネットワークに接続する仕組み。

P.40



### NAT

プライベートIPとグローバルIPを変換する仕組み。

P.41

51

# JSON

「閉じタグの海」から抜け出したデータの書き方

## ひとことという

{ } と [ ] だけで表現する、JavaScript のオブジェクトそのままの見た目のデータ形式。

### 1 なぜ生まれた？

API やシステム間でのデータ交換に XML を使うと、開始・終了タグの記述量が多くなり、パース処理も複雑になりがちだった。より簡潔に書けるデータ形式が求められていた。



### 2 それまでどうしていた？

2000 年代までは XML で構造化データをやりとりするのが主流だった。

|       | XML          | JSON            |
|-------|--------------|-----------------|
| 書き方   | 開始・終了タグが必須   | { } と [ ] だけ    |
| 読みやすさ | ネストが深いと追いつらい | JS のオブジェクトと同じ感覚 |
| パース   | 専用パーサーが必要    | JS ならそのまま扱える    |

### 3 JSON で何ができるようになった？



設定や API レスポンスを簡潔に表現



言語をまたいでそのままやりとり



配列とオブジェクトだけで大抵表現できる

## ひとことメモ

JSON は「JavaScript Object Notation」の略。  
名前の通り JS のオブジェクトそのままの書き方だけど、今は JS 以外の言語でも共通語として使われている。

## 関連キーワード

比較する技術



### XML

タグで構造を表現する古参フォーマット。今も設定や仕様書で現役。

P.54



### YAML

インデントで構造を表す、設定ファイル向けの書き方。

P.52

よく使う場面



### API

JSON は API のレスポンス形式として最も広く使われている。

P.61

# 52

# YAML

インデントだけで構造を語る、設定ファイルの定番

## ひとことという

波括弧を使わず、インデント(字下げ)だけで階層構造を表現するデータ形式。

1

## なぜ生まれた？

設定ファイルをJSONで書くとコメントを残せず、括弧やカンマの対応関係も見落としやすかった。人が手で書きやすく、意図を書き添えられる設定形式が求められた。



2

## それまでどうしていた？

JSONでも設定は書けるが、コメントが書けず、括弧やカンマの数え間違いも起きやすい。

|         | JSON      | YAML       |
|---------|-----------|------------|
| 見た目     | { } と、が並ぶ | インデントのみ    |
| コメント    | 書けない      | # で書ける     |
| 向いている場面 | APIのレスポンス | 手で書く設定ファイル |

3

## YAML で何ができるようになった？



手書きしやすい設定ファイルが書ける



コメントを残しながら設定できる



GitHub Actions やCIの定義に使われる

## ひとことメモ

YAMLは「YAML Ain't Markup Language」の再帰的な略称。読みやすさ重視な分、インデントのミスがそのまま解釈エラーになるので、エディタのタブ設定表示は必須。

## 関連キーワード

比較する技術



### JSON

括弧で構造を表すデータ形式。APIレスポンスで定番。

P.51



### TOML

YAMLよりシンプルな階層表現を旨指した設定形式。

P.53

# 53

# TOML

「インデントミスに悩みたくない」から生まれた設定形式

## ひとことというと

key = value の並びで設定を書く、あいまいさの少ないシンプルな設定ファイル形式。

## 関連キーワード

比較する技術



### YAML

インデントで階層構造を表す設定ファイル形式。

P.52



### JSON

括弧で構造を表すデータ形式。プログラムからの生成向き。

P.51

## 1 なぜ生まれた？

パッケージ管理ツールの設定ファイルには、複雑な入れ子構造よりも誰が読んでも一意に解釈できるシンプルな記法が向いていた。あいまいさの少ない設定形式が求められた。



## 2 それまでどうしていた？

YAML は表現力が高い分、インデントの深さやリストの書き方に複数の流儀があり、ツールによって解釈が割れることがあった。

|         | YAML          | TOML           |
|---------|---------------|----------------|
| 書き方の自由度 | 高い(複数の書き方がある) | 低い(迷いにくい)      |
| ネスト表現   | インデントで表現      | [section] で区切る |
| 向いている場面 | 複雑な設定・CI 定義   | ツールの設定ファイル 1 枚 |

## 3 TOML で何ができるようになった？



key = value で直感的に書ける



Cargo.toml など  
パッケージ設定の  
定番



書き方のブレが少なく読み間違えにくい

### ひとことメモ

TOML は「Tom's Obvious, Minimal Language」の略。  
名前の通り「誰が読んでも解釈がブレない」ことを目指して設計されている。

# 54

# XML

## 今も現役の「タグで語る」データ形式

### ひとことというと

<tag>で囲んでデータの意味を表現する、開始・終了タグが特徴のマークアップ形式。

1

### なぜ生まれた？

システム間でデータをやり取りする際、値だけでなく意味や階層構造まで一緒に表現できる、共通のマークアップ形式が求められていた。



2

### それまでどうしていた？

JSONが普及する前は、Web APIも設定ファイルもXMLで書くのが当たり前だった。

|        | XML登場前(プレーンテキスト等) | XML                 |
|--------|-------------------|---------------------|
| 構造の表現  | 決まったルールがない        | タグで階層と属性を表現         |
| 検証     | 独自ルールで確認          | スキーマ(DTD/XSD)で検証できる |
| 今の立ち位置 | —                 | 設定・帳票・SOAP系APIで現役   |

3

### XML で何ができるようになった？



タグ名だけで意味が伝わる可読性



スキーマで構造を厳密に検証できる



帳票やドキュメント形式として今も現役

### ひとことメモ

JSONの台頭でAPIの主役の座は譲ったが、厳密な検証が必要な業務システムや文書フォーマットとしては今もXMLが根強い。

## 関連キーワード

比較する技術



### JSON

括弧で構造を表す軽量なデータ形式。今のAPIの主流。

P.51



### スキーマ (Schema)

データの形を定義し、正しいかどうかを検証する仕組み。

P.56

# CSV

Excelとの橋渡し役、カンマ区切りの素朴な形式

## ひとことという

値をカンマ(,)で区切って1行1レコードを表す、最もシンプルな表形式のデータ形式。

## 関連キーワード

比較する技術



### JSON

括弧で構造を表す  
軽量なデータ形  
式。ネスト構造も表  
現できる。

P.51

## 1 なぜ生まれた?

表形式のデータをExcelやスプレッドシートでそのまま開けるようにするには、JSONやXMLのような入れ子構造を持たない、単純な区切り文字形式が必要だった。



## 2 それまでどうしていた?

JSONやXMLのような入れ子構造を表すデータ形式に対して、CSVは表形式のデータをそのまま渡したいときに使われる。

|         | JSON           | CSV                    |
|---------|----------------|------------------------|
| 構造      | 入れ子(ネスト)を表現できる | 1行1レコードの平坦な表のみ         |
| 開きやすさ   | エディタやコードで確認    | Excelやスプレッドシートでそのまま開ける |
| 向いている場面 | APIのレスポンス      | ダウンロードデータ・大量データの受け渡し   |

## 3 CSV で何ができるようになった?



Excelやスプレッドシートでそのまま開ける



大量データの入出力・ダウンロードに使う



仕組みが単純で他システムとも連携しやすい

### ひとことメモ

CSVは「Comma-Separated Values」の略。  
値の中にカンマや改行があると引用符で囲む必要があり、単純に見えて実は細かいルールがある。

# スキーマ (Schema)

「このデータ、形は合ってる？」に答える設計図

## ひとことという

データがどんな項目を持ち、どんな型・制約であるべきかを定義した"データの設計図"。

### 1 なぜ生まれた？

サーバーに送られてくるデータの形式が保証されていないと、必須項目の欠落や型の不一致に気づけず、エラーが起きてから原因を調べることになっていた。データの形をあらかじめ定義しておく仕組みが必要だった。



### 2 それまでどうしていた？

スキーマがない状態では、データの形が正しいかどうかを目視やその場しのぎのif文で確認するしかなかった。

|        | スキーマなし         | スキーマあり         |
|--------|----------------|----------------|
| 形式チェック | 目視やif文で都度確認    | 定義に沿って自動で検証    |
| ドキュメント | コードを読まないと分からない | スキーマ自体が仕様書になる  |
| 変更の影響  | 気づかず壊れることがある   | 違反がすぐエラーとして分かる |

### 3 スキーマ(Schema) で何ができるようになった？



データの形を自動で検証できる



必須項目・型・制約を明文化できる



DBやAPI、設定ファイルなど幅広く使う

## ひとことメモ

スキーマはDBのテーブル定義だけでなく、JSONやXMLの形式チェック(JSON Schema, XSD)、APIの仕様定義(OpenAPI)など、いろいろな場面に登場する言葉。

## 関連キーワード

関連する語



### XML

タグで構造を表現するデータ形式。DTD/XSDというスキーマ機構を持つ。

P.54



### OpenAPI

APIの入出力の形をスキーマとして定義する仕様。

P.65

# API

「画面を見せずに機能だけ貸す」ための窓口

## ひとことという

アプリやサービス同士が、決められたルールに沿ってやり取りするための"窓口"。

### 1 なぜ生まれた？

外部サービスの機能やデータを自分のアプリから利用したいとき、画面をスクレイピングする以外に構造化された取得手段がなかった。



### 2 それまでどうしていた？

外部のサービスが持つ機能やデータを使いたいとき、以前は画面ごと取り込む(スクレイピング)しかなかった。

|        | 画面のスクレイピング    | API            |
|--------|---------------|----------------|
| 取得する対象 | 見た目込みのHTML全体  | 必要なデータだけ       |
| 安定性    | デザイン変更で簡単に壊れる | 仕様が変わらない限り安定   |
| やり取りの形 | 人が読む前提のページ    | プログラムが読む前提のデータ |

### 3 API で何ができるようになった？



他社のサービスの機能をそのまま使える



決まったルールでデータだけをやり取り



フロントとサーバーの間の窓口にもなる

## ひとことメモ

APIは「Application Programming Interface」の略。  
「プログラム同士が会話するための窓口」くらいに考えるとイメージしやすい。

## 関連キーワード

関連する語



### Endpoint

APIの窓口の中の、個別の入り口となるURL。

P.62



### REST

APIの設計スタイルの1つ。URLと動詞で操作を表す。

P.63



### GraphQL

必要なデータだけを1回で取得できるAPIの仕組み。

P.64

# Endpoint

APIという建物の、それぞれの「入り口」

## ひとことというと

APIの中で、特定の機能やデータにアクセスするための個別のURL。

### 1 なぜ生まれた？

APIを1つの窓口としてまとめて捉えると、機能ごとにどこへリクエストを送ればよいか分かりにくく、個々の入り口を明確に区別する必要があった。



### 2 それまでどうしていた？

APIを「1つの大きな窓口」とだけ捉えていると、実際にどこにリクエストを送ればいいのか迷う。

|       | APIをひとまとめに捉える    | Endpointという単位で捉える            |
|-------|------------------|------------------------------|
| 粒度    | サービス全体           | 機能ごとのURL (/users, /posts など) |
| 見つけ方  | ドキュメントを一から読む     | 仕様書で機能とURLが対応している            |
| 変更の影響 | APIが変わったと大雑把に捉える | どのエンドポイントが変わったか特定できる         |

### 3 Endpoint で何ができるようになった？



機能ごとにURLを分けて設計できる



/users や /posts のような単位で管理



ドキュメントやログでも扱いやすい単位

## ひとことメモ

エンドポイントは「APIの中の1つの窓口」というイメージ。  
/api/v1/users のようなURLの、動詞(GET/POST)とセットで1つの操作を表す。

## 関連キーワード

関連する語



### API

アプリ同士がやり取りするための窓口全体。

P.61



### slug

URLの一部に使われる、人にも読める識別子。

P.67



### REST

URLと動詞で操作を表現するAPI設計スタイル。

P.63

# REST

URLと動詞だけで意味を伝えるAPI設計の作法

## ひとことという

URL(名詞)とHTTPメソッド(動詞)の組み合わせで、操作の意味を表現するAPI設計のスタイル。

### 1 なぜ生まれた?

APIの設計者ごとにURLの命名や動詞の使い方がバラバラだと、規模が大きくなるほど一貫性が失われ、他の開発者が理解しづらくなっていた。



### 2 それまでどうしていた?

RESTを意識しない設計では、操作ごとにURLをその都度考えて命名することになり、統一感がなくなりがちだった。

|       | RESTを意識しないAPI                 | REST                              |
|-------|-------------------------------|-----------------------------------|
| URL設計 | /getUser, /deleteUser など動詞混じり | /users のように名詞のみ                   |
| 操作の表現 | URLの中に動詞を書く                   | GET/POST/PUT/DELETE などHTTPメソッドで表現 |
| 一貫性   | 設計者ごとにルールがブレる                 | リソース単位で統一しやすい                     |

### 3 REST で何ができるようになった?



URLをリソース(名詞)だけで設計できる



HTTPメソッドで操作の意味を表現



設計者が変わってもルールがブレにくい

## ひとことメモ

RESTは「REpresentational State Transfer」の略で、厳密には設計思想(アーキテクチャスタイル)の名前。実務では「URLは名詞、操作はHTTPメソッドで」くらいの理解で十分戦える。

## 関連キーワード

比較する技術



### GraphQL

必要なデータだけを1つのエンドポイントで取得できる設計。

P.64

関連する語



### Endpoint

RESTのURL設計の対象になる、個々の入り口。

P.62



### Status Code

REST APIのレスポンスで結果を伝える3桁の数字。

P.66

# GraphQL

「欲しいデータだけちょうだい」を叶えるAPIの仕組み

## ひとことという

欲しい項目だけを指定して、1回のリクエストでまとめて取得できるAPIの問い合わせ言語。

### 1 なぜ生まれた？

画面に必要なデータを揃えるためにREST APIを複数呼び出す実装では、リクエスト回数が増え、過不足のあるデータが返ってくることも多かった。



### 2 それまでどうしていた？

RESTでは、1つのエンドポイントが返すデータの形はあらかじめ固定されている。

|          | REST               | GraphQL         |
|----------|--------------------|-----------------|
| 取得できるデータ | エンドポイントごとに固定       | 欲しい項目だけ自分で指定    |
| リクエスト回数  | 画面に必要な分だけ複数回       | 1回のリクエストでまとめて取得 |
| 過不足      | 使わない項目まで返ってくることもある | 過不足なく必要な分だけ返る   |

### 3 GraphQL で何ができるようになった？



必要な項目だけを指定して取得できる



複数のリソースを1回のリクエストで取得



型(スキーマ)でデータの形が保証される

## ひとことメモ

GraphQLは1つのURL(エンドポイント)に対して、クエリという「欲しいデータの形」を送ることで動く。RESTとどちらが良いというより、画面が必要とするデータの複雑さで使い分けるもの。

## 関連キーワード

比較する技術



### REST

URLと動詞で操作を表現するAPI設計スタイル。

P.63

関連する語



### スキーマ (Schema)

GraphQLでもデータの形を定義するために使われる。

P.56

# OpenAPI

「口頭で伝えていたAPI仕様」をファイルにした約束事

## ひとことという

APIのエンドポイントや入出力の形を、YAMLやJSONで記述する仕様のフォーマット。

## 関連キーワード

関連する語



### API

OpenAPIはこのAPIの仕様を記述するためのフォーマット。

P.61



### スキーマ (Schema)

OpenAPIの中でリクエスト・レスポンスの形を定義する仕組み。

P.56

## 1 なぜ生まれた？

API仕様が文書化されていない現場では、フロントエンドとバックエンドの間でレスポンス形式のすり合わせを、その都度コードを読んで行う必要があった。



## 2 それまでどうしていた？

OpenAPIがない現場では、API仕様はドキュメントに手書きするか、コードを読んで確認するしかなかった。

|         | 手書きの仕様書・口頭確認       | OpenAPI                |
|---------|--------------------|------------------------|
| 仕様の形    | Wordやスプレッドシートに個別記述 | 決まったフォーマット (YAML/JSON) |
| 実装との整合  | 更新を忘れて仕様と実装がズれる    | ツールで検証・モックまで生成できる      |
| 共有のしやすさ | ファイルを都度共有          | Swagger UIなどでそのまま閲覧できる |

## 3 OpenAPI で何ができるようになった？



API仕様を1つのファイルにまとめられる



Swagger UIなどでブラウザから確認できる



モックサーバーやクライアントコードを自動生成

## ひとことメモ

OpenAPIは元々「Swagger」という名前のツールから始まった仕様。APIの仕様書をコードと同じように「読める形式」で管理できるのが最大のメリット。

# Status Code

サーバーが返す「結果の一言」を数字にしたもの

## ひとことという

リクエストの結果が成功か失敗か、失敗ならどんな理由かを伝える3桁の数字。

### 1 なぜ生まれた？

エラーの内容をレスポンス本文の文章だけで判断していると、成功・失敗の区別や原因の特定に毎回時間がかかっていた。



### 2 それまでどうしていた？

ステータスコードを見ない・意識しない実装では、エラーの原因を毎回レスポンスの中身の文章だけから推測するしかない。

|          | 本文のメッセージだけで判断 | Status Codeで判断            |
|----------|---------------|---------------------------|
| 成功／失敗の判定 | 文章を読んで解釈      | 200番台か400/500番台かで機械的に判定   |
| 原因の切り分け  | 都度サーバーログを確認   | 404=見つからない、401=未認証など型で分かる |
| 実装のしやすさ  | 文字列を都度パースして分岐 | ステータスコードで分岐処理を書ける         |

### 3 Status Code で何ができるようになった？



成功／失敗を数字だけで機械的に判定



404や500など原因の見当がつく



フロント側で分岐処理を書きやすい

## ひとことメモ

200番台は成功、300番台はリダイレクト、400番台はリクエスト側のミス、500番台はサーバー側の問題、とざっくり覚えておくだけでも十分役に立つ。

## 関連キーワード

関連する語



### REST

Status CodeはREST APIのレスポンスで結果を伝える手段。

P.63

# slug

URLの中に生きる、人が読める識別子

## ひとことというと

記事やページを識別するために使う、URLに埋め込める短い文字列(例: /posts/how-to-use-docker)。

## 関連キーワード

関連する語



### Endpoint

slugはエンドポイントのURLの一部として使われることが多い。

P.62

## 1 なぜ生まれた？

記事やページのURLを数字のIDだけで構成すると、人間にもクローラーにも内容が伝わらず、検索結果での見え方にも不利だった。



## 2 それまでどうしていた？

IDだけをURLに使っていると、人間にもクローラーにもそのページの中身が伝わらない。

|           | IDだけのURL    | slugを使ったURL              |
|-----------|-------------|--------------------------|
| 見た目       | /posts/1024 | /posts/how-to-use-docker |
| 人への伝わりやすさ | 中身が推測できない   | タイトルの内容が一目で分かる           |
| SEOへの影響   | キーワードが含まれない | URLにもキーワードが含まれる          |

## 3 slug で何ができるようになった？



URLに人が読める文字列を埋め込める



Endpointの一部として識別子に使える



SEOやシェア時の見た目が改善する

### ひとことメモ

slugは基本的に「小文字・半角英数・ハイフン区切り」で作るのが定番。日本語タイトルをそのままslugにするとURLが読みにくくなるため、英訳やローマ字化して使うことが多い。

# 71

# gRPC

サーバー同士の会話をHTTPより速く・厳密にする通信方式

## ひとことというと

Googleが開発した、サーバー同士の内部通信を高速かつ型安全に行うための通信の仕組み。

## 1 なぜ生まれた？

マイクロサービス間の通信でJSON形式のREST APIを多用すると、サービス数が増えるほど通信量・処理コストが積み重なり、パフォーマンスが課題になっていた。



## 2 それまでどうしていた？

これまでのHTTPリクエスト/レスポンス型の通信(Ch2で扱ったもの)は、人が読めるテキスト(JSON等)でやり取りするため分かりやすい反面、データ量や速度の面で不利になることがあった。

|         | HTTP + JSON     | gRPC                     |
|---------|-----------------|--------------------------|
| データ形式   | 人が読めるテキスト(JSON) | バイナリ形式(Protocol Buffers) |
| 速度・データ量 | 比較的大きく遅い        | 軽量で高速                    |
| 向いている場面 | ブラウザとのやり取り      | サーバー同士の内部通信              |

## 3 gRPC で何ができるようになった？



バイナリ形式で高速に通信できる



マイクロサービス間の通信に向いている



スキーマ(型定義)から複数言語のコードを生成

## ひとことメモ

gRPCはブラウザからそのまま呼ぶのには向いておらず、主にサーバー同士・マイクロサービス間の通信で使われる。人に見せる画面のAPIはREST/GraphQL、社内の裏側はgRPC、という使い分けもよくある。

## 関連キーワード

関連する語



### REST

URLと動詞で操作を表現する、人にも読みやすいAPI設計。

P.63



### スキーマ(Schema)

gRPCも通信するデータの形をスキーマとして定義する。

P.56

# WebSocket

「聞かれるまで話せない」HTTPの壁を壊した通信方式

## ひとことという

一度接続を確立したら、サーバーとクライアントが双方向にいつでもデータを送り合える通信の仕組み。

## 関連キーワード

関連する語



### Pub/Sub

変化があった相手に自動で知らせる、より広い仕組み。

P.74



### Webhook

こちらでイベント発生時に通知する仕組みだが、通信の形が異なる。

P.73

## 1 なぜ生まれた？

チャットや通知のようにリアルタイム性が求められる機能を通常のHTTPリクエストで実現しようとすると、一定間隔でAPIを呼び続ける(ポーリング)必要があり、サーバー負荷が増大していた。



## 2 それまでどうしていた？

通常のHTTPリクエスト/レスポンス(Ch2)は、クライアントが聞かない限りサーバーからは何も送られてこない。

|         | HTTPのポーリング        | WebSocket       |
|---------|-------------------|-----------------|
| 通信の方向   | クライアントから聞きに行く一方通行 | 接続後は双方向にいつでも送れる |
| リアルタイム性 | 一定間隔でしか更新されない     | 変化があった瞬間に届く     |
| 向いている場面 | 通常のページ表示          | チャット・通知・対戦ゲームなど |

## 3 WebSocket で何ができるようになった？



接続したまま双方向にやり取りできる



リアルタイムに近い速さで更新が届く



ポーリングに比べてサーバー負荷を抑えられる

## ひとことメモ

WebSocketは最初にHTTPで接続してから、そのまま専用の通信路に"アップグレード"する仕組み。チャットや通知、株価のようなリアルタイム更新が必要な画面でよく使われる。

# 73

# Webhook

「向こうから電話がかかってくる」通知の仕組み

## ひとことという

サービス側で何かが起きたときに、登録しておいた URL へ自動でリクエストを送ってくれる仕組み。

### 1 なぜ生まれた？

外部サービスの状態変化（決済完了など）を検知するために定期的に API を呼び続けると、確認間隔ぶんのタイムラグが生じ、サーバー負荷も無駄にかかっていた。



### 2 それまでどうしていた？

Webhook がない場合、外部サービスの状態変化を知るには、こちらから定期的に API を叩いて確認（ポーリング）するしかない。

|       | 定期的なポーリング       | Webhook               |
|-------|-----------------|-----------------------|
| 確認の主体 | 自分から定期的に聞きに行く   | 相手が変わった瞬間に知らせてくれる     |
| タイムラグ | ポーリングの間隔ぶん遅れる   | ほぼリアルタイムに届く           |
| 実装の準備 | API を叩くコードだけでよい | 通知を受け取る URL（受け口）を用意する |

### 3 Webhook で何ができるようになった？



外部サービスからの通知を受け取れる



ポーリングよりリアルタイムに近い



決済・デプロイ通知など幅広く使われる

## ひとことメモ

Webhook は「登録した URL に向かって、相手が POST リクエストを送ってくる」だけの単純な仕組み。GitHub の Push 通知や Stripe の決済結果通知などで馴染み深い。

## 関連キーワード

関連する語



### Pub/Sub

Webhook と同じく通知の仕組みだが、間に仲介役を挟む点が異なる。

P.74



### Endpoint

Webhook の通知を受け取るための窓口として自分の API に用意する。

P.62

# 74

# Pub/Sub

送り手と受け手を「知り合わせない」通知の仕組み

### ひとことというと

発行者 (Publisher) がメッセージを送り、購読者 (Subscriber) が仲介役を通じて受け取る非同期の通信モデル。

## 1 なぜ生まれた？

1つのイベント(注文など)を複数の送信先(在庫システム・通知システムなど)に連携する際、送信元のコードに宛先を直接書き込んでいくと、宛先が増えるたびに送信元の修正が必要になっていた。



## 2 それまでどうしていた？

Webhook のように送信元が受信先を直接知っている構成では、通知先が増えるたびに送信元の実装を修正する必要がある。

|        | Webhook で直接送る       | Pub/Sub             |
|--------|---------------------|---------------------|
| 送信先の把握 | 送信元が受信先の URL を知っている | 送信元は仲介役(トピック)に送るだけ  |
| 受信先の追加 | 送信元のコードを修正して追加      | 購読するだけで新しい受信先を追加できる |
| 関係性    | 送信元と受信先が密結合         | 間にトピックを挟んで疎結合       |

## 3 Pub/Sub で何ができるようになった？



送信元と受信先を直接つなぐずに済む



受信先を後から自由に追加できる



複数のサービスへの同時通知に向いている

### ひとことメモ

Pub/Sub は「Publish(発行) / Subscribe(購読)」の略。  
Webhook が 1 対 1 の電話だとすると、Pub/Sub は 1 つの掲示板(トピック)に貼り出して、見たい人が見に来るイメージ。

## 関連キーワード

関連する語



### Webhook

こちらは送信元が受信先を直接知っている、1 対 1 に近い通知方式。

P.73



### WebSocket

こちらは接続を保ったまま双方向にやり取りする通信方式。

P.72

# 81

# Database

データを「ファイルの海」から救い出した仕組み

## ひとことという

データを一箇所にまとめて、複数人で安全に読み書きでき、検索しやすく保存する仕組み。

## 1 なぜ生まれた？

顧客データやアプリの状態をファイルで直接管理すると、複数人が同時に編集した際にデータが上書きされて消失する危険があった。



## 2 それまでどうしていた？

ファイルに直接データを書き込んで、そのまま保存・共有していた。

|        | ファイル保存      | データベース            |
|--------|-------------|-------------------|
| 同時アクセス | 上書きされて壊れやすい | 複数人が同時に安全に読み書きできる |
| 検索     | 全部読んで目で探す   | 条件を指定して一瞬で検索できる   |
| 整合性    | 壊れても気づきにくい  | ルールで壊れたデータを弾ける    |

## 3 Database で何ができるようになった？



大量データも高速に検索できる



複数人で同時に読み書きできる



ルールでデータの整合性を守れる

## ひとことメモ

データベースは「データの保管庫」であり「番人」でもある。保存するだけでなく、壊れないように守ってくれる。

## 関連キーワード

データベースの種類



### RDB

表同士を関係で結びつけて管理する方式。

P.82



### NoSQL

表の形にこだわらない柔軟なデータベース。

P.84

構成する要素



### Table

データを行と列で整理する表そのもの。

P.85

# RDB

表と表の「関係」でデータを整理する考え方

## ひとことという

行と列を持つ「表(テーブル)」同士を関連付けてデータを管理する方式。

### 1 なぜ生まれた?

1つの巨大な表にあらゆる情報を詰め込んで管理すると、同じ値を繰り返し入力することになり、更新漏れやデータの不整合が起きやすかった。



### 2 それまでどうしていた?

1つの巨大な表に、あらゆる情報を詰め込んで管理していた。

|        | 1つの表         | RDB           |
|--------|--------------|---------------|
| データの重複 | 同じ情報を何度も書く   | 表を分けて重複を減らせる  |
| 更新     | 複数箇所を直す必要あり  | 1箇所直せば済む      |
| 整合性    | 矛盾したデータが混在する | 関係のルールで矛盾を防げる |

### 3 RDB で何ができるようになった?



表を分けて重複を減らせる



関係のルールでデータを守れる



複雑な業務データも整理できる

## ひとことメモ

RDBは「表を分けて、関係でつなぐ」発想。分けた分だけ、整合性を保つ仕組みが必要になる。

## 関連キーワード

実装ソフトと対になる考え方



### RDBMS

RDBの考え方を実装したソフトウェア。

P.83



### NoSQL

表の形にこだわらない別のアプローチ。

P.84

関連する要素



### Table

行と列でデータを整理する表。

P.85



### Foreign Key

表同士の関係を表すキー。

P.88

# RDBMS

RDBの考え方を実際に動かすソフトウェア

## ひとことという

RDB(関係データベース)の考え方を実装し、実際にデータを保存・操作できるようにするソフトウェア。

### 1 なぜ生まれた？

RDBという設計上の考え方だけでは、実際にデータを保存・検索・更新する具体的な処理を実行できなかった。



### 2 それまでどうしていた？

RDBの仕組みを、必要になるたびに自分でゼロから実装しようとしていた。

|     | 自前で実装          | RDBMS                   |
|-----|----------------|-------------------------|
| 信頼性 | 自分でバグと戦うことになる  | 実績あるソフトウェアが保証してくれる      |
| 機能  | 検索やトランザクションを自作 | SQLや排他制御が最初から使える        |
| 選択肢 | ゼロから設計するしかない   | MySQLやPostgreSQLなどから選べる |

### 3 RDBMS で何ができるようになった？



RDBの仕組みをすぐ使い始められる



トランザクションや制約を任せられる



クラウドサービスとしても使える

## ひとことメモ

RDBは考え方、RDBMSはそれを動かす実物。MySQLもPostgreSQLも、全部「RDBMSの一種」。

## 関連キーワード

関連する考え方



### RDB

表同士を関係で結びつけるデータの考え方。

P.82

操作する言語



### SQL

RDBMSに指示を出すための共通言語。

P.90

代表的な製品



### MySQL

世界で広く使われるRDBMS製品。



### PostgreSQL

機能が豊富なRDBMS製品。

# NoSQL

「表」の形にこだわらないデータベースの総称

## ひとことというと

テーブルの形に縛られず、柔軟な形でデータを保存できるデータベースの総称。

### 1 なぜ生まれた？

サービスの急成長で、ユーザーごとに項目が異なるようなデータをRDBの固定された表構造に収めようとすると、カラムが増え続けて設計が破綻しかなかった。



### 2 それまでどうしていた？

RDB(表構造)だけを使って、あらゆる種類のデータを管理しようとしていた。

|         | RDBのみ          | NoSQL            |
|---------|----------------|------------------|
| データの形   | 事前に列を固定する必要がある | 形が違うデータも自由に保存できる |
| スケール    | 1台のサーバーで頑張りがち  | 複数台に分散しやすい       |
| 向いている場面 | 整合性重視の業務データ    | ログ・キャッシュなど柔軟なデータ |

### 3 NoSQL で何ができるようになった？



形の違うデータも柔軟に保存できる



大量データを分散して扱える



読み書きを高速化しやすい

## ひとことメモ

NoSQLは「RDBの代わり」ではなく「使い分ける相棒」。データの性質に合わせて選べばいい。

## 関連キーワード

対になる考え方



### RDB

表同士を関係で結びつけるデータの考え方。

P.82



### RDBMS

RDBを実装したソフトウェア。

P.83

代表的な製品



### Redis

高速なキー・バリュ型データベース。



### MongoDB

柔軟なドキュメント型データベース。

# 85

# Table

データを行と列で整理する「表」そのもの

## ひとことという

データを行(レコード)と列(カラム)に整理して格納する、データベースの基本単位。

### 1 なぜ生まれた?

データを決まった形式のないテキストや配列にそのまま書き込むと、後から検索や更新を正確に行うことができなかった。



### 2 それまでどうしていた?

テキストや配列に、決まった形もなくデータを書き並べていた。

|    | 形が不定         | Table             |
|----|--------------|-------------------|
| 構造 | 毎回バラバラな形式になる | 列でルールが決まっている      |
| 検索 | どこに何があるか探す   | 列名で欲しいデータを取り出せる   |
| 追加 | 書式がブレやすい     | 同じ形式のレコードとして追加できる |

### 3 Table で何ができるようになった?



同じ形式でデータを並べられる



列名で欲しいデータをすぐ取り出せる



型やルールでデータの形を保てる

## ひとことメモ

テーブルは「エクセルの表」に近いイメージ。列(カラム)でルールを決め、行(レコード)でデータを積む。

## 関連キーワード

構成する要素



### Record

テーブルの1行分のデータ。

P.86



### Primary Key

1件を一意に特定する値。

P.87



### Foreign Key

他の表とつながりを持たせるキー。

P.88

# Record

テーブルに積まれる「1 件分」のデータ

## ひとことというと

テーブルの中の 1 行分、つまり「1 件分」としてまとめたデータ。

## 関連キーワード

関連する要素



### Table

レコードが積まれる表そのもの。

P.85



### Primary Key

レコードを一意に特定する値。

P.87

## 1 なぜ生まれた？

データを項目ごとにバラバラに扱うと、どこからどこまでが「1 件分」なのかが曖昧になり、一部の項目だけを更新して不整合を起こしやすかった。



## 2 それまでどうしていた？

データの単位を意識せず、項目ごとにバラバラに扱っていた。

|       | 項目バラバラ         | Record            |
|-------|----------------|-------------------|
| 単位    | 項目だけを見て操作する    | 1 件分をまとめて扱える      |
| 追加・削除 | 一部の項目だけ更新しがち   | 1 行まるごと追加・削除できる   |
| 識別    | 「誰のデータか」が曖昧になる | 主キーで 1 件を確実に特定できる |

## 3 Record で何ができるようになった？



1 件分のデータをまとめて扱える



行単位で追加・更新・削除できる



主キーで 1 件を確実に特定できる

## ひとことメモ

レコードは「テーブルという表の中の 1 行」。  
Excel でいう「1 行分のデータ」と同じ感覚。

87

# Primary Key

「この行は絶対にこれ」と言い切るための印

## ひとことという

テーブルの中で1件のレコードを一意に特定するための、重複しない値。

### 1 なぜ生まれた？

名前やメールアドレスなど意味を持つ項目でレコードを特定すると、同じ値を持つ別のレコードまで誤って操作してしまう恐れがあった。



### 2 それまでどうしていた？

名前やメールアドレスなど、意味のある項目を頼りにレコードを特定していた。

|     | 名前で特定         | Primary Key      |
|-----|---------------|------------------|
| 一意性 | 同じ値の人が現れうる    | 絶対に重複しない値を保証する   |
| 変更  | 名前変更で特定方法が崩れる | 値が変わらないIDで安定して参照 |
| 事故  | 別人のデータを誤更新しうる | 常に狙った1件だけを操作できる  |

### 3 Primary Key で何ができるようになった？



常に1件だけを正確に特定できる



重複や誤更新を防げる



他の表からも安全に参照できる

## ひとことメモ

主キーは「この1件」を指し示す絶対的な住所。  
名前やメールではなく、専用のIDで管理するのが安全。

## 関連キーワード

関連する要素



### Record

主キーで特定される1件分のデータ。

P.86



### Foreign Key

主キーを他の表から参照するキー。

P.88

# Foreign Key

親を消したら子供が迷子になる、を防ぐ仕組み

### ひとことという

別のテーブルの主キーを参照して、テーブル同士の関係を表す値。

## 関連キーワード

関連する要素



### Table

キーで関係づけられる表同士。

P.85



### Primary Key

外部キーが参照する側のキー。

P.87

## 1 なぜ生まれた？

テーブル同士のつながりをアプリ側のコードだけで管理すると、親データを削除した際に、存在しない親を指す壊れた子データが残ってしまう恐れがあった。



## 2 それまでどうしていた？

テーブル同士のつながりを、アプリ側のコードだけで気をつけて管理していた。

|        | アプリ側で担保          | Foreign Key       |
|--------|------------------|-------------------|
| 親の削除   | 子データが孤児として残る     | DBが削除を止める／連動して消せる |
| 不正なデータ | 存在しない親を指すデータが作れる | 存在する親しか指せない       |
| 依存関係   | コードを読まないと分からない   | テーブル定義を見れば分かる     |

## 3 Foreign Key で何ができるようになった？



表同士のつながりをDBが保証する



親を消すときの挙動を制御できる



存在しないデータへの参照を防げる

### ひとことメモ

外部キーは「親と子の絆を守る」仕組み。  
親を消すときに子がどうなるかを、DBに約束させられる。

# Index

本の「索引」と同じ、検索を速くする仕組み

## ひとことというと

特定の列の検索を高速化するために作る、データの「索引」。

### 1 なぜ生まれた？

索引がない状態でテーブルを検索すると、データ件数が増えるほど全件を読み比べる処理コストが増大し、検索が遅くなっていった。



### 2 それまでどうしていた？

索引がない状態で、テーブルを頭から全部読んで探していた。

|      | インデックスなし      | Index           |
|------|---------------|-----------------|
| 検索方法 | 全件を1行ずつ確認する   | 索引から一気に絞り込む     |
| 速度   | データが増えるほど遅くなる | 大量データでも高速なまま    |
| コスト  | 検索は速いが書き込みは軽い | 書き込みや保存容量は少し増える |

### 3 Index で何ができるようになった？



大量データでも検索が高速なまま



待たされるページが減る



速さと更新コストのバランスを取れる

## ひとことメモ

インデックスは「本の索引」と同じ発想。  
全部読まなくても、目的のページに一直線で行ける。

## 関連キーワード

関連する要素



### Table

インデックスが張られる表。

P.85



### Primary Key

通常は自動でインデックスが張られる。

P.87

# SQL

データベースに話しかけるための共通言語

## ひとことという

データベースに対して、検索・追加・更新・削除を指示するための言語。

## 関連キーワード

関連する要素



### RDBMS

SQLで操作する対象のソフトウェア。

P.83



### ORM

SQLを直接書かずに操作する仕組み。

P.91

## 1 なぜ生まれた？

データベース製品ごとに操作方法が異なると、製品を変更するたびに操作方法を一から覚え直す必要があった。



## 2 それまでどうしていた？

各データベース製品が、それぞれ独自の操作方法を持っていた。

|       | 製品ごとに独自        | SQL                    |
|-------|----------------|------------------------|
| 学習コスト | 製品ごとに覚え直す      | 一度覚えれば多くの製品で通用         |
| 書き方   | 製品依存でバラバラになる   | SELECT / INSERT など共通構文 |
| 移行    | 製品を変えたと書き直しになる | 基本構文はそのまま使い回せる         |

## 3 SQL で何ができるようになった？



共通の書き方でデータを操作できる



条件を指定して正確に絞り込める



多くのDB製品で同じ知識が使える

### ひとことメモ

SQLは「データベースの共通語」。製品が変わっても、基本の文法はほぼそのまま通用する。

# ORM

SQLを書かずにデータベースを操作する翻訳者

## ひとことという

プログラムのオブジェクトとテーブルを対応させ、SQLを直接書かずに操作できるようにする仕組み。

1

## なぜ生まれた？

SQL文を文字列として自分で組み立てると、値の埋め込み方を誤った際にSQLインジェクションの脆弱性を作り込んでしまう恐れがあった。



2

## それまでどうしていた？

SQL文を自分で組み立てて、文字列としてそのまま実行していた。

|     | SQL直書き           | ORM                |
|-----|------------------|--------------------|
| 書き方 | SQL文を文字列で組み立てる   | コードのオブジェクトとして操作    |
| 安全性 | 書き方次第で脆弱性を作りうる   | 安全な形に自動で変換してくれる    |
| 可搬性 | DB製品ごとにSQLを書き分ける | 同じコードで複数のDBに対応しやすい |

3

## ORMで何ができるようになった？



SQLを意識せずデータ操作できる



安全な形にSQLを組み立ててくれる



コードの記述量を減らせる

## ひとことメモ

ORMは「コードとSQLの通訳」。  
楽になる分、裏で何のSQLが実行されているかは知っておきたい。

## 関連キーワード

関連する要素



### SQL

ORMが裏側で組み立てている言語。

P.90



### RDBMS

ORMが最終的に操作するソフトウェア。

P.83

# 92

# Migration

DBの設計変更を「手作業SQL」から解放した仕組み

## ひとことという

テーブルの追加・変更などのDB設計変更を、コードの形で記録・管理する仕組み。

### 1 なぜ生まれた？

本番環境に手作業でALTER TABLEなどのSQLを実行すると、環境ごとにカラムの型や状態がずれ、リリース後にエラーが発生しやすかった。



### 2 それまでどうしていた？

テーブル変更のSQLを、環境ごとに手作業で実行していた。

|     | 手作業でSQL       | Migration          |
|-----|---------------|--------------------|
| 再現性 | 環境ごとに結果がズレうる  | 同じ変更を全環境に確実に反映     |
| 履歴  | いつ何を変えたか分からない | 変更履歴がコードとして残る      |
| 共有  | 口頭やメモで伝達しがち   | コードをチームで共有・レビューできる |

### 3 Migration で何ができるようになった？



変更履歴を時系列で管理できる



全環境に同じ変更を確実に反映できる



変更をコードとしてレビューできる

## ひとことメモ

マイグレーションは「DB版のバージョン管理」。手でSQLを打つ前に、まずマイグレーションを疑う。

## 関連キーワード

一緒に使う仕組み



### Seeder

確認用データを自動投入する仕組み。

P.93

関連する要素



### RDBMS

マイグレーションで変更する対象。

P.83

# Seeder

「動作確認用のデータ」を毎回手打ちしないでいい仕組み

## ひとことという

開発・テスト用のダミーデータを、コードで自動的に投入する仕組み。

### 1 なぜ生まれた？

動作確認用のデータを管理画面やSQLで毎回手動で登録すると、環境を作り直すたびに同じ作業を繰り返す必要があった。



### 2 それまでどうしていた？

確認用データを、管理画面やSQLで毎回手動で入力していた。

|     | 手動入力           | Seeder           |
|-----|----------------|------------------|
| 再現性 | 毎回微妙に違うデータになる  | 同じデータをいつでも再現できる  |
| 手間  | 環境ごとに手で入力する    | コマンド1つで一括投入できる   |
| 共有  | テストデータが人によって違う | チームで同じテストデータを使える |

### 3 Seeder で何ができるようになった？



確認用データを一括で投入できる



環境構築の時間を短縮できる



チームで同じデータを共有できる

## ひとことメモ

シーダーは「テストデータの型紙」。  
毎回手で作る代わりに、コードに一度書けば使い回せる。

## 関連キーワード

一緒に使う仕組み



### Migration

テーブルの構造そのものを変更する仕組み。

P.92

# Transaction

「途中でやめる」ができない一連の処理をまとめる仕組み

### ひとことという

複数の処理を「全部成功」か「全部やり直し」のどちらかにまとめる仕組み。

## 1 なぜ生まれた？

複数の処理を1つずつ順番に実行すると、途中でサーバーが落ちた際に、一部の処理だけが反映された中途半端な状態が残ってしまう恐れがあった。



## 2 それまでどうしていた？

複数の処理を、1つずつ順番に実行して結果を信じていた。

|      | 個別に実行           | Transaction      |
|------|-----------------|------------------|
| 途中失敗 | 中途半端な状態が残る      | 全部なかったことできる      |
| 一貫性  | 一部だけ成功した状態になりうる | 全部成功するまでは反映されない  |
| 対象   | 1件ずつ処理する        | 複数の処理をひとまとまりで扱える |

## 3 Transaction で何ができるようになった？



全部成功か全部失敗かにできる



処理の途中で落ちても安全



複数処理を1つの単位で扱える

### ひとことメモ

トランザクションは「オールオアナッシング」の約束。中途半端な状態を、DBが絶対に見せない。

## 関連キーワード

関連する仕組み



### Lock

同時アクセスの衝突を防ぐ仕組み。

P.95

関連する要素



### RDBMS

トランザクションを保証するソフトウェア。

P.83

# Lock

「同時に触ると壊れる」を防ぐ、データの鍵

## ひとことという

複数の処理が同じデータに同時にアクセスしたときの衝突を防ぐ仕組み。

### 1 なぜ生まれた？

同時アクセスを考慮せずに処理すると、複数の処理が同じデータへほぼ同時にアクセスした際、在庫チェックのようなチェックをすり抜けて不整合が生じる恐れがあった。



### 2 それまでどうしていた？

同時アクセスを考えず、処理は順番に実行されるものだと思い込んでいた。

|        | ロックなし            | Lock           |
|--------|------------------|----------------|
| 同時アクセス | 同じデータを同時に書き換えられる | 一方が終わるまで待たせられる |
| 整合性    | 在庫や残高の数が合わない     | 1件ずつ確実に処理される   |
| 代償     | 事故が起きるまで気づきにくい   | 待ち時間や順番待ちが発生する |

### 3 Lock で何ができるようになった？



同時に書き換えられるのを防げる



在庫や残高の整合性を守れる



処理の順番を保証できる

## ひとことメモ

ロックは「今使ってます」の札。  
札がある間、他の人は順番待ちになる。

## 関連キーワード

関連する仕組み



### Transaction

複数の処理を1つの単位にまとめる仕組み。

P.94

ロックの種類



### 行ロック

更新中の1行だけをロックする方式。

P.96



### テーブルロック

テーブル全体をロックする方式。

P.97



### デッドロック

ロック同士が待ち合って止まる状態。

P.98

# 行ロック

「その1行だけ」を貸し切るロック

## ひとことというと

更新中の1行(レコード)だけをロックし、他の行はそのまま操作できるようにする仕組み。

### 1 なぜ生まれた？

1件の更新のためにテーブル全体をロックすると、無関係な行への処理まで待たされ、システム全体の応答が遅くなっていった。



### 2 それまでどうしていた？

テーブル全体をロックして、1件の更新のために全部を止めていた。

|         | テーブルロック     | 行ロック         |
|---------|-------------|--------------|
| ロック範囲   | テーブル全体      | 更新中の行だけ      |
| 他の処理    | 無関係の行も待たされる | 別の行は同時に処理できる |
| 向いている場面 | 一括で大量更新するとき | 個別の注文・更新処理   |

### 3 行ロックで何ができるようになった？



該当の1行だけを  
貸し切る



他の行の処理は止  
めずに進められる



必要最小限の範囲  
だけ制御できる

## ひとことメモ

行ロックは「その1行だけ」に鍵をかける発想。  
無関係な行まで巻き込まない分、待ち時間が短くて済む。

## 関連キーワード

関連する仕組み



### Lock

同時アクセスの衝突を防ぐ仕組み全般。

P.95



### テーブルロック

表全体を対象にする粗いロック。

P.97

# テーブルロック

表まるごとを貸し切る、強力だが荒っぽいロック

**ひとことという**  
テーブル全体をロックし、他の処理からの読み書きを丸ごと止める仕組み。

## 1 なぜ生まれた？

大量データを一括更新する際に行ロックだけで対応すると、更新対象の行が多いほどロックの管理コスト自体が処理を遅くしてしまっていた。



## 2 それまでどうしていた？

行ロックだけで、大量データの一括更新も乗り切ろうとしていた。

|          | 行ロックのみ         | テーブルロック       |
|----------|----------------|---------------|
| 対象       | 更新する行の数だけロック管理 | テーブル単位でまとめて制御 |
| 向いている場面  | 個別の更新処理        | 一括更新やメンテナンス作業 |
| 他の処理への影響 | 関係ない行は動ける      | テーブル全体が使えなくなる |

## 3 テーブルロックで何ができるようになった？



テーブル全体をまとめて制御できる



大量データの一括更新に強い



用途に応じてロックの粒度を選べる

### ひとことメモ

テーブルロックは「表まるごと貸し切り」。強力な分、使いどころを選ばないと待ち行列を作ってしまう。

## 関連キーワード

関連する仕組み



### 行ロック

該当する行だけを対象にする細かいロック。

P.96



### Lock

同時アクセスの衝突を防ぐ仕組み全般。

P.95

# デッドロック

お互いが相手の鍵を待ち続けて、永遠に進めなくなる状態

## ひとことというと

複数の処理が互いに相手のロック解放を待ち合って、どちらも先に進めなくなる状態。

### 1 なぜ生まれた？

複数の処理がそれぞれ異なる順序でロックを取得すると、互いに相手のロック解放を待ち合い、両方の処理が先に進めなくなる恐れがあった。



### 2 それまでどうしていた？

ロックの取得順序を意識せず、処理ごとに好きな順番でロックを取っていた。

|       | 順序バラバラ       | デッドロック対策       |
|-------|--------------|----------------|
| ロック順序 | 処理ごとにバラバラ    | 常に同じ順番で取得する    |
| 発生時   | お互い待ち続けてフリーズ | DBが検知して片方を強制終了 |
| 対策    | 気づかず放置しがち    | 順序統一やタイムアウトで予防 |

### 3 デッドロックにどう対応できる？



循環待ちの状態だと理解できる



タイムアウトで強制的に解消できる



ロック順序を揃えて予防できる

## ひとことメモ

デッドロックは「お見合い状態」。どちらかが折れない限り、待ち続けても解決しない。

## 関連キーワード

関連する仕組み



### Lock

同時アクセスの衝突を防ぐ仕組み全般。

P.95



### 行ロック

デッドロックが起きうる細かいロック単位。

P.96



### テーブルロック

デッドロックが起きうる粗いロック単位。

P.97

## ループの中でクエリを撃ちまくってしまう大罪

### ひとことという

一覧表示のループの中で関連データを都度取得してしまい、件数分だけSQLが増えてしまう問題。

### 1 なぜ生まれた？

一覧画面でループの中から関連データを都度呼び出す実装では、表示件数と同じ回数だけSQLが発行され、件数が増えるほど処理が重くなっていた。



### 2 それまでどうしていた？

気づかないうちは、ループの中で関連データを呼び出すコードを普通に書いていた。

|           | 気づかず書いたコード    | eager loadingで対策 |
|-----------|---------------|------------------|
| 発行されるクエリ数 | 1+N回          | 1~2回             |
| 表示速度      | 件数に比例して遅くなる   | 件数が増えてもほぼ一定      |
| 気づき方      | 本番で遅くなって発覚しがち | gemやログで事前に検出できる  |

### 3 N+1 で何ができるようになった？



一覧の表示が速くなる



件数が増えても劣化しにくい



クエリ数を意識する癖がつく

### ひとことメモ

N+1は「ループ1回につきクエリ1回」が積み重なった結果。件数が10倍になればクエリも10倍、だから一覧画面ほど痛手が大きい。

## 関連キーワード

関連するキーワード



### Cache

同じ問い合わせを何度もしないための一時保管。

P.100



### コネクションプール

クエリが増えると接続数の上限にも響いてくる。

P.115

# Cache

同じ答えを何度も計算しないための一時保管

## ひとことという

一度取得・計算した結果を保存しておき、次回以降はそれを使い回して高速化する仕組み。

## 関連キーワード

関連するキーワード



### Redis

メモリ上で動く、キャッシュの置き場所として定番。

P.108



### N+1

ループ内クエリが多発も、キャッシュで緩和できることがある。

P.99

## 1 なぜ生まれた？

同じ集計や問い合わせをアクセスのたびにDBへ直接投げ直すと、内容が変わっていても毎回同じ処理コストがかかっていた。



## 2 それまでどうしていた？

キャッシュを知る前は、表示のたびに毎回DBへ直接問い合わせていた。

|        | 毎回DBに問い合わせ  | キャッシュを使う       |
|--------|-------------|----------------|
| 応答時間   | 毎回の処理時間がかかる | 2回目以降はほぼ即返る    |
| DBへの負荷 | 常にフルで問い合わせる | 負荷を大きく減らせる     |
| データの鮮度 | 常に最新        | 古い値を返す期間が発生しうる |

## 3 Cache で何ができるようになった？



応答が速くなる



DB への負荷を減らせる



速さと鮮度のトレードオフを学べる

## ひとことメモ

キャッシュは「賞味期限付きのメモ」。  
便利だけど、いつ・どう消すか(キャッシュ無効化)を決めておかないと、古いデータを返し続ける事故になる。

# 正規化

同じデータをあちこちに置かないための設計ルール

## ひとことという

データの重複を減らすために、テーブルを目的ごとに分割して整理する設計手法。

1

## なぜ生まれた？

1つのテーブルに同じ情報を繰り返し書き込むと、一部だけ更新を反映し忘れて、同じ対象なのにデータが食い違ってしまふ恐れがあった。



2

## それまでどうしていた？

正規化を知らないうちは、1つの大きなテーブルに情報を全部詰め込んでいた。

|         | 正規化前(1テーブルに詰め込み) | 正規化後(テーブルを分割)  |
|---------|------------------|----------------|
| データの重複  | 同じ情報を何度も書く       | 重複を減らせる        |
| 更新のしやすさ | 直し漏れが起きやすい       | 1箇所直せば反映される    |
| JOINの発生 | ほぼ不要             | テーブルをまたぐ分だけ増える |

3

## 正規化 で何ができるようになった？



更新漏れを防げる



データの整合性を保ちやすい



無駄な容量を減らせる

## ひとことメモ

正規化は「同じ情報は1箇所にだけ書く」がゴール。ただしやりすぎるとJOINだらけで遅くなるので、非正規化とのバランスが大事になる。

## 関連キーワード

差分ペア



### 非正規化

あえて重複を持たせて読み取りを速くする、正規化の逆張り。

P.102

関連するキーワード



### ER図

正規化した結果のテーブル関係を絵にしたもの。

P.103

# 非正規化

「正しさ」より「速さ」を選ぶ設計判断

## ひとことという

正規化で分けたデータをあえて重複させて持たせ、JOINを減らして読み取りを速くする設計。

### 1 なぜ生まれた？

正規化を徹底したテーブル構成では、画面表示のたびに多数のテーブルをJOINする必要があり、表示速度が低下しやすかった。



### 2 それまでどうしていた？

正規化だけで設計していた頃は、JOINの数がページを増やすたびにどんどん増えていった。

|        | 正規化のみ         | 非正規化を併用           |
|--------|---------------|-------------------|
| 読み取り速度 | JOINが多く遅くなりがち | JOINを減らして速くできる    |
| JOINの数 | 関連の数だけ増える     | 重複を持たせた分だけ減る      |
| 更新の手間  | 1箇所直せばよい      | 重複箇所すべてを同期する必要がある |

### 3 非正規化 で何ができるようになった？



読み取りが速くなる



JOINを減らせる



集計値をキャッシュ的に持てる

## ひとことメモ

非正規化は「読み取り最適化のための重複」。重複データを更新するたびに複数箇所を直す責任とセットで導入する必要がある。

## 関連キーワード

差分ペア



### 正規化

重複を減らし、更新漏れを防ぐための基本の設計手法。

P.101

関連するキーワード



### Cache

非正規化と同じく、読み取り速度を優先する手段のひとつ。

P.100

# ER図

## テーブル同士の関係を絵にした設計図

### ひとことというと

テーブルの構造と、テーブル同士のつながり(1対多など)を図で表したものだ。

1

### なぜ生まれた?

テーブル数が多いプロジェクトでは、コードやSQLを読むだけではテーブル同士の関係性を把握しづらかった。



2

### それまでどうしていた?

ER図がない、もしくは古いままの現場では、ドキュメントを読んでも関係が分からなかった。

|            | ER図なし・古いまま    | ER図を整備・更新     |
|------------|---------------|---------------|
| 新規参入者の理解速度 | コードを読み解く必要がある | 図を見れば全体像がつかめる |
| 設計変更時の影響範囲 | 見落としやすい       | 関係を辿って確認しやすい  |
| 共有のしやすさ    | 口頭・文章で説明が必要   | 図1枚で共有できる     |

3

### ER図で何ができるようになった?



全体構造が一目でわかる



設計レビューがしやすい



変更の影響範囲を追やすい

### ひとことメモ

ER図は正しく更新され続けて初めて価値を持つ。  
古いER図は「嘘の地図」になるので、スキーマ変更時は必ず一緒に直す。

## 関連キーワード

### 関連するキーワード



#### 正規化

ER図に描かれるテーブル分割の考え方そのもの。

P.101



#### 非正規化

正規化した図をあえて崩す判断も、ER図の上で検討する。

P.102

# 104 | UUID

連番じゃない「ぶつからないID」

ひとことというと

ランダム性の高い長い文字列でIDを作り、複数の場所で同時に発行してもぶつからないようにする仕組み。

1 なぜ生まれた？

auto incrementの連番IDをそのままURLなどで公開すると、IDから件数や作成順序といった情報が推測されてしまう恐れがあった。



2 それまでどうしていた？

それまではauto increment(連番)のPrimary Keyを、そのまま外部にも見せていた。

|              | auto increment<br>(連番) | UUID          |
|--------------|------------------------|---------------|
| 採番のタイミング     | DBへの登録時に決まる            | アプリ側で事前に決められる |
| 複数サーバーでの衝突   | 採番がぶつかる可能性がある          | ほぼ衝突しない       |
| 外部公開時の推測しやすさ | 連番から件数が推測できる           | 推測されにくい       |

3 UUID で何ができるようになった？



分散環境でも安全に採番できる



連番から件数を推測されにくい



登録前にIDを決めておける

ひとことメモ

UUIDは便利だが、文字列が長い分インデックスも重くなりがち。「全部UUIDにする」のではなく、必要な場所だけ使うのが現実的。

関連キーワード

関連するキーワード



ER図

Primary KeyをUUIDにするかは設計段階で決めておきたい。

P.103

# MySQL

「とりあえずこれ」で選ばれ続ける定番RDB

## ひとことという

世界中の現場で長く使われてきた、実績重視のオープンソースリレーショナルデータベース。

1

## なぜ生まれた？

新規プロジェクトでDBを選定する際、社内外に知見や情報が豊富で、トラブル時にも解決策を見つけやすい製品が求められた。



2

## それまでどうしていた？

DB選定を経験する前は、DBの違いを意識せず「なんとなく有名だから」で決めていた。

|            | なんとなく選ぶ   | 理由を言語化して選ぶ              |
|------------|-----------|-------------------------|
| 情報量・実績     | 意識していなかった | 困ったときの情報が見つけやすい         |
| ホスティングの選択肢 | 意識していなかった | クラウド・レンタルサーバーで広くサポート    |
| 機能の豊富さ     | 意識していなかった | PostgreSQLと都度比較する視点を持てる |

3

## MySQL で何ができるようになった？



困ったときの情報が  
見つけやすい



幅広い環境でサポ  
ートされている



中～大規模まで実  
績で安心できる

## ひとことメモ

MySQLは「みんな使っているから安心」を選定理由にできる数少ない技術。とはいえPostgreSQLとの機能差は年々縮まっているので、都度比較する癖はつけておきたい。

## 関連キーワード

比較する技術



### PostgreSQL

機能の豊富さで選ばれることが多いRDB。

P.106



### SQLite

サーバー不要で、小規模用途に向くRDB。

P.107

# PostgreSQL

「機能で選ぶ」ならこっち、なRDB

## ひとことという

JSON型や高度な検索機能など、機能の豊富さに定評があるオープンソースリレーショナルデータベース。

### 1 なぜ生まれた？

複雑な検索条件や位置情報検索、柔軟なデータ型など、標準的なRDBの機能だけでは対応しづらい要件が出てきた。



### 2 それまでどうしていた？

MySQL一択だと思っていたが、要件によっては機能面で足りないことに気づいた。

|            | MySQL    | PostgreSQL        |
|------------|----------|-------------------|
| 型の豊富さ      | 基本的な型が中心 | JSON型・配列型なども扱える   |
| 拡張機能       | 限定的      | 全文検索・位置情報検索などが充実  |
| 標準SQLへの準拠度 | 独自拡張が多い  | 標準SQLに忠実で挙動が読みやすい |

### 3 PostgreSQL で何ができるようになった？



複雑なデータ構造も保存できる



拡張機能で高度な検索ができる



SQL標準準拠で挙動を追いやすい

## ひとことメモ

「MySQLかPostgreSQLか」は宗教論争になりがちだが、実際はプロジェクトの要件(欲しい機能・チームの知見)で決めればいい話。

## 関連キーワード

比較する技術



### MySQL

情報量と実績の多さで選ばれることが多いRDB。

P.105

# SQLite

サーバーいらずの「ファイル1つで完結するDB」

## ひとことという

サーバープロセスを立てず、1つのファイルとしてデータベースを扱える軽量なリレーショナルデータベース。

### 1 なぜ生まれた？

個人開発や簡単な検証のたびに本格的なDBサーバーを構築するのは、規模に見合わないコストがかかっていた。



### 2 それまでどうしていた？

SQLiteを知る前は、小さいアプリでも毎回サーバー型のRDBを立てて使っていた。

|            | サーバー型RDB       | SQLite       |
|------------|----------------|--------------|
| セットアップの手軽さ | サーバーの起動・設定が必要  | ファイル1つで即使える  |
| 同時書き込みへの強さ | 複数接続の同時書き込みに強い | 同時書き込みは苦手    |
| 向いている用途    | 本番の複数人利用       | 試作・テスト・小規模用途 |

### 3 SQLite で何ができるようになった？



インストール不要  
ですぐ試せる



テスト・開発環境で  
高速に動く



ファイル1つで持ち  
運べる

## ひとことメモ

SQLiteは「試作・テスト・小規模」の相棒。  
同時アクセスが多い本番サービスでは、素直にMySQLやPostgreSQLに切り替えた方がいい。

## 関連キーワード

比較する技術



### MySQL

本番の複数人利用  
に向く定番RDB。

P.105



### PostgreSQL

機能重視ならこちらの  
RDB。

P.106

# Redis

メモリ上で爆速に動く「その場しのぎ」の頼れる相棒

## ひとことというと

データをメモリ上に持つことで、キャッシュやセッション管理などを高速に処理できるインメモリ型データベース。

## 関連キーワード

関連するキーワード



### Cache

Redisはキャッシュの保存先として使われることが多い。

P.100



### MongoDB

同じNoSQLでも、こちらはドキュメント指向データベース。

P.109

## 1 なぜ生まれた？

セッション情報やランキングのような一時的で頻繁にアクセスされるデータを毎回RDBに問い合わせると、応答速度がボトルネックになっていた。



## 2 それまでどうしていた？

Redis導入前は、セッションやランキングのような一時的なデータもRDBに保存していた。

|         | RDBに保存       | Redisに保存          |
|---------|--------------|-------------------|
| 読み書きの速度 | ディスクI/Oが発生する | メモリ上なので非常に速い      |
| データの永続性 | 基本的に永続化されている | 設定しないと再起動で消える     |
| 向いている用途 | 正確さが重要なデータ   | セッション・ランキング・キャッシュ |

## 3 Redis で何ができるようになった？



読み書きが非常に速い



キャッシュ・セッション管理に最適



ランキングなど独自の使い方ができる

## ひとことメモ

Redisは「消えてもいいデータの置き場所」として使うのが基本。重要なデータの唯一の保存先にはしない。

# MongoDB

テーブルの形を決めずに突っ走れるNoSQL

## ひとことという

スキーマ(テーブル構造)を事前に固定せず、JSONに近い形式でデータを柔軟に保存できるNoSQLデータベース。

### 1 なぜ生まれた？

仕様が固まっていない段階で頻繁にカラムを変更すると、そのたびにマイグレーションを書く必要があり、開発の負担になっていた。



### 2 それまでどうしていた？

RDB一本で作っていた頃は、スキーマ変更のたびにマイグレーションと向き合っていた。

|               | RDB(スキーマ固定)       | MongoDB(スキーマレス) |
|---------------|-------------------|-----------------|
| データ構造の自由度     | カラム追加にマイグレーションが必要 | 構造を決めずに保存できる    |
| JOIN(結合)のしやすさ | テーブルをまたいで結合しやすい   | 結合は基本的に苦手       |
| データの整合性       | DB側である程度保証される     | アプリ側で保証する必要がある  |

### 3 MongoDB で何ができるようになった？



構造が変わりやすいデータを扱いやすい



JSON形式のデータをそのまま保存できる



水平方向にスケールさせやすい

## ひとことメモ

「スキーマレス」は自由な分だけ、整合性を守るのはアプリ側の責任になる。何でもMongoDBにするのではなく、向いているデータの形かどうかを見極める。

## 関連キーワード

関連するキーワード



### 正規化

RDBの正規化とは対照的な、構造を固定しない考え方。

P.101



### Redis

同じNoSQLでも、こちらはインメモリのキー・バリュー型。

P.108

# View(ビュー)

よく使うSQLに名前をつけて「見た目だけのテーブル」にする

## ひとことという

複雑なSELECT文をあらかじめ定義しておき、あたかも1つのテーブルであるかのように参照できるようにする仕組み。

## 関連キーワード

関連するキーワード



### Stored Procedure

SELECT文だけでなく、処理そのものをDB側に置く仕組み。

P.111



### Cache

重い集計を都度計算させたくない点はビューと共通。

P.100

## 1 なぜ生まれた?

複数テーブルをJOINする同じ集計ロジックのSQLをあちこちのコードに書くと、修正漏れによって画面ごとに結果が食い違う不整合が起きやすかった。



## 2 それまでどうしていた?

ビューを知らない頃は、同じJOIN・集計ロジックのSQLをあちこちを書いていました。

|          | 都度SQLを書く        | ビューを使う         |
|----------|-----------------|----------------|
| SQLの重複   | あちこちに同じSQLが散らばる | ビューとして1箇所にまとまる |
| 修正時に直す箇所 | 書いた分だけ全部直す必要がある | ビューの定義を直せばよい   |
| 実データの保存  | 該当しない           | ビュー自体は実体を持たない  |

## 3 View(ビュー)で何ができるようになった?



複雑なSQLを使い回せる



ロジックを1箇所にまとめられる



テーブルのように扱える

### ひとことメモ

ビューはあくまで「保存されたSELECT文」で、実データは持たない。頻繁に叩く重い集計は、マテリアライズドビューやキャッシュも検討したい。

# Stored Procedure

DBの中に処理そのものを住ませる

## ひとことというと

一連のSQL処理をDBサーバー側に手続きとして保存しておき、呼び出すだけで実行できるようにする仕組み。

### 1 なぜ生まれた？

大量データの集計処理をアプリ側のコードで行うと、DBとアプリの間で何度もデータをやり取りする分だけ時間がかかっていた。



### 2 それまでどうしていた？

ストアドプロシージャを使う前は、アプリ側のコードで処理を全部書いていた。

|              | アプリ側で処理          | ストアドプロシージャ       |
|--------------|------------------|------------------|
| 処理速度(通信回数)   | DBとのやり取りが多く発生する  | DB内で完結し通信を減らせる   |
| バージョン管理のしやすさ | Gitで差分・レビューがしやすい | DB側にあり管理が難しくなりがち |
| 複数アプリからの利用   | アプリごとに実装しがち      | 同じ処理を共通で呼び出せる    |

### 3 Stored Procedure で何ができるようになった？



DBとアプリ間の通信を減らせる



複雑な処理をDB内で完結できる



複数アプリから同じ処理を呼び出せる

## ひとことメモ

便利だが「ロジックがDBの中に隠れる」のはチーム開発では諸刃の剣。採用するなら管理・レビュー方法もセットで決めておきたい。

## 関連キーワード

関連するキーワード



### View(ビュー)

処理ではなく、SELECT文をまとめて名前をつける仕組み。

P.110

# ACID

「データが壊れない」ことを約束する4つの性質

## ひとことという

原子性・一貫性・独立性・永続性という、トランザクションが満たすべき4つの性質の頭文字。

## 関連キーワード

関連するキーワード



### MongoDB

ACIDの厳格さより、柔軟さ・速度を優先することが多い。

P.109



### Sharding

DBを分割すると、ACIDを保つ難易度も上がる。

P.113

## 1 なぜ生まれた？

複数のSQLを個別に実行すると、一部だけ成功して失敗した際に、データが中途半端な状態のまま残ってしまう恐れがあった。



## 2 それまでどうしていた？

トランザクションを意識する前は、1つ1つのSQLをただ個別に実行していた。

|              | 個別にSQL実行      | トランザクションでまとめる |
|--------------|---------------|---------------|
| 途中で失敗したときの挙動 | 中途半端な状態が残る    | 全部なかったことのできる  |
| 同時アクセス時の安全性  | データが壊れる可能性がある | 独立性が保たれる      |
| 障害後のデータ      | 消える可能性がある     | 永続性が保証される     |

## 3 ACID で何ができるようになった？



途中失敗しても中途半端にならない



同時アクセスでもデータが壊れない



障害後もデータが消えない

### ひとことメモ

ACIDは「絶対を守ってほしい最低限のお約束」。逆にNoSQLの多くは、ACIDの一部を緩めることで速度やスケーラビリティを稼いでいる。

# 113

# Sharding

1つのDBじゃ抱えきれなくなったデータを「横に分ける」

## ひとことという

1つの大きなテーブル・DBを複数のサーバーに分割して、それぞれに一部のデータだけを持たせる仕組み。

## 1 なぜ生まれた？

サービスの成長でデータ量・アクセス数が増えると、1台のDBサーバーだけでは処理を捌ききれなくなっていた。



## 2 それまでどうしていた？

シャーディング前は、1台のDBサーバーにスペックを盛って(スケールアップして)対応していた。

|               | スケールアップ(1台を強化) | シャーディング(複数台に分割)   |
|---------------|----------------|-------------------|
| 対応できるデータ量     | 1台のスペックが上限になる  | 台数を増やして拡張できる      |
| JOIN・トランザクション | 同じDB内なら容易      | シャードをまたぐと難しくなる    |
| 構成の複雑さ        | シンプル           | 分割ルール的设计・運用が複雑になる |

## 3 Sharding で何ができるようになった？



1台の限界を超えて扱える



台数を増やして負荷分散できる



成長に合わせて拡張できる

## ひとことメモ

シャーディングは強力だが後戻りが大変な設計判断。「本当に今シャーディングが必要か」は、まずレプリケーションやキャッシュで粘れないか検討してから。

## 関連キーワード

関連するキーワード



### レプリケーション

分割ではなく複製で負荷を分散する、もう1つの手段。

P.114



### ACID

DBを分割するほど、この4性質を保つのが難しくなる。

P.112

# レプリケーション

同じデータを複数のDBにコピーして「読み取りを分散」

## ひとことという

1つのDBのデータを別のDBサーバーに複製し続け、読み取りアクセスなどを分散させる仕組み。

## 関連キーワード

関連するキーワード



### Sharding

複製ではなく分割で負荷を分散する、もう1つの手段。

P.113

## 1 なぜ生まれた?

アクセスが増えるとDBへの読み取り(SELECT)が集中し、1台のサーバーだけでは負荷が偏ってCPU使用率が張り付いてしまっていた。



## 2 それまでどうしていた?

レプリケーション導入前は、1台のDBに全ての読み書きが集中していた。

|             | DB1 台に集中    | レプリケーション(複製して分散) |
|-------------|-------------|------------------|
| 読み取りの負荷分散   | 1 台に集中する    | 複数台に分散できる        |
| 障害時の切り替え    | 1 台が落ちると止まる | 他のレプリカで継続できる     |
| データ反映のタイムラグ | 発生しない       | わずかな遅延が発生する      |

## 3 レプリケーション で何ができるようになった?



読み取りアクセスを分散できる



1 台が落ちてでも他で継続できる



バックアップとしても使える

### ひとことメモ

レプリケーションは万能ではなく、複製が反映されるまでのわずかなズレ(レプリケーション遅延)は必ず起きる。「更新した直後に自分で見えるデータ」は特に注意が必要。

# コネクションプール

DBとの「つながっぱなし」の回線をみんなで使い回す

## ひとことという

DBへの接続をあらかじめ複数用意してプールしておき、リクエストごとに使い回すことで接続のオーバーヘッドを減らす仕組み。

### 1 なぜ生まれた？

リクエストのたびにDBへの接続を新規に張ると、接続処理自体のコストが積み重なり、同時接続数の上限に達しやすかった。



### 2 それまでどうしていた？

コネクションプールを意識する前は、リクエストのたびにDBへの接続を新しく張っていた。

|          | 都度接続       | コネクションプールで<br>使い回し |
|----------|------------|--------------------|
| 接続確立のコスト | 毎回発生する     | あらかじめ用意した接続を使い回す   |
| 同時接続数の管理 | 意識しづらい     | 上限を決めて管理できる        |
| 接続不足時の挙動 | エラーが発生しやすい | 待機させて捌くこともできる      |

### 3 コネクションプール で何ができるようになった？



接続確立の時間を  
節約できる



同時接続数を制御  
しやすい



高負荷時も安定し  
て捌ける

## ひとことメモ

プールのサイズは「多ければ良い」ではない。  
DBサーバー側の上限やアプリのサーバー台数とのバランスを見て決める必要がある。

## 関連キーワード

関連するキーワード



### N+1

クエリ数が増えるほど、接続の使い回し方も重要になる。

P.99



### Cache

接続や問い合わせそのものを減らす、もう1つの手段。

P.100

# 121 | HTML

Web ページの骨組みを作るマークアップ言語

## ひとことという

見出しや段落、画像などの「要素」を並べて、Web ページの構造を作るための言語。

### 1 なぜ生まれた？

研究者同士で文書を共有するために、装飾がなくても構造がわかる書き方が必要だった。そこでタグで意味を示す「マークアップ」という発想からHTMLが生まれた。



### 2 それまでどうしていた？

それまでは、Word 文書のような「見たまま」を編集するファイルでやり取りしていた。

|          | 普通の文書ファイル        | HTML          |
|----------|------------------|---------------|
| 見た目の編集   | 見たまま(WYSIWYG)で編集 | タグで構造だけを指定    |
| 共有方法     | ファイルをやり取り        | URL で誰でも閲覧できる |
| 機械的な読み取り | レイアウト情報が中心       | 見出し・段落など意味が明確 |

### 3 HTML で何ができるようになった？



見出しや段落を意味づけできる



リンクで文書同士をつなげる



ブラウザさえあれば誰でも見られる

## ひとことメモ

HTML が担うのは「文章の意味・構造」。  
見た目は CSS、動きは JavaScript が担当するため、3 つはセットで語られることが多い。

## 関連キーワード

セットで使う技術



### CSS

HTML の見た目を整える言語。

P.122



### JavaScript

HTML に動きを与える言語。

P.123



### DOM

HTML を JS から操作できる形にしたもの。

P.127

## 見た目を文章の中身から切り離した言語

### ひとことというと

色・余白・レイアウトなど「見た目」だけをHTMLから分離して指定する言語。

1

### なぜ生まれた？

昔はHTMLタグそのものに色やフォントを直接埋め込んでいたため、デザインを変えるたびに全ページのタグを書き換える必要があった。



2

### それまでどうしていた？

それまではHTMLタグに直接、見た目の指定を書き込んでいた。

|        | HTMLに直接指定     | CSS                  |
|--------|---------------|----------------------|
| 見た目の変更 | 全ページのタグを書き換え  | 1つのファイルを直せば全体に反映     |
| 保守性    | HTMLとデザインが密結合 | 構造(HTML)と見た目(CSS)を分離 |
| 再利用    | ページごとに書き直す    | クラス指定で使い回せる          |

3

### CSS で何ができるようになった？



色・余白・フォントを一括管理できる



画面サイズに応じて見た目を変えられる



アニメーションも表現できる

### ひとことメモ

「HTMLは構造、CSSは見た目」と覚えると迷わない。  
CSSフレームワークを使うと、この見た目の指定を一から書かずに済む。

## 関連キーワード

関連する技術



### HTML

CSSが見た目を整える対象の文書構造。

P.121



### CSSフレームワーク

よく使うCSSをあらかじめ用意した工具箱。

P.141

# JavaScript

ブラウザに「動き」を与えたプログラミング言語

## ひとことというと

Webページ内で計算や画面操作を行い、ユーザー操作に応じて動的に変化させる言語。

## 関連キーワード

関連する技術



### HTML

JavaScriptが操作する対象の構造。

P.121



### CSS

JavaScriptが書き換える対象の見た目。

P.122



### TypeScript

JavaScriptに型を追加した言語。

P.124



### DOM

JavaScriptから操作するHTMLの木構造。

P.127

## 1 なぜ生まれた？

HTML/CSSだけでは静止した文書しか作れず、入力チェックやアニメーションなど「動き」が必要な場面に対応できなかった。



## 2 それまでどうしていた？

それまではHTML/CSSだけの、動きのない静的なページが中心だった。

|          | 静的なHTML/CSSのみ          | JavaScript追加後     |
|----------|------------------------|-------------------|
| ページの振る舞い | リンクをクリックしたら別ページに遷移するだけ | クリックした場で表示を変えられる  |
| 入力チェック   | サーバーに送ってからエラー表示        | 送信前にブラウザ内でチェックできる |
| 更新方法     | ページ全体を再読み込み            | 一部だけ書き換え可能        |

## 3 JavaScript で何ができるようになった？



クリックや入力に反応できる



ページの一部だけ書き換えられる



サーバーと通信もできる

### ひとことメモ

今ではJavaScriptはブラウザだけでなく、Node.jsなどでサーバー側でも動く。

TypeScriptはこのJavaScriptに型を足したもの。

# TypeScript

JavaScriptに「型」の安全ベルトをつけた言語

## ひとことという

JavaScriptに型情報を追加し、実行前にバグの芽をエディタ上で検出できるようにした言語。

1

## なぜ生まれた？

JavaScriptは型が自由な分、「文字列のつもりが実は数値だった」のようなミスが実行するまで発覚しにくく、大規模開発ほど事故が起きやすかった。



2

## それまでどうしていた？

それまではJavaScriptのまま、型を意識せずに開発していた。

|        | JavaScript          | TypeScript         |
|--------|---------------------|--------------------|
| 型のチェック | 実行するまでわからない         | 書いている時点でエディタが警告    |
| 大規模開発  | どこで何の型が渡るか読みづらい     | 型定義がドキュメント代わりになる   |
| 実行環境   | そのままブラウザ/Node.jsで動く | コンパイルしてJSに変換してから動く |

3

## TypeScript で何ができるようになった？



変数の型を事前に定義できる



エディタの補完が効きやすくなる



コンパイル時にミスに気づける

## ひとことメモ

「動くけど正しいか実行するまでわからない」JSに対し、「書いた時点で正しさを確認できる」のがTS。その分コンパイルという手間が増える。

## 関連キーワード

関連する技術



### JavaScript

TypeScriptのベースになった言語。

P.123



### バニラ

型を付けない素のJavaScriptで書くこと。

P.125

# バニラ (Vanilla JS)

フレームワークなしの「素の JavaScript」

## ひとことというと

ReactやVueなどのライブラリを使わず、標準機能だけで書く JavaScript のこと。

## 関連キーワード

関連する技術



### JavaScript

バニラJSの元になる標準言語。

P.123



### DOM

バニラJSが標準APIで直接操作する対象。

P.127

## 1 なぜ生まれた？

昔はjQueryのようなライブラリを使うのが当たり前で、素のJSで書くことがむしろ珍しくなり、「何も足していない」定番フレーバーになぞらえて呼ばれ始めた。



## 2 それまでどうしていた？

それまではブラウザごとの挙動の違いを吸収するために、jQueryなどが重宝されていた。

|         | jQuery           | バニラJS(標準API)                    |
|---------|------------------|---------------------------------|
| 要素の取得   | \$('#id') のように書く | document.querySelector() のように書く |
| 依存      | ライブラリの読み込みが必須    | 追加ライブラリ不要                       |
| ブラウザ対応差 | jQueryが吸収してくれた   | 標準APIが揃った今はほぼ気にしなくてよい           |

## 3 バニラ(Vanilla JS) で何ができるようになった？



追加のライブラリなしで動く



標準APIだけで完結する



書き方の自由度は自分次第

## ひとことメモ

昔はブラウザごとの挙動差を吸収するためにjQueryが重宝されたが、標準APIが整った今はバニラJSだけで十分書けることが増えた。

# ES Modules

JavaScript ファイルを「部品」として輸出入する仕組み

## ひとことという

import/export を使って、JavaScript のコードをファイル単位で分割・再利用できるようにする標準の仕組み。

1

## なぜ生まれた？

昔は<script>タグを並べる順番でグローバル変数がぶつかったり、ファイル同士の依存関係がコードから追えなかったりした。



2

## それまでどうしていた？

それまでは<script>タグを並べる順番に頼って、グローバル空間を共有していた。

|        | グローバルスクリプト             | ES Modules                   |
|--------|------------------------|------------------------------|
| ファイル分割 | <script>タグの数だけ読み込み順に注意 | import/export で依存関係を明示       |
| 変数の衝突  | グローバル空間を全ファイルで共有       | ファイルごとにスコープが分離               |
| 再利用    | コピーで使い回すことも多かった        | export した関数やクラスを import するだけ |

3

## ES Modules で何ができるようになった？



必要な部分だけ import できる



依存関係がコードから追える



ビルドツールとも相性がよい

## ひとことメモ

ES Modules は Node.js の CommonJS (require/module.exports) と並び、もう一つの「部品分け」の方式。ブラウザが標準対応したことで主流になった。

## 関連キーワード

関連する技術



JavaScript

ES Modules が分割対象とする言語。

P.123

# 127 | DOM

ブラウザがHTMLを「木構造」として持ったメモリ上の姿

### ひとことという

ブラウザが読み込んだHTMLを、JavaScriptから操作できるようツリー状のオブジェクトとして表現したもの。

## 1 なぜ生まれた？

HTMLはただの文字列(テキスト)なので、そのままではJavaScriptから「このボタンの色を変える」といった操作ができなかった。



## 2 それまでどうしていた？

それまではHTMLをただの文字列として扱うしかなく、JSから直接触れなかった。

|         | HTML文字列そのまま      | DOM(オブジェクトの木)            |
|---------|------------------|--------------------------|
| JSからの操作 | 文字列を書き換えても反映されない | getElementByIdなどで直接操作できる |
| 構造の把握   | テキストを読んで判断するしかない | 親子・兄弟関係をたどれる             |
| 更新      | ページ全体を再解釈        | 該当ノードだけ差し替え可能            |

## 3 DOM で何ができるようになった？



要素を取得・変更できる



イベントを取り付けられる



一部のノードだけ書き換えられる

### ひとことメモ

「HTMLを解析してできた操作用の木」がDOM。  
Virtual DOMは、この本物のDOMを直接触る代わりに仮の木で差分計算してから反映する工夫。

## 関連キーワード

関連する技術



### HTML

DOMのもとになる文書構造。

P.121



### Event

DOMの要素に取り付ける反応の仕組み。

P.128



### Virtual DOM

本物のDOMの代わりに使う仮想の木。

P.137

# Event

「クリックされた」を合図にコードを動かす仕組み

## ひとことという

クリックやキー入力など、ユーザーやブラウザの動作をきっかけにJavaScriptの処理を実行させる仕組み。

1

## なぜ生まれた？

JavaScriptは基本的に上から順番に実行されるだけなので、「ボタンが押されたら」のようにタイミングを待って処理を始めたい場面に対応できなかった。



2

## それまでどうしていた？

それまでは、状態が変わっていないか定期的にチェックし続ける発想しかなかった。

|         | 定期的な状態チェック        | Event                   |
|---------|-------------------|-------------------------|
| 処理の開始   | 定期的に状態をチェックし続ける   | 発生したその瞬間に呼ばれる           |
| 無駄な処理   | 何も起きてなくてもチェックし続ける | 何も起きなければ何もしない           |
| コードの書き方 | if文で状態を監視         | addEventListenerで登録するだけ |

3

## Event で何ができるようになった？



クリックや入力に  
反応できる



複数の処理を同時  
に待ち受けられる



非同期処理ともよ  
く組み合わせる

## ひとことメモ

「onClick」のような名前を見たら、だいたいこのEventの仕組みが背後にある。Reactなどのライブラリは、これをより宣言的に書けるようにしている。

## 関連キーワード

関連する技術



### DOM

Eventを取り付ける対象の要素。

P.127



### 非同期

### (Asynchronous)

Eventの発生を待つ考え方の土台。

P.138



### Component

Eventへの反応をまとめて持つ部品。

P.129

# Component

画面を「部品」として組み立てる考え方

## ひとことというと

ボタンやカードなど、画面の一部をひとまとまりの部品として切り出し、組み合わせて画面を作る考え方。

### 1 なぜ生まれた？

ページ全体でHTML/CSS/JSを書いていると、同じような見た目の部品をあちこちにコピペし、1箇所直すたびに全部を探して修正する必要があった。



### 2 それまでどうしていた？

それまではページ単位でHTML/CSS/JSをその都度書いていた。

|        | ページ単位で記述             | Component単位で部品化          |
|--------|----------------------|--------------------------|
| 再利用    | 似た見た目をコピペで量産         | 1つのComponentを呼び出すだけ      |
| 修正     | 使っている箇所を全部探して直す      | Component本体を直せば全箇所に反映    |
| 状態と見た目 | HTMLとJSが別ファイルで散らばりがち | StateやPropsとセットで1部品にまとまる |

### 3 Component で何ができるようになった？



見た目と振る舞いをひとまとめにできる



何度でも使い回せる



組み合わせて画面を作れる

## ひとことメモ

ReactやVueは、このComponentという考え方を中心に画面を作るライブラリ。Componentの中身を決めるのがStateとProps。

## 関連キーワード

関連する技術



### State

Componentが内部に持つ今の状態。

P.130



### Props

親から子のComponentへ渡す値。

P.131



### Virtual DOM

Componentの変化を効率よく反映する仕組み。

P.137

# State

Componentが内部に持つ「今の状態」

## ひとことという

開閉中かどうか、入力中の文字など、Componentが内部で持ち続ける「今の状態」のデータ。

1

## なぜ生まれた？

画面はクリックや入力のたびに見た目が変わる。「今どういう状態か」をどこかに覚えておかないと、変化のたびに画面全体を作り直すことになってしまう。



2

## それまでどうしていた？

それまではDOMを直接書き換えて、その場その場で見た目を管理していた。

|          | DOMを直接書き換え       | Stateで管理            |
|----------|------------------|---------------------|
| 見た目の変更   | 都度DOMを探して書き換える   | Stateを更新するだけで見た目が追従 |
| 現在の状態の把握 | DOMを見て判断するしかない   | 変数(State)を見ればわかる    |
| バグの発生源   | 見た目とデータがずれることがある | データ(State)が唯一の正解    |

3

## State で何ができるようになった？



「今の値」を保持できる



更新すると画面に反映される



Propsとして子に渡すこともできる

## ひとことメモ

「Stateが変わったら画面も変わる」という考え方が、ReactやVueの核。Virtual DOMは、このStateの変化を効率よく画面に反映するための仕組み。

## 関連キーワード

関連する技術



### Component

Stateを内部に持つ部品。

P.129



### Props

Stateを子に渡すときの形。

P.131



### Virtual DOM

Stateの変化を効率よく反映する仕組み。

P.137

# Props

親から子のComponentへ渡す「荷物」

## ひとことという

親のComponentから子のComponentへ、表示内容や設定値を渡すためのデータ。

## 関連キーワード

関連する技術



### Component

Propsを受け取る部品。

P.129



### State

Propsのもとになることが多いデータ。

P.130

## 1 なぜ生まれた？

同じボタンComponentでも「送信」と表示したい場所と「キャンセル」と表示したい場所がある。Componentは使い回しつつ、表示内容だけ呼び出し側で変えたいというニーズがあった。



## 2 それまでどうしていた？

それまではComponentごとに文言を書き換えたコピーを作っていた。

|         | コピーして文言を変更                 | Propsで外から渡す              |
|---------|----------------------------|--------------------------|
| 表示の出し分け | ボタンごとにComponentをコピーして文言を変更 | 1つのComponentにPropsで文言を渡す |
| 修正のしやすさ | コピーした数だけ修正が必要              | Component本体を直せば全箇所に反映    |
| データの向き  | 決まっていない                    | 親→子への一方向                 |

## 3 Props で何ができるようになった？



呼び出し側から表示内容を指定できる



Componentを使い回しやすくなる



Stateと組み合わせて使う

## ひとことメモ

Propsは「親から子への一方向」の受け渡し。  
逆に子から親へ伝えたい時は、関数をPropsとして渡してもらい、それを呼び出す形にする。

# CSR

## ブラウザ側でHTMLを組み立てる方式

### ひとことというと

サーバーからは最小限のHTMLとJavaScriptだけを送り、実際の画面はブラウザ側のJavaScriptが組み立てる方式。

### 1 なぜ生まれた？

ReactやVueのようなComponent志向のライブラリが普及し、画面遷移のたびにサーバーへ問い合わせず、ブラウザ内で高速に画面を差し替えたいというニーズが強まった。



### 2 それまでどうしていた？

それまではアクセスのたびにサーバーが完成したHTMLを生成して返していた。

|           | 旧来のサーバーレンダリング    | CSR                     |
|-----------|------------------|-------------------------|
| 初回表示までの流れ | サーバーが完成形のHTMLを返す | 空に近いHTMLを返し、JSが画面を組み立てる |
| 画面遷移      | ページごとにサーバーへリクエスト | 遷移後の差分だけJSが書き換え         |
| サーバーの負荷   | 表示のたびにHTMLを生成    | 初回以降はデータだけ返せばよい         |

### 3 CSR で何ができるようになった？



ページ遷移がスnakになる



Component中心に作りやすい



初期表示にはJS読み込みの時間がかかる

### ひとことメモ

CSRは「見た目を組み立てる仕事」をサーバーからブラウザに移した方式。その分、初回表示が遅くなりやすいという弱点をSSRが補う。

## 関連キーワード

関連するレンダリング方式



### Component

CSRで画面を組み立てる部品の単位。

P.129



### SSR

サーバー側で先にHTMLを完成させる方式。

P.133



### Hydration

届いたHTMLにあとから動きを与える処理。

P.136

# SSR

サーバー側でHTMLを完成させてから届ける方式

### ひとことという

ブラウザで組み立てる前に、サーバー側で完成したHTMLを生成してから返す方式。

## 1 なぜ生まれた？

CSRは初回表示までJSの読み込み・実行を待つ必要がある、表示が遅れたり検索エンジンが内容をうまく認識できないという課題があった。



## 2 それまでどうしていた？

それまではCSRで、空のHTMLとJSを送ってからブラウザ側が画面を組み立てていた。

|           | CSR                    | SSR                    |
|-----------|------------------------|------------------------|
| 初回表示までの流れ | 空のHTML+JSを送りブラウザが組み立てる | サーバーが完成したHTMLを生成して送る   |
| 表示速度(初回)  | JSの読み込み・実行を待つ          | HTMLが届いた時点で見た目は表示される   |
| SEO       | 検索エンジンが内容を認識しづらい場合がある  | 最初から完成したHTMLなので認識されやすい |

## 3 SSR で何ができるようになった？



初回表示が速い



検索エンジンに強い



表示後にHydrationでJSを有効化する必要がある

### ひとことメモ

SSRは「初回だけサーバーで完成品を作る」方式。ただしアクセスのたびに生成し直すので負荷は上がる。変化が少ないページはSSGの方が効率的。

## 関連キーワード

関連するレンダリング方式



### CSR

ブラウザ側でHTMLを組み立てる方式。

P.132



### SSG

ビルド時にあらかじめHTMLを作る方式。

P.134



### Hydration

SSRで届いたHTMLにあとから動きを与える処理。

P.136

# SSG

ビルド時にあらかじめHTMLを作っておく方式

## ひとことという

アクセスのたびに生成するSSRとは違い、ビルドの時点で全ページのHTMLを事前に作っておく方式。

1

## なぜ生まれた？

SSRはアクセスされるたびにサーバーでHTMLを生成するため、内容がほとんど変わらないページでも毎回同じ処理を繰り返し、サーバー負荷や速度の面で無駄が多かった。



2

## それまでどうしていた？

それまではSSRで、リクエストが来るたびにサーバーがHTMLを生成していた。

|              | SSR                 | SSG               |
|--------------|---------------------|-------------------|
| HTML生成のタイミング | リクエストが来るたびに生成       | ビルド時にあらかじめ全部生成    |
| サーバーの負荷      | アクセスごとに処理が発生        | 生成済みファイルを配るだけ     |
| 内容の更新        | リクエスト時点の最新データを反映できる | 再ビルドしないと内容が更新されない |

3

## SSG で何ができるようになった？



配信がとにかく速い



サーバー負荷がほぼゼロ



更新のたびに再ビルドが必要

## ひとことメモ

SSGは「変化の少ないページ」向きの方式。更新頻度が高いページで鮮度も速さも欲しい場合に生まれたのがISR。

## 関連キーワード

関連するレンダリング方式



### SSR

アクセスのたびにHTMLを生成する方式。

P.133



### ISR

SSGを裏側で少しずつ更新する方式。

P.135

# 135 | ISR

静的なのに「裏側でこっそり」更新される方式

### ひとことというと

SSGの速さを保ちながら、一定時間ごとや条件に応じて裏側でページを再生成し、内容の鮮度も両立させる方式。

## 1 なぜ生まれた？

SSGは速いが、内容を更新するたびにサイト全体を再ビルドする必要があり、更新頻度の高いページには不向きだった。



## 2 それまでどうしていた？

それまではSSGで、更新するたびにサイト全体を再ビルドしていた。

|       | SSG                  | ISR                 |
|-------|----------------------|---------------------|
| 内容の更新 | サイト全体を再ビルドしないと反映されない | 該当ページだけ裏側で再生成       |
| 表示速度  | 常に生成済みファイルを返すので速い    | 同じく生成済みファイルを返すので速い  |
| 鮮度    | 再ビルドするまで古いまま         | 一定時間経過や条件で自動的に新しくなる |

## 3 ISR で何ができるようになった？



SSG並みの表示速度を維持



全体の再ビルドが不要



裏側で必要なページだけ更新

### ひとことメモ

ISRは「SSGの速さ」と「SSRの新しさ」のいいとこ取りを狙った方式。Next.jsなどのフレームワークが対応していることが多い。

## 関連キーワード

関連するレンダリング方式



### SSG

ビルド時にHTMLを事前生成する方式。

P.134



### SSR

アクセスのたびにHTMLを生成する方式。

P.133

# Hydration

届いたHTMLに「あとから」命を吹き込む処理

## ひとことというと

SSR/SSGで先に届いたHTMLに対して、ブラウザ側でJavaScriptを結びつけ、クリックなどに反応できる状態にする処理。

1

## なぜ生まれた？

SSR/SSGは見た目のHTMLを先に届けてくれるが、そのHTMLはただの文字列であり、まだボタンを押しても何も起きない「見た目だけ」の状態になっている。



2

## それまでどうしていた？

それまでは(Hydrationなしで)届いたHTMLがそのまま最終形だった。

|            | SSR/SSGのみ<br>(Hydrationなし) | SSR/SSG +<br>Hydration |
|------------|----------------------------|------------------------|
| 表示されるHTML  | 完成した見た目のHTMLが届く            | 同じく完成した見た目のHTMLが届く     |
| クリックなどへの反応 | ただのHTML文字列なので何も起きない        | JSが結びつき、反応できるようになる     |
| 必要な処理      | なし                         | ブラウザ側でJavaScriptを実行し直す |

3

## Hydration で何ができるようになった？



見た目は先に、反応はあとから



SSRやSSGとセットで使われる



完了までクリックしても反応しないことがある

## ひとことメモ

「見えているのに押せない」違和感の正体がHydration待ちであることが多い。この待ち時間を減らす工夫(部分的なHydrationなど)も近年進んでいる。

## 関連キーワード

関連するレンダリング方式



### SSR

Hydrationが必要になる方式のひとつ。

P.133



### SSG

Hydrationが必要になる方式のひとつ。

P.134



### CSR

HydrationではなくJSが最初から画面を組み立てる方式。

P.132

# Virtual DOM

本物のDOMを直接触らないための「下書き」

## ひとことという

実際のDOMを直接操作する代わりに、メモリ上に仮のツリーを作り、変化した部分だけを計算してから本物のDOMに反映する仕組み。

### 1 なぜ生まれた？

本物のDOMを直接何度も書き換えると、ブラウザが都度レイアウトの再計算を行うため、Stateが頻繁に変わる画面では動作が重くなりがちだった。



### 2 それまでどうしていた？

それまではStateが変わるたびに、本物のDOMを直接書き換えていた。

|         | DOMを直接操作          | Virtual DOM経由              |
|---------|-------------------|----------------------------|
| 更新の単位   | 変更のたびに本物のDOMを書き換え | メモリ上の仮想ツリーで差分を計算してからまとめて反映 |
| パフォーマンス | 更新が多いと重くなりやすい     | 差分だけ反映するので効率的              |
| 書き方     | DOM操作を自分で細かく制御    | Stateの変化を宣言するだけで反映は任せられる   |

### 3 Virtual DOM で何ができるようになった？



差分だけを計算して反映



本物のDOM操作を減らせる



Stateの変化に強い

## ひとことメモ

「仮想」とはいえ実体はただのJavaScriptのオブジェクト。ReactはこのVirtual DOMを比較(diff)する仕組みで有名になった。

## 関連キーワード

関連する技術



### DOM

Virtual DOMが最終的に反映する本物の木構造。

P.127



### State

Virtual DOMが差分を計算するきっかけになるデータ。

P.130



### Component

Virtual DOMを使って更新される部品の単位。

P.129

# 非同期 (Asynchronous)

「待ち時間」のあいだ他の処理を進める考え方

## ひとことというと

通信やファイル読み込みなど、時間のかかる処理の完了を待たずに、他の処理を並行して進められるようにする考え方。

### 1 なぜ生まれた？

JavaScriptは基本的に処理を1つずつ順番にこなすため、通信などの時間がかかる処理をそのまま待つ書き方をすると、その間ブラウザ全体が固まってしまう。



### 2 それまでどうしていた？

それまでは同期処理として、1つの処理が終わるまで次を待つのが基本だった。

|        | 同期処理                  | 非同期処理                  |
|--------|-----------------------|------------------------|
| 通信中の挙動 | 結果が返るまで他の処理が止まる       | 結果を待たずに他の処理が続けられる      |
| 画面の反応  | 通信中は固まったように見えることがある   | 通信中も操作を受け付けられる         |
| 書き方の歴史 | コールバック関数を渡す(コールバック地獄) | Promiseやasync・awaitで整理 |

### 3 非同期(Asynchronous)で何ができるようになった？



通信中も画面が固まらない



複数の処理を並行して進められる



書き方はPromiseなどで進化してきた

## ひとことメモ

「非同期」という考え方自体は昔からあったが、書き方はコールバック→Promise→async/awaitの順に読みやすく進化してきた。

## 関連キーワード

関連する技術



Promise

非同期処理の結果をまとめて扱うオブジェクト。

P.139



async・await

Promiseを同期処理のように書ける構文。

P.140



Event

非同期の発生をきっかけにする仕組み。

P.128

# Promise

「いつか終わる処理」の結果を表す約束手形

## ひとことというと

非同期処理の結果（成功か失敗か）を、あとから受け取る形でまとめて扱えるようにするオブジェクト。

## 関連キーワード

関連する技術



### 非同期 (Asynchronous)

Promiseが扱う処理そのものの考え方。

P.138



### async・await

Promiseを読みやすく書くための構文。

P.140

## 1 なぜ生まれた？

非同期処理をコールバック関数で書くと、処理が増えるたびにコールバックの中にコールバックを書くことになり（コールバック地獄）、順番やエラー処理が非常に読みにくくなっていた。



## 2 それまでどうしていた？

それまではコールバック関数を渡す方式で、非同期処理をつなげていた。

|               | コールバック関数                | Promise             |
|---------------|-------------------------|---------------------|
| 複数の非同期処理をつなげる | コールバックの中にコールバックがネストしていく | .then()で横に並べてつなげられる |
| エラー処理         | 各コールバックごとに書く必要がある       | .catch()でまとめて扱える    |
| 見た目           | ネストが深くなり右にどんどん寄っていく     | 処理の順番が上から下に読める      |

## 3 Promise で何ができるようになった？



成功・失敗をまとめて扱える



.then()で処理をつなげられる



async・awaitでさらに読みやすく書ける

### ひとことメモ

「コールバック地獄」を解消するために生まれたのがPromise。とはいえ.then()の連続もまだ読みにくく、これをさらに直感的にしたのがasync/await。

# async・await

非同期処理を「同期っぽく」読める書き方

## ひとことというと

Promiseを使った非同期処理を、あたかも上から順番に処理しているかのように書けるようにする構文。

### 1 なぜ生まれた？

Promiseの.then()は読みやすくなったとはいえ、処理が増えると.then()が連なって見づらく、変数の受け渡しも複雑になりがちだった。



### 2 それまでどうしていた？

それまではPromiseの.then()を数珠つなぎにする書き方が中心だった。

|         | Promiseの.then()    | async・await               |
|---------|--------------------|---------------------------|
| 見た目     | .then()を数珠つなぎにする   | awaitを付けるだけで上から順に読める      |
| エラー処理   | .catch()をチェーンにつなげる | try/catchという馴染みのある書き方が使える |
| 変数の受け渡し | .then()の引数として受け取る  | 通常の変数のように代入して使える          |

### 3 async・await で何ができるようになった？



同期処理のような自然な見た目



try/catchでエラー処理できる



中身の仕組みはPromiseのまま

## ひとことメモ

async/awaitは魔法ではなく、Promiseを読みやすく書くための「糖衣構文」。中身の非同期処理そのものは変わらない。

## 関連キーワード

関連する技術



### Promise

async・awaitの土台になっているオブジェクト。

P.139



### 非同期 (Asynchronous)

async・awaitが扱う処理の考え方。

P.138

# CSSフレームワーク

よく使うデザインをあらかじめ用意しておく  
工具箱

## ひとことという

ボタンやレイアウトなど、よく使うデザインパーツのCSSをあらかじめ用意しておき、クラス名を付けるだけで使えるようにしたもの。

## 関連キーワード

関連する技術



### CSS

CSSフレームワークの土台になる言語。

P.122



### UIコンポーネントライブラリ

見た目に加えて動きまで含む部品を提供するもの。

P.142

## 1 なぜ生まれた？

CSSは自由度が高い分、余白や色、レイアウトなどの細部を一から調整するのに時間がかかり、デザイナーがいないチームでは見た目の統一も難しかった。



## 2 それまでどうしていた？

それまではCSSを一から書いて、見た目を1つずつ作り込んでいた。

|          | CSSを一から書く           | CSSフレームワーク          |
|----------|---------------------|---------------------|
| 見た目の作成   | 余白・色・角丸などを1つずつ指定    | 用意されたクラスを付けるだけ      |
| デザインの統一感 | チームやページごとにばらつきが出やすい | フレームワークのルールで自然と揃う   |
| 学習コスト    | CSSの知識が幅広く必要        | クラス名の一覧を覚えれば書き始められる |

## 3 CSSフレームワークで何ができるようになった？



クラス名を付けるだけで見た目が整う



チーム内でデザインが揃いやすい



独自色を出すにはカスタマイズが必要

### ひとことメモ

BootstrapやTailwind CSSなどが代表格。  
似た役割にUIコンポーネントライブラリがあるが、そちらはボタンなどの「部品」そのものを提供する点が違う。

# UIコンポーネントライブラリ

動くパーツごと完成させて配ってくれる工具箱

## ひとことという

ボタンやモーダルなど、見た目だけでなく開閉などの動作(振る舞い)まで含んだComponentを、そのまま使える形で提供するライブラリ。

1

## なぜ生まれた?

CSSフレームワークは見た目こそ整えてくれるが、モーダルの開閉などの「動き」の部分はJavaScriptを自分で書く必要があり、Componentごと一から作るのは手間が大きかった。



2

## それまでどうしていた?

それまではCSSフレームワークで見た目だけ整え、動きの部分は自分で実装していた。

|         | CSSフレームワーク          | UIコンポーネントライブラリ          |
|---------|---------------------|-------------------------|
| 提供される範囲 | 見た目(クラス名)のみ         | 見た目+開閉などの動作込みのComponent |
| 実装コスト   | モーダルなどの開閉ロジックは自分で書く | importして使うだけで動く         |
| カスタマイズ性 | 自由度が高い分すべて自作        | 用意されたPropsの範囲での調整が中心    |

3

## UIコンポーネントライブラリで何ができるようになった?



ボタンやモーダルがそのまま使える



見た目と動きがセットで届く



Props経由で細かく調整できる

## ひとことメモ

React向けのMUIやshadcn/ui、Vue向けのVuetifyなどが代表例。CSSフレームワークが「見た目の材料」なら、こちらは「完成した部品」を提供するイメージ。

## 関連キーワード

関連する技術



### CSSフレームワーク

見た目の材料を提供するもの。

P.141



### Component

UIコンポーネントライブラリが提供する部品の単位。

P.129



### Props

UIコンポーネントライブラリの部品を調整する手段。

P.131

# Node.js

JavaScript を「ブラウザの外」で動かした技術

## ひとことという

ブラウザの中でしか実行できなかった JavaScript を、サーバーやPC上で動かせるようにするランタイム。

## 関連キーワード

仲間のランタイム



### Bun

Node.js互換で起動・実行をまるごと高速化。

P.144



### Deno

Node.js作者が作り直した安全志向のランタイム。

P.145

## 1 なぜ生まれた?

JavaScript はブラウザの中でしか実行できず、サーバーサイドは別の言語で書くしかなかった。



## 2 それまでどうしていた?

サーバーサイドは Ruby・PHP・Java など、フロントとは別の言語で書いていた。

|        | サーバー専用言語             | Node.js             |
|--------|----------------------|---------------------|
| 言語     | Ruby / PHP / Java など | JavaScript(フロントと共通) |
| 実行エンジン | 各言語専用のランタイム          | Chrome の V8 エンジン    |
| 得意分野   | 汎用処理全般               | I/O 待ちが多い並行処理       |

## 3 Node.js で何ができるようになった?



サーバーサイドを JS で書ける



フロントとバックの言語を統一



npm エコシステムを利用可能

## ひとことメモ

Node.js は「JS をどこでも動かす」の元祖。  
この後 Bun・Deno という2つの後継が生まれることになる。

# Bun

Node.js の「遅さ」をまとめて解決しようとしたランタイム

## ひとことという

Node.js 互換を保ちながら、実行・パッケージ管理・テストまで1つのバイナリで高速化したランタイム。

### 1 なぜ生まれた？

Node.js は便利だったが、起動の遅さや package.json 周りの遅さへの不満が積み重なっていた。



### 2 それまでどうしていた？

Node.js では、実行は node、パッケージ管理は npm、テストは Jest のように道具を組み合わせていた。

|         | Node.js (組み合わせ型)       | Bun                       |
|---------|------------------------|---------------------------|
| 実行エンジン  | V8                     | JavaScriptCore (Safari 系) |
| パッケージ管理 | npm / yarn を別途利用       | 標準搭載で高速                   |
| ツール構成   | node + npm + Jest など別々 | 1つのバイナリに統合                |

### 3 Bun で何ができるようになった？



起動・実行がとにかく速い



パッケージ管理も内蔵



テストランナーも標準装備

## ひとことメモ

Node.js 互換を意識しているので、既存コードもだいたいそのまま動く。「速さ」で殴る系ツールの先駆け。

## 関連キーワード

比較する技術



### Node.js

JSをブラウザの外で動かす元祖ランタイム。

P.143



### Deno

安全性を重視したもう1つの後継ランタイム。

P.145

# Deno

Node.js を作った人が「作り直した」ランタイム

## ひとことという

デフォルトでは何もアクセスできず、TypeScript をそのまま実行できる安全志向の JS ランタイム。

## 関連キーワード

比較する技術



### Node.js

JSをブラウザの外で動かす元祖ランタイム。

P.143



### Bun

Node.js互換で速さに振り切ったランタイム。

P.144

## 1 なぜ生まれた？

Node.js は自由度が高い反面、npm パッケージが暗黙にファイルやネットワークへアクセスでき、安全性への不安があった。



## 2 それまでどうしていた？

Node.js では、パッケージは特に制限なくファイル・ネットワーク・環境変数へアクセスできていた。

|            | Node.js            | Deno                      |
|------------|--------------------|---------------------------|
| 権限         | デフォルトで全許可          | デフォルトで全拒否(明示許可制)          |
| TypeScript | 別途トランスパイル設定が必要     | 標準でそのまま実行可能               |
| パッケージ管理    | npm + node_modules | URL importも可能(npm 互換も追加済) |

## 3 Deno で何ができるようになった？



ファイル/ネット権限を明示指定



TypeScript をそのまま実行



Deno Deploy で手軽にデプロイ

## ひとことメモ

Node.js の作者 Ryan Dahl が「あの設計は失敗だった」と反省して作り直した、という差分エピソードの持ち主。

# React

画面を「部品」の組み合わせで作る発想を広めたライブラリ

## ひとことという

UIを再利用可能なコンポーネント単位で組み立てるためのJavaScriptライブラリ。

1

### なぜ生まれた？

素のJS・DOM操作で画面を作ると、状態が変わるたびにDOMを手動で書き換える処理が複雑化し、バグの温床になっていた。



2

### それまでどうしていた？

それまではjQueryなどでDOMを直接操作し、画面の状態を自分で追跡・更新していた。

|       | 素のJS / jQuery | React               |
|-------|---------------|---------------------|
| 更新方法  | DOMを直接書き換え    | 状態(State)からUIを再計算   |
| 再利用   | コピペで使い回し      | コンポーネントとして部品化       |
| 学習コスト | 低い            | JSX・Hooksなど独自の書き方あり |

3

### React で何ができるようになった？



画面をコンポーネント単位で管理



状態が変わると自動で再描画



Next.js など周辺エコシステムが豊富

## ひとことメモ

差分の核は「Stateが変わったらUIも自動で追従する」という宣言的UIの発想。それまでは全部手動だった。

## 関連キーワード

比較する技術



**Vue**

HTMLに近い書き方で始められるフレームワーク。

P.147

関連する技術



**Next.js**

Reactにルーティングなどを標準搭載したフレームワーク。

P.148



**Redux**

アプリ全体の状態を1箇所にまとめる状態管理。

P.163

# Vue

HTMLに近い書き味で始められる、もう1つの人気フレームワーク

## ひとことというと

テンプレート構文でHTMLに近い書き方をしながら、コンポーネント単位で画面を作れるフレームワーク。

## 関連キーワード

比較する技術



### React

状態からUIを再計算するコンポーネント指向ライブラリ。

P.146

関連する技術



### Nuxt.js

Vueにルーティングなどを標準搭載したフレームワーク。

P.149



### Pinia

Vue公式が推奨する状態管理ライブラリ。

P.165

## 1 なぜ生まれた？

React は強力だったが、JSXという独自記法の学習コストがあり、「もっとHTML/CSSに近い書き方で始めたい」という声があった。



## 2 それまでどうしていた？

React では、JSXの中にHTMLに似せたJS構文でUIを書いていた。

|          | React(JSX)         | Vue(テンプレート)           |
|----------|--------------------|-----------------------|
| 記法       | JS内にHTMLっぽい構文(JSX) | HTMLに<template>タグで分離  |
| 学習の入り口   | JSの知識が前提           | HTML/CSSからでも始めやすい     |
| 状態管理の書き方 | useStateなどのHooks   | ref / reactiveなど専用API |

## 3 Vue で何ができるようになった？



HTML/CSS/JS  
を分けて書ける



双方向バインディングが手軽



公式ドキュメントが読みやすい

## ひとことメモ

ReactとVueは「コンポーネント化」という同じ課題への別解、という差分の関係。

# Next.js

Reactだけでは決めきれない「ルーティング  
どうする問題」を解決したフレームワーク

## ひとことという

Reactにルーティングやサーバーサイドレンダリング  
などを標準搭載した、本番運用向けフレームワーク。

1

### なぜ生まれた？

Reactは画面部品を作る仕組みは提供するが、ページ遷移やSEO対策(SSR)は自分で組み合わせる必要があり、毎回同じ判断をしていた。



2

### それまでどうしていた？

Reactだけの構成では、ルーティングライブラリやSSR用サーバーを個別に用意していた。

|        | Reactのみ(素の構成) | Next.js              |
|--------|---------------|----------------------|
| ルーティング | 別ライブラリを追加・設定  | ファイル配置だけで自動生成        |
| 描画方式   | 基本CSRのみ       | SSR / SSG / ISRを選択可能 |
| 初期設定   | 自分で構成を決める     | フレームワークが規約として提供      |

3

### Next.js で何ができるようになった？



ファイルベースでルーティング自動化



SSR/SSGを標準サポート



Vercelへワンコマンドでデプロイ

## ひとことメモ

「Reactでアプリを作る時の面倒な意思決定」を肩代わりしてくれる、という差分。

## 関連キーワード

土台の技術



### React

コンポーネント単位でUIを組み立てるライブラリ。

P.146

比較する技術



### Nuxt.js

VueにとってのNext.js的な存在。

P.149

# Nuxt.js

Vueにとっての Next.js 的存在

## ひとことという

Vueにルーティングやサーバーサイドレンダリングなどを標準搭載した、本番運用向けフレームワーク。

### 1 なぜ生まれた？

Vueも同様に、ルーティングやSSRを個別に組み合わせる必要があり、プロジェクトごとに構成がバラつく問題があった。



### 2 それまでどうしていた？

Vueのみの構成では、vue-routerなどを個別に追加し、SSRも自前でサーバーを用意していた。

|        | Vueのみ(素の構成)     | Nuxt.js              |
|--------|-----------------|----------------------|
| ルーティング | vue-routerを個別設定 | ファイル配置だけで自動生成        |
| 描画方式   | 基本CSRのみ         | SSR / SSG / ISRを選択可能 |
| 設定ファイル | プロジェクトごとにバラバラ   | 規約で統一される             |

### 3 Nuxt.js で何ができるようになった？



ファイルベースでルーティング自動化



SSR/SSGを標準サポート



Vueのモジュールを組み込みやすい

## ひとことメモ

NextとNuxtは「Reactの流儀」「Vueの流儀」という差分のペア、と覚えておくと整理しやすい。

## 関連キーワード

土台の技術



### Vue

HTMLに近い書き味のコンポーネント指向フレームワーク。

P.147

比較する技術



### Next.js

ReactにとってのNuxt.js的な存在。

P.148

# Vite

ビルドを「待つ」時代を終わらせた開発サーバー

## ひとことという

ブラウザのESモジュール機能を使って、開発中の画面表示をほぼ一瞬で反映するビルドツール。

1

## なぜ生まれた？

従来のビルドツールは、ファイルを1つ書き換えるたびにプロジェクト全体をバンドルし直す必要があり、待ち時間が長かった。



2

## それまでどうしていた？

従来はWebpackなどでJS全体を事前にバンドルしてからブラウザに渡していた。

|        | Webpack等(バンドル型) | Vite                |
|--------|-----------------|---------------------|
| 開発時の起動 | 全体をバンドルしてから起動   | ESモジュールでそのまま配信      |
| 変更の反映  | 変更範囲によって再バンドル   | 変更ファイルだけ即時反映(高速HMR) |
| 本番ビルド  | 同じ仕組みでビルド       | Rollupで最適化ビルド       |

3

## Vite で何ができるようになった？



開発サーバーの起動が高速



保存した瞬間に画面へ反映



本番用ビルドは別途最適化

## ひとことメモ

「開発中の体感速度」に振り切ったツール、という差分。React/Vueどちらの開発でも定番になっている。

## 関連キーワード

関連する技術



### React

Viteの定番テンプレートの1つ。

P.146



### ESLint

開発時のコードチェックを担うLinter。

P.151

# ESLint

「動くけど危ないコード」を書いた瞬間に教えてくれる仕組み

## ひとことという

コードのルール違反やバグの芽を、実行する前に静的解析で指摘してくれるLinter。

## 関連キーワード

高速化を競う後継



### Biome

Lintと整形をRustで1本にまとめたツール。

P.152



### OXC

パーサーからRustで作り直すプロジェクト。

P.153

## 1 なぜ生まれた？

未使用変数や書き方のばらつき、うっかりバグにレビューまで気づけず、レビューコストや事故が増えていた。



## 2 それまでどうしていた？

それまでは、コードレビューで人力で指摘するか、規約を文書として共有するだけだった。

|            | 人カレビュー・規約文書   | ESLint          |
|------------|---------------|-----------------|
| チェックのタイミング | レビュー時(後追い)    | 保存・コミット時(即座)    |
| 基準のブレ      | レビュアーによって差が出る | ルールとして統一・自動化    |
| 対応範囲       | 見つけたものだけ      | プロジェクト全体を機械的に走査 |

## 3 ESLint で何ができるようになった？



ルール違反を自動検出



保存時に自動修正も可能



チーム全体でルールを統一

## ひとことメモ

「レビューで毎回同じ指摘をする」を無くすための仕組み。この後、速さを競う後継ツールが次々と生まれる。

# Biome

ESLintより速く、Prettierの仕事も1本でこなすツール

## ひとことという

Lint(コードチェック)とFormat(整形)をRust製の1つのツールでまとめて高速に行うツール。

1

## なぜ生まれた?

ESLintとPrettierを両方使うのが定番だったが、設定ファイルが2つになり、プロジェクトが大きくなるほど実行が遅くなっていた。



2

## それまでどうしていた?

従来はESLint(Lint用)とPrettier(整形用)をそれぞれJavaScriptで実装し、組み合わせて使っていた。

|        | ESLint + Prettier<br>(組み合わせ) | Biome          |
|--------|------------------------------|----------------|
| 役割     | Lintと整形を別ツールで担当              | 1つのツールでLint+整形 |
| 実装言語   | JavaScript                   | Rust(高速)       |
| 設定ファイル | 2つ(.eslintrc / .prettierrc)  | 1つに統一          |

3

## Biome で何ができるようになった?



Lint・整形が高速



1 ツールで設定が完結



ESLint設定からの移行支援あり

## ひとことメモ

「速さ」で殴る系ツールの1つ。  
この流れはさらにOXCへと続いていく。

## 関連キーワード

比較する技術



### ESLint

JS実装の老舗 Linter。

P.151



### OXC

パース処理から Rustで高速化する次の一手。

P.153

# OXC

JSツールチェーン全部をRustで書き直す、  
という荒業

## ひとことというと

パーサー・Linter・整形・ビルドまで、JS周辺ツールを  
まとめてRustで高速に作り直すプロジェクト。

### 1 なぜ生まれた？

BiomeでLint+整形は速くなったが、パース(コード解析)自体やビルドまで含めた全体は、まだJS実装のツールが多く残っていた。



### 2 それまでどうしていた？

それまでは、Lintと整形はBiomeやESLintのように個別最適化され、パーサーやバンドラーは各ツールがバラバラに実装していた。

|      | JS実装のツール群(個別最適化) | OXC                  |
|------|------------------|----------------------|
| 対象範囲 | Lint / 整形など個別ツール | パーサー〜ビルドまで一貫してRust化  |
| 速度   | ツールごとに差がある       | 全体を通してネイティブ速度        |
| 採用状況 | 各社が個別に高速化を試す     | Viteなど他ツールへの組み込みが進行中 |

### 3 OXC で何ができるようになった？



パース処理から  
ネイティブ高速化



他ツールへの組み  
込みが進行中



JSツールチェーン  
全体を刷新

## ひとことメモ

ESLint → Biome → OXCと続く「Lintツールの高速化競争」の  
最新章。次はどこまで速くなるか。

## 関連キーワード

高速化競争のはじまり



### ESLint

JS実装の老舗  
Linter。

P.151



### Biome

Rustで1本化した  
中継地点。

P.152

# Tailwind CSS

「クラス名を考える時間」を無くしたCSSの書き方

## ひとことという

あらかじめ用意された小さなユーティリティクラスをHTMLに直接並べてスタイルを作るCSSフレームワーク。

### 1 なぜ生まれた？

CSSでは、要素ごとに意味のあるクラス名を考えて別ファイルにスタイルを書く必要があり、命名や設計に時間がかっていた。



### 2 それまでどうしていた？

それまでは、独自クラス名を都度設計するCSS設計(BEMなど)で対応していた。

|        | 独自CSS設計 / BEMなど  | Tailwind CSS         |
|--------|------------------|----------------------|
| 書く場所   | 別ファイルにCSSを記述     | HTMLのclass属性に直接記述    |
| 命名     | 要素ごとに意味のある名前を考える | 用意されたユーティリティ名を組み合わせる |
| 見た目の変更 | CSSファイルを探して修正    | HTML上でクラスを足し引き       |

### 3 Tailwind CSS で何ができるようになった？



クラス名を考えなくてよい



HTML上で見た目が完結



未使用クラスはビルド時に自動除去

## ひとことメモ

「CSS設計 vs ユーティリティファースト」という思想の差分。賛否ある分だけ広く使われている。

## 関連キーワード

組み合わせる技術



### MUI

既製のUI部品ごと使えるコンポーネント集。

P.155



### shadcn/ui

コードをコピーして使うUI集。

P.156

ボタンやフォームを1から作らなくていい、部品まるごとの詰め合わせ

ひとことというと

Googleのマテリアルデザインに沿ったボタン・フォームなどのReact用UIコンポーネント集。

1 なぜ生まれた？

Tailwind CSSでスタイルを当てるのは自由だが、ボタンやモーダルなど「よくある部品」を毎回1から組み立てるのは手間だった。



2 それまでどうしていた？

それまでは、Tailwindなどでスタイルだけを当てつつ、UI部品自体は自分でHTML構造から組み立てていた。

|         | 自前でUI部品を実装   | MUI                |
|---------|--------------|--------------------|
| 部品の中身   | 都度HTML構造から実装 | ボタン・モーダル等を既製品として提供 |
| 見た目の統一感 | 都度デザイン判断が必要  | マテリアルデザインの基準に沿う    |
| カスタマイズ  | 自由(だが手間)     | テーマ機能で配色などを調整      |

3 MUI で何ができるようになった？



既製のUI部品を組み込むだけ



マテリアルデザインで統一感



テーマ設定で配色を一括変更

ひとことメモ

Tailwindが「素材」ならMUIは「完成品の部品棚」、という差分の対比で覚えるとわかりやすい。

関連キーワード

比較する技術



Tailwind CSS

ユーティリティクラスでスタイルを組み立てる。

P.154



shadcn/ui

コードをコピーして使う別解のUI集。

P.156

# shadcn/ui

「ライブラリ」ではなく「コピペするコード」という新発想のUI集

## ひとことという

MUIのような既製部品ライブラリではなく、必要なだけコードをプロジェクトにコピーして使うUIコンポーネント集。

1

## なぜ生まれた？

MUIのようなライブラリは便利だが、内部の見た目を細かくカスタムしづらく、ライブラリの更新にも振り回されがちだった。



2

## それまでどうしていた？

MUIなどでは、npmでライブラリを追加し、用意されたコンポーネントをそのままimportして使っていた。

|        | MUI等(ライブラリ追加型)      | shadcn/ui              |
|--------|---------------------|------------------------|
| 導入方法   | npm installでライブラリ追加 | CLIでソースコードごとプロジェクトへコピー |
| カスタマイズ | ライブラリの制約内で調整        | コードを直接編集して自由に変更        |
| 依存関係   | ライブラリ本体に依存し続ける      | コピー後は自分のコードとして管理       |

3

## shadcn/ui で何ができるようになった？



コードを直接プロジェクトへコピー



見た目を細部まで自由に編集



Tailwind CSSと組み合わせて使う

## ひとことメモ

「ライブラリを入れる」から「お手本コードをもらう」への発想転換、という差分。

## 関連キーワード

比較する技術



### MUI

既製部品をライブラリとして利用する方法。

P.155



### Tailwind CSS

見た目を組み立てるユーティリティCSS。

P.154

# Electron

Webの技術だけでデスクトップアプリを作る発想

## ひとことという

HTML・CSS・JavaScriptで書いた画面を、そのままWindows/Mac用のデスクトップアプリとして動かす仕組み。

## 関連キーワード

比較する技術



### PWA

ストア審査なしでアプリっぽさを出す別解。

P.158

## 1 なぜ生まれた？

デスクトップアプリを作るには、OSごとに別の言語・UI部品を覚えて実装する必要があり、コストが高かった。



## 2 それまでどうしていた？

それまでは、C++/C#/SwiftなどOSごとのネイティブ言語・フレームワークで別々に実装していた。

|      | ネイティブ実装(OSごと)       | Electron                     |
|------|---------------------|------------------------------|
| 言語   | C++ / C# / Swift など | HTML / CSS / JavaScript      |
| 対応OS | OSごとに別実装が必要         | 1つのコードで複数OSに対応               |
| 代表例  | —                   | VS Code / Slack / Discord など |

## 3 Electron で何ができるようになった？



1つのコードで複数OSに対応



Web技術の知識をそのまま流用



ブラウザ機能ごとアプリに同梱

## ひとことメモ

アプリ内にChromiumを丸ごと同梱するため、動作は重めになりがち、というのが差分の裏側。

# PWA

アプリストアを経由せず、Webサイトを「アプリっぽく」使わせる仕組み

## ひとことというと

ホーム画面への追加やオフライン動作など、通常のWebサイトにネイティブアプリに近い体験を持たせる技術。

### 1 なぜ生まれた？

ネイティブアプリはストア審査やインストールの手間があり、「開くだけで使えて、でもアプリっぽい体験も欲しい」という要望があった。



### 2 それまでどうしていた？

それまでは、ネイティブアプリ(iOS/Android別)かElectronのようなデスクトップ包装で「アプリらしさ」を実現していた。

|         | ネイティブアプリ / Electron | PWA               |
|---------|---------------------|-------------------|
| 配布方法    | ストア経由 / インストーラー配布   | URLを開くだけ・ホーム画面に追加 |
| オフライン対応 | 各プラットフォームの機能で実装     | Service Workerで実現 |
| 審査      | ストア審査が必要な場合あり       | 審査不要(Webの延長)      |

### 3 PWA で何ができるようになった？



ホーム画面に追加できる



オフラインでも一部動作



ストア審査なしで配信・更新

## ひとことメモ

「インストールの壁を越えずにアプリっぽさを出す」という差分の落とし所。

## 関連キーワード

比較する技術



### Electron

Web技術でデスクトップアプリを作る仕組み。

P.157

関連する指標



### Core Web Vitals

ページの体感品質を数値化する指標。

P.162

# WebAssembly

ブラウザで JavaScript 以外の言語を高速に動かす仕組み

## ひとことという

C/C++/Rustなどで書いたコードをブラウザ上でネイティブに近い速度で実行できるバイナリ形式。

## 関連キーワード

関連する技術



### Core Web Vitals

速度も含めたページ品質を測る指標。

P.162

## 1 なぜ生まれた？

画像・動画処理やゲームなど重い処理をJavaScriptだけで行うと速度が足りず、ブラウザでは実現しづらい用途があった。



## 2 それまでどうしていた？

それまでは、重い処理はサーバー側で行うか、JavaScriptを愚直にチューニングして何とか動かしていた。

|       | JS 実行のみ / サーバーで処理    | WebAssembly                      |
|-------|----------------------|----------------------------------|
| 実行速度  | JS エンジン 依存で限界がある     | ネイティブに近い速度                       |
| 書ける言語 | JavaScript (派生含む) のみ | C / C++ / Rust / Go などをコンパイルして実行 |
| 用途例   | —                    | 画像編集・ゲーム・動画処理など                  |

## 3 WebAssembly で何ができるようになった？



重い処理をブラウザ内で高速実行



JS と連携して部分的に利用可能



C/Rust 資産を Web に持ち込める

### ひとことメモ

「JS を速くする」のではなく「JS 以外を持ち込む」という発想の差分。

# a11y(アクセシビリティ)

「使える人」を無意識に絞り込んでいないか、  
を問い直す視点

## ひとことという

視覚・聴覚・身体的な違いに関わらず、誰もが  
Webサイトを使えるようにする設計・実装の総称。

1

## なぜ生まれた？

見た目や動作の実装ばかりに気を取られ、スクリーンリーダーや  
キーボード操作だけで使うユーザーへの配慮が後  
回しになっていた。



2

## それまでどうしていた？

それまでは「動けばOK」「見た目が良ければOK」という基準で実装され、a11y  
は余力があれば対応する程度だった。

|        | 見た目・動作優先の実装        | a11y対応の実装              |
|--------|--------------------|------------------------|
| 対象ユーザー | 標準的な操作ができる<br>人が前提 | 視覚/聴覚/身体的な違いがある人も含む    |
| チェック項目 | デザイン通りに動くか         | altテキスト・コントラスト比・操作性も確認 |
| 評価軸    | 見た目の完成度            | WCAGなどの基準への準拠度         |

3

## a11y(アクセシビリティ) で何ができるようになった？



誰でも使える設計  
を評価軸に追加



altテキストやラベル  
を適切に付与



キーボードのみで  
も操作可能に

## ひとことメモ

160アクセシビリティ対応は追加機能ではなく、  
「そもそも誰が使えなかったか」を問い直す作業、という差分。

## 関連キーワード

関連する語



i18n(国際化)

言語や地域による  
差分を吸収する対応。

P.161



Core Web Vitals

速度など体感品質  
を測る指標。

P.162

# i18n(国際化)

「日本語しか出ない」から卒業するための下ごしらえ

## ひとことというと

アプリの文言や日付・通貨の表記を、言語や地域ごとに切り替えられるようにする仕組み・対応のこと。

## 関連キーワード

関連する語



a11y(アクセシビリティ)

誰もが使える設計にするための視点。

P.160

## 1 なぜ生まれた？

最初は日本語だけを想定して文言をコードに直接書いていたが、海外展開が必要になり、全箇所を洗い出す羽目になった。



## 2 それまでどうしていた？

それまでは、表示文言をコンポーネントの中に直接ハードコーディングしていた。

|         | 文言を直接ハードコーディング | i18n対応         |
|---------|----------------|----------------|
| 文言の管理   | コード内に直接記述      | 言語ごとの辞書ファイルに分離 |
| 言語切り替え  | 対応不可(作り直しが必要)  | 辞書を差し替えるだけで対応  |
| 日付・通貨表記 | 決め打ちの書式        | ロケールに応じて自動整形   |

## 3 i18n(国際化) で何ができるようになった？



言語ごとに辞書ファイルを切り替え



日付・通貨をロケール対応で表示



後から言語追加がしやすい

## ひとことメモ

「後からやろう」が一番コストが膨らみやすい対応の1つ、という差分の教訓。

# Core Web Vitals

「なんとなく重い」を数値で説明できるようにした指標

## ひとことという

表示速度・操作反応・レイアウトのガタつきなど、Webページの体感品質を3つの指標で数値化したもの。

### 1 なぜ生まれた？

「サイトが遅い気がする」「レイアウトがガクッと動く」といった体感を、チーム内でうまく共有・比較できずにいた。



### 2 それまでどうしていた？

それまでは、ページ速度は主観的な感想や、ツールごとにバラバラな独自指標で語られていた。

|         | 主観的な感想 / 独自指標 | Core Web Vitals            |
|---------|---------------|----------------------------|
| 評価方法    | 「なんか遅い」等の感覚   | LCP / INP / CLSという共通指標で数値化 |
| 比較のしやすさ | ツール・人によってバラバラ | 業界標準の指標で横並び比較可能            |
| SEOとの関係 | 明確な関連は薄かった    | 検索順位の評価要素の1つに              |

### 3 Core Web Vitals で何ができるようになった？



表示速度をLCPで数値化



操作の反応速度をINPで測定



レイアウトのガタつきをCLSで測定

## ひとことメモ

「速い/遅い」の水掛け論を、共通の物差しで終わらせるための指標。

## 関連キーワード

関連する語



### PWA

アプリっぽい体験を実現する技術。

P.158



### a11y(アクセシビリティ)

誰もが使える設計にするための視点。

P.160

# 163 | Redux

「どのコンポーネントの状態が正しいんだっけ」を終わらせた仕組み

## ひとことというと

アプリ全体の状態 (State) を 1 箇所にまとめて管理する、React 向けの状態管理ライブラリ。

### 1 なぜ生まれた？

画面が複雑になるにつれ、Props 経由で何段もコンポーネントを跨いで状態を渡す「バケツリレー」が発生し、変更箇所を追えなくなっていた。



### 2 それまでどうしていた？

それまでは、React の標準機能 (useState/Props) だけで、必要な状態を親から子へ渡していた。

|         | useState + Props<br>のバケツリレー | Redux                       |
|---------|-----------------------------|-----------------------------|
| 状態の置き場所 | 各コンポーネントに分散                 | ストアという 1 箇所に集約              |
| 更新方法    | 各コンポーネントで直接更新               | Action/Reducer という決まった手順で更新 |
| 追跡のしやすさ | どこで変わったか追にくい                | 変更履歴を 1 本の流れとして追跡可能         |

### 3 Redux で何ができるようになった？



状態を 1 箇所のストアに集約



離れたコンポーネント間でも状態共有



決まった手順でしか状態を変更できない

## ひとことメモ

自由度を下げる代わりに「予測可能性」を手に入れた、という差分のトレードオフ。

## 関連キーワード

土台の技術



### React

コンポーネント単位でUIを組み立てるライブラリ。

P.146

軽量な後継



### Zustand

手順の少ない軽量な状態管理。

P.164

# Zustand

Reduxの「お作法の多さ」に疲れた人たちが  
選んだ軽量な状態管理

## ひとことというと

Action/Reducerのような決まった手順を省き、少ないコードでReactの状態を共有できる軽量な状態管理ライブラリ。

### 1 なぜ生まれた？

Reduxは状態を予測可能に保てる一方、覚えることや書く量が多く、小〜中規模の画面にはオーバースペックに感じられていた。



### 2 それまでどうしていた？

それまでは、状態管理といえばRedux一択で、小さな機能追加にもRedux一式の手順を書いていた。

|         | Redux(手順が多い)                         | Zustand            |
|---------|--------------------------------------|--------------------|
| 書く量     | Action/Reducer/Dispatchの3つの関数でストアを定義 | 素のJSの感覚でストアを定義するだけ |
| 学習コスト   | 専用の設計思想を理解する必要                       | 素のJSの感覚に近く読み書きしやすい |
| 向いている規模 | 大規模・チーム開発向け                          | 小〜中規模でも導入しやすい      |

### 3 Zustandで何ができるようになった？



少ないコードで状態を共有



覚えることが少ない



必要な機能だけ後から追加可能

## ひとことメモ

「Reduxほど厳格でなくていい」という現場の本音から生まれた選択肢、という差分。

## 関連キーワード

比較する技術



### Redux

手順が多いが予測可能性の高い状態管理。

P.163



### Pinia

Vue版の似た立ち位置の状態管理。

P.165

# Pinia

Vue 公式が選んだ、Vuex に代わる状態管理

## ひとことという

Vue 公式が推奨する、TypeScript との相性やシンプルさを重視した状態管理ライブラリ。

## 関連キーワード

土台の技術



### Vue

HTML に近い書き味のコンポーネント指向フレームワーク。

P.147

比較する技術



### Zustand

React 版の似た立ち位置の状態管理。

P.164

## 1 なぜ生まれた？

Vue の状態管理は Vuex が定番だったが、TypeScript との相性の悪さや Mutation/Action の分離など、手順の多さが負担になっていた。



## 2 それまでどうしていた？

それまでは、Vuex という公式の状態管理ライブラリで Mutation・Action を分けて状態を更新していた。

|               | Vuex<br>(Mutation/Action<br>分離) | Pinia               |
|---------------|---------------------------------|---------------------|
| 更新の手順         | Mutation と Action を分けて定義        | Action だけで状態を直接更新可能 |
| TypeScript 対応 | 型推論が弱く相性が悪い                     | 型推論が効きやすい設計         |
| 公式の立ち位置       | 長らく公式推奨だった                      | 現在の公式推奨はこちら         |

## 3 Pinia で何ができるようになった？



ストアをシンプルな構文で定義



型推論が効いて書きやすい



複数ストアに分割して管理しやすい

### ひとことメモ

React の Redux → Zustand、Vue の Vuex → Pinia という「シンプル化」の流れは共通、という差分のまとめ。

# 171

# Ruby

「人間が読みやすい」を設計思想の中心に置いた言語

## ひとことという

書きやすさ・読みやすさを最優先に設計された、Ruby on Railsの生みの親でもあるプログラミング言語。

1

## なぜ生まれた？

Perlなど既存の言語は便利だが、人間の感情に寄り添っていないと感じたまつもとゆきひろ氏が、「プログラマが幸せになる言語」を目指して開発した。



2

## それまでどうしていた？

それまではCやPerlなど、書きやすさより実行効率や記号的な簡潔さを優先する言語が主流だった。

|          | C / Perlなど    | Ruby        |
|----------|---------------|-------------|
| 設計の軸     | 実行速度・記号の簡潔さ   | 書きやすさ・読みやすさ |
| 文法       | 記号中心          | 英語に近い自然な文法  |
| オブジェクト指向 | 言語による         | 全てがオブジェクト   |
| 向いている場面  | システムプログラミングなど | Web開発・スクリプト |

3

## Ruby で何ができるようになった？



書きやすいコード  
が書ける



Webアプリを素早  
く開発できる



豊富なgemで機  
能を拡張できる

## ひとことメモ

Rubyは「人間にとっての読みやすさ」を最優先にした言語。同じ処理でも複数の書き方が許されているのは、この思想の表れ。

## 関連キーワード

関連する技術



### Rails

RubyのWebフレームワーク。設定より規約の思想で作られている。

P.177

比較する言語



### Python

同じく書きやすさを重視するスクリプト言語。

P.172

# 172 Python

シンプルな文法で、Web 開発から AI・データ分析まで橋渡しする言語

## ひとことという

インデントで構造を表す簡潔な文法を持ち、Web 開発・データ分析・機械学習まで幅広く使われる言語。

### 1 なぜ生まれた？

「読みやすいコードこそ書きやすいコード」という思想のもと、Guido van Rossum が誰にでも読める言語を目指して開発した。



### 2 それまでどうしていた？

それまでは {} や end でブロックを表す言語が主流で、書き手によってコードの見た目がバラバラになりがちだった。

|         | 他の主要言語<br>(Java/C++ など) | Python         |
|---------|-------------------------|----------------|
| ブロックの表現 | { } や end               | インデント (強制)     |
| 文法量     | 多い・冗長になりがち              | 少ない・簡潔         |
| 用途の広さ   | 得意分野が限定的                | Web・AI・自動化まで横断 |
| 学習コスト   | 高め                      | 低め (初学者にも人気)   |

### 3 Python で何ができるようになった？



シンプルな文法で  
書ける



AI・データ分析に  
も使える



ライブラリが豊富

## ひとことメモ

Python は Ruby と同じく「書きやすさ」を重視するが、インデントを文法として強制する点が違う。  
どのファイルを見てもコードの見た目が揃うのはこのため。

## 関連キーワード

関連する技術



### Django

電池付属の思想を持つフルスタックフレームワーク。

P.180



### FastAPI

型ヒントを活かした高速な API フレームワーク。

P.181



### Flask

必要な機能だけを足していく軽量フレームワーク。

P.182

比較する言語



### Ruby

同じく書きやすさを重視するスクリプト言語。

P.171

# Go

Googleが「大規模開発のシンプルさ」のために作った言語

## ひとことという

並行処理を言語レベルでサポートし、シンプルな文法と高速な実行を両立する Google 製の言語。

1

### なぜ生まれた？

Googleの大規模なコードベースで、C++の複雑なビルドやJavaの冗長さに悩まされていたエンジニアたちが、「シンプルに書いて、速く動く」言語を目指して開発した。



2

### それまでどうしていた？

それまではC++やJavaで並行処理を書く場合、スレッドやロックを自前で細かく管理する必要があった。

|          | C++ / Java      | Go               |
|----------|-----------------|------------------|
| 並行処理の書き方 | スレッド・ロックを自前管理   | goroutineで軽量に書ける |
| 文法の量     | 多機能・複雑になりがち     | 意図的にシンプル         |
| コンパイル    | 遅い・セットアップが重い場合も | 高速コンパイル・単一バイナリ   |
| 向いている場面  | 大規模アプリ全般        | インフラ・API・CLIツール  |

3

### Go で何ができるようになった？



高速に動作する



並行処理を手軽に書ける



単一バイナリで配布できる

## ひとことメモ

Goは「書けることを増やす」より「書けることを絞る」方向に振り切った言語。選択肢を減らすことで、誰が書いても読みやすいコードになる。

## 関連キーワード

関連する技術



### Gin

Goの速さを活かしたシンプルなWebフレームワーク。

P.183

比較する言語



### Rust

速度に加えてメモリ安全性をコンパイル時に保証する言語。

P.176

「Web ページに動きをつける」ために生まれ、  
今も Web を支える言語

## ひとことという

HTML に埋め込んで使える手軽さから広まり、大規模な Web サービスの基盤にもなっているサーバーサイド言語。

## 関連キーワード

関連する技術



### Laravel

PHP をモダンにしたフルスタックフレームワーク。

P.184

比較する言語



### Ruby

同じく書きやすさを軸にしたスクリプト言語。

P.171

## 1 なぜ生まれた？

個人サイトのアクセス数を数える簡単なツールとして誕生し、「HTML に直接埋め込んでサーバーサイドの処理を書ける」  
手軽さから、Web 専用言語として発展していった。



## 2 それまでどうしていた？

それまでの Web ページは CGI として C 言語や Perl スクリプトを別途用意し、HTML と処理を分けて書く必要があった。

|           | CGI (C/Perl など) | PHP                |
|-----------|-----------------|--------------------|
| HTML との関係 | 別ファイル・別プロセスで連携  | HTML に直接埋め込む       |
| 導入の手軽さ    | サーバー設定が煩雑       | レンタルサーバーでもすぐ動く     |
| 学習コスト     | 高め              | 低め (Web 特化で分かりやすい) |
| 向いている場面   | 汎用的な処理全般        | 動的な Web ページ生成      |

## 3 PHP で何ができるようになった？



HTML に処理を埋め込める



手軽に Web サーバーを動かせる



CMS・EC の基盤として定番

## ひとことメモ

PHP は「まず Web ページを動かす」ことに全振りした言語。  
WordPress をはじめ、今も世界中の Web サイトを裏で支えている。

# Java

「どこでも同じように動く」を掲げ、大規模開発の標準になった言語

## ひとことというと

JVM上で動作し、書いたコードをOSを問わず動かせることを武器に、企業システムの定番であり続ける言語。

### 1 なぜ生まれた？

家電向けの組み込み言語として開発が始まったが、「一度書けば、どこでも動く (Write Once, Run Anywhere)」という思想がWeb・業務システムの世界に刺さり、企業システムの標準言語として広まった。



### 2 それまでどうしていた？

それまではCやC++でOSごとにコンパイルし直す必要があり、動作環境を変えるたびに移植作業が発生していた。

|         | C / C++        | Java            |
|---------|----------------|-----------------|
| 動作環境    | OSごとに再コンパイルが必要 | JVM上でOSを問わず動く   |
| メモリ管理   | 手動 (自分で解放)     | 自動 (ガベージコレクション) |
| 型の厳しさ   | 言語による          | 静的型付けで厳格        |
| 向いている場面 | OS・ハードウェア寄りの処理 | 大規模・長期運用の業務システム |

### 3 Java で何ができるようになった？



OSを問わず動かせる



大規模開発でも破綻しにくい



企業システムの定番として実績豊富

## ひとことメモ

Javaは「一度書けば、どこでも動く」を実現するためにJVMという実行環境を挟んでいる。  
冗長と言われつつも、この安定感が今も選ばれ続ける理由。

## 関連キーワード

関連する技術



### Spring Boot

Javaの設定を自動化し、すぐに動くアプリを作れるフレームワーク。

P.185

比較する言語



### Go

シンプルさと並行処理のしやすさを重視した言語。

P.173

# Rust

「速さ」と「安全」を、実行時ではなくコンパイル時に両立させる言語

## ひとことという

メモリ管理の安全性をコンパイラがチェックすることで、C/C++並みの速度と高い安全性を両立する言語。

## 関連キーワード

比較する言語



Go

シンプルさと開発速度を重視した言語。

P.173

### 1 なぜ生まれた？

C/C++は高速だが、メモリの解放忘れや二重解放といったバグが実行時にクラッシュや脆弱性を引き起こしやすかった。  
この「速いが危ない」を解消するため、Mozillaが開発した。



### 2 それまでどうしていた？

それまでのC/C++では、メモリの確保・解放をプログラマが手動で管理し、ミスがあっても実行するまで気づけないことが多かった。

|         | C / C++        | Rust               |
|---------|----------------|--------------------|
| メモリ管理   | 手動(解放忘れのリスクあり) | コンパイラが所有権をチェック     |
| バグの発見   | 実行時にクラッシュして判明  | コンパイル時にエラーとして検出    |
| 実行速度    | 速い             | 同等に速い              |
| 向いている場面 | OS・組み込み・ゲームなど  | 同上+ WebAssembly など |

### 3 Rust で何ができるようになった？



安全性と速度を両立できる



メモリ安全性をコンパイル時に保証



C/C++並みの実行速度

#### ひとことメモ

Rustの「所有権」という仕組みは、書くときは厳しく感じるが、その分「動いてしまえばまず落ちない」コードになる。

# Rails

「設定より規約」で、Web アプリ開発の定番の型を作ったフレームワーク

## ひとことという

Ruby 製の Web フレームワーク。「設定より規約 (CoC)」の思想で、少ないコードで Web アプリを組み立てられる。

1

## なぜ生まれた？

Web アプリを作るたびに、ルーティングや DB 接続など同じような設定を毎回書く必要があった。この繰り返しをなくすため、DHH (デイビッド・ハイネマイヤー・ハンソン) が Ruby の表現力を活かして開発した。



2

## それまでどうしていた？

それまでの Web アプリ開発では、フレームワークごとに細かい設定ファイルを書き、命名やディレクトリ構成も一から決める必要があった。

|         | 設定ベースのフレームワーク | Rails                 |
|---------|---------------|-----------------------|
| 開発方針    | 設定ファイルで細かく指定  | 規約に従えば設定不要 (CoC)      |
| DB 操作   | SQL を都度記述     | ActiveRecord で直感的に操作  |
| 初期構築    | 構成をゼロから設計     | rails new で即座に土台が完成   |
| 向いている場面 | 自由な設計が必要な場合   | 素早く Web サービスを立ち上げたい場合 |

3

## Rails で何ができるようになった？



短期間で Web アプリを構築できる



MVC で役割を整理できる



gem で機能を簡単に追加できる

## ひとことメモ

Rails の「設定より規約」は、迷う時間を減らすための思想。決まったやり方に乗れば、あとはビジネスロジックに集中できる。

## 関連キーワード

関連する技術



### Ruby

書きやすさ・読みやすさを重視するプログラミング言語。

P.171

比較するフレームワーク



### Laravel

PHP 版のフルスタックフレームワーク。

P.184



### Django

Python 版のフルスタックフレームワーク。

P.180

# Express

Node.js に最小限の骨組みだけを足した、  
軽量 Web フレームワーク

## ひとことという

Node.js 上で動く、必要なものを自分で組み合わせるミニマルな Web フレームワーク。

## 関連キーワード

比較するフレームワーク



### Hono

Edge環境向けに  
生まれた、より軽量  
なフレームワーク。

P.179

## 1 なぜ生まれた？

Node.js だけで Web サーバーを書こうとすると、ルーティングやリクエスト処理を毎回自前で実装する必要があった。  
この定型作業を薄いレイヤーで肩代わりするため  
に開発された。



## 2 それまでどうしていた？

それまでは Node.js の標準モジュール (http) だけを使い、ルーティングなどを自分でゴリゴリ実装する必要があった。

|         | Node.js 標準 (http<br>モジュール) | Express            |
|---------|----------------------------|--------------------|
| ルーティング  | 自前で分岐処理を実装                 | app.get() などで簡潔に記述 |
| 機能の追加   | 全て自作                       | ミドルウェアを差し込むだけ      |
| 構成の自由度  | 低い (そもそも骨組みがない)            | 高い (薄いので好きに組める)    |
| 向いている場面 | 極小のスクリプト                   | API・軽量な Web サーバー   |

## 3 Express で何ができるようになった？



最小限の記述で  
API を作れる



ミドルウェアで機  
能を拡張できる



構成を自由に設計  
できる

### ひとことメモ

Express は「機能を持たせすぎない」ことが強み。  
薄いからこそ、必要なライブラリだけを自分で組み合わせられる。

# Hono

Cloudflare WorkersなどのEdge環境  
向けに生まれた、超軽量フレームワーク

## ひとことという

TypeScriptで書かれ、Node.js以外のEdgeランタイムでも高速に動く軽量Webフレームワーク。

1

### なぜ生まれた？

Cloudflare WorkersのようなEdge環境ではNode.js前提のExpressがそのまま動かず、実行時間の制約も厳しい。  
この環境でも軽く速く動くフレームワークとして開発された。



2

### それまでどうしていた？

それまでExpressのようなフレームワークはNode.js環境を前提しており、Edgeランタイムでは動作しないか、動いても重かった。

|         | Express (Node.js 前提) | Hono                              |
|---------|----------------------|-----------------------------------|
| 動作環境    | Node.js 前提           | Cloudflare Workers<br>等マルチランタイム対応 |
| 起動の軽さ   | 比較的重い                | 超軽量・高速起動                          |
| 型の扱い    | JavaScriptが中心        | TypeScriptファースト                   |
| 向いている場面 | 通常のサーバー              | Edge・サーバーレス環境                     |

3

### Hono で何ができるようになった？



Edge環境でも高速に動く



起動が軽くレスポンスが速い



複数のランタイムで動かせる

## ひとことメモ

Honoは「どこでも同じように速く動く」ことを目指したフレームワーク。  
Node.js専用ではない設計が、Edgeコンピューティング時代に合っている。

## 関連キーワード

比較するフレームワーク



### Express

Node.js向けの定番の軽量フレームワーク。

P.178

# Django

「電池付属」でWebアプリに必要な機能をひとまとめにしたフレームワーク

## ひとことという

認証・管理画面・ORMなど、Webアプリに必要な機能を標準で備えたPython製のフルスタックフレームワーク。

### 1 なぜ生まれた？

新聞社のニュースサイトを短納期で量産する必要があり、認証や管理画面など「毎回必要になる機能」を一つ一つ自作していた間に合わなかった。この課題から、必要な機能を最初から揃えた「電池付属 (batteries included)」の思想で開発された。



### 2 それまでどうしていた？

それまでのPython製Webフレームワークは、ルーティングなど最小限の機能しか持たず、認証や管理画面は別途ライブラリを探して組み合わせる必要があった。

|         | 軽量フレームワーク<br>(Flask など) | Django                    |
|---------|-------------------------|---------------------------|
| 付属機能    | 最小限 (必要なら自分で追加)         | 認証・管理画面・ORM<br>など標準搭載     |
| 管理画面    | 自作が必要                   | 自動生成される<br>(Django Admin) |
| 構成の自由度  | 高い                      | やや決まった型に沿う                |
| 向いている場面 | 小規模・独自構成のAPI            | 機能が多い業務系Webアプリ            |

### 3 Django で何ができるようになった？



必要な機能が最初から揃っている



管理画面が自動で手に入る



大規模なWebアプリを素早く構築できる

## ひとことメモ

Djangoの「電池付属」は、必要なものを探し回る時間をなくす思想。  
180 だった差が、Djangoにある機能を使いこなしで進められる。

## 関連キーワード

関連する技術



### Python

シンプルな文法を持つプログラミング言語。

P.172

比較するフレームワーク



### FastAPI

型ヒントを活かした高速なAPIフレームワーク。

P.181



### Flask

必要な機能だけを足していく軽量フレームワーク。

P.182



### Rails

Ruby版のフルスタックフレームワーク。

P.177

# FastAPI

型ヒントを書くだけでAPI仕様書まで自動生成される、モダンなAPIフレームワーク

## ひとことという

Pythonの型ヒントを使って、高速なAPIサーバーとドキュメントを同時に生成できるフレームワーク。

1

## なぜ生まれた？

既存のPython製フレームワークはAPIドキュメントを手動で書くか、別ツールで管理する必要があり、コードと仕様書がずれがちだった。型ヒントから自動でAPI仕様書(OpenAPI)を生成できるフレームワークとして開発された。



2

## それまでどうしていた？

それまでのPython製フレームワークでは、型チェックやAPI仕様書は別のライブラリを組み合わせで用意する必要があった。

|           | Flask など(型・仕様書は別途) | FastAPI         |
|-----------|--------------------|-----------------|
| 型ヒント      | 任意(チェックは別ツール)      | 必須に近く、そのまま検証に使う |
| APIドキュメント | 手動で作成・管理           | 型ヒントから自動生成      |
| 実行速度      | 標準的                | 非同期処理で高速        |
| 向いている場面   | 幅広いWebアプリ          | API・マイクロサービス    |

3

## FastAPI で何ができるようになった？



非同期処理で高速に動く



API仕様書が自動で手に入る



型ヒントでバグを早期発見できる

## ひとことメモ

FastAPIは「型ヒント」を単なる補助情報ではなく、バリデーションとドキュメント生成の入力として使い倒しているのが特徴。

## 関連キーワード

関連する技術



### Python

シンプルな文法を持つプログラミング言語。

P.172

比較するフレームワーク



### Django

電池付属の思想を持つフルスタックフレームワーク。

P.180



### Flask

必要な機能だけを足していく軽量フレームワーク。

P.182

# Flask

必要な部品だけを自分で足していく、  
Python製の軽量フレームワーク

## ひとことというと

最小限の機能だけを持ち、必要なライブラリを自分で  
組み合わせて育てていくPython製の軽量フレームワ  
ーク。

### 1 なぜ生まれた？

Djangoのようなフルスタックのフレームワークは  
機能が多く、小さなアプリやプロトタイプには  
重すぎると感じる開発者がいた。最小限の機能だけ  
を持つフレームワークとして開発された。



### 2 それまでどうしていた？

それまでのフルスタックフレームワーク(Djangoなど)は、小さなアプリを作る  
場合でも多機能な構成に付き合う必要があった。

|         | Django(フルスタッ<br>ク)   | Flask             |
|---------|----------------------|-------------------|
| 付属機能    | 認証・ORM・管理画面<br>が標準搭載 | 最小限(ルーティング<br>程度) |
| 構成の自由度  | やや決まった型に沿う           | 高い(必要な分だけ足<br>す)  |
| 学習コスト   | やや高い(覚える機能<br>が多い)   | 低い(小さく始められ<br>る)  |
| 向いている場面 | 機能が多い業務系<br>Webアプリ   | 小規模API・プロトタイ<br>プ |

### 3 Flask で何ができるようになった？



最小構成からすぐ  
に書き始められる



必要な機能だけ拡  
張できる



小規模～中規模に  
向いている

## ひとことメモ

FlaskはDjangoの「全部入り」に対する、  
「必要な分だけ持つ」という差分から生まれたフレームワーク。

## 関連キーワード

関連する技術



### Python

シンプルな文法を  
持つプログラミング  
言語。

P.172

比較するフレームワーク



### Django

電池付属の思想を  
持つフルスタックフ  
レームワーク。

P.180



### FastAPI

型ヒントを活かした  
高速なAPIフレー  
ムワーク。

P.181

# Gin

Goの速さを活かしてる、シンプルなWebフレームワーク

## ひとことというと

Go 言語の標準ライブラリだけでは冗長になりがちなルーティング処理を、シンプルな記法で扱えるようにしたWebフレームワーク。

1

## なぜ生まれた？

Goの標準ライブラリ(net/http)だけでWebサーバーを書くと、ルーティングやミドルウェアの処理が冗長になりがちだった。Goの速さを活かしつつ、書きやすさも両立させるために開発された。



2

## それまでどうしていた？

それまではGoの標準ライブラリだけでルーティングを書き、パラメータの取得なども自前で処理する必要があった。

|         | net/http(標準ライブラリ) | Gin                  |
|---------|-------------------|----------------------|
| ルーティング  | 自前で条件分岐を実装        | router.GET()などで簡潔に記述 |
| ミドルウェア  | 仕組みを自作            | 標準で組み込み・追加が容易        |
| 実行速度    | 速い                | 標準ライブラリに迫る速さを維持      |
| 向いている場面 | 極小のAPI            | REST API・マイクロサービス    |

3

## Gin で何ができるようになった？



Go 標準に迫る速度を維持できる



シンプルな記法でAPIを作れる



ミドルウェアを手軽に追加できる

## ひとことメモ

GinはGoの「速いが冗長になりがち」という弱点を、書きやすさで補ったフレームワーク。

## 関連キーワード

関連する技術



Go

並行処理を得意とするシンプルな言語。

P.173

# 184 | Laravel

「PHPは冗長」という評判を覆した、モダンなフルスタックフレームワーク

## ひとことというと

認証・ルーティング・ORMなどを整った形で備え、PHPでのWebアプリ開発を一気にモダンにしたフレームワーク。

### 1 なぜ生まれた？

PHPには多くのフレームワークが乱立していたが、設計が古かったり機能が断片的だったりして「モダンな開発体験」に欠けていた。Railsなどを参考に、洗練された設計のフレームワークとして開発された。



### 2 それまでどうしていた？

それまでのPHPフレームワークは機能がバラバラで、認証やルーティングを組み合わせるだけでも設定に手間がかかった。

|         | 従来のPHPフレームワーク | Laravel             |
|---------|---------------|---------------------|
| 機能の揃い方  | 断片的・組み合わせが必要  | 認証・ORM・キューなど統合済み    |
| 開発体験    | 設定や記法が古い      | モダンな記法・豊富なドキュメント    |
| DB操作    | SQLを都度記述      | Eloquent ORMで直感的に操作 |
| 向いている場面 | 小規模なWebページ    | 本格的なWebアプリ・SaaS     |

### 3 Laravel で何ができるようになった？



モダンな作法でWebアプリを構築できる



認証・ORMなどが標準で揃っている



豊富なパッケージで拡張できる

## ひとことメモ

LaravelはPHPを「昔ながらの言語」から「今も選ばれる言語」に引き戻した立役者ともいわれるフレームワーク。

## 関連キーワード

関連する技術



**PHP**  
HTMLに埋め込んで使えるサーバーサイド言語。

P.174

比較するフレームワーク



**Rails**  
Ruby版のフルスタックフレームワーク。

P.177

# Spring Boot

Javaの「設定地獄」を解消し、すぐに動くアプリを作れるようにしたフレームワーク

## ひとことという

煩雑だったSpringフレームワークの設定を自動化し、Javaで素早くアプリケーションを起動できるようにしたフレームワーク。

### 1 なぜ生まれた？

従来のSpring FrameworkはXMLなどの設定ファイルが膨大になりがちで、「動かすまでが長い」と言われていた。この設定の煩雑さを解消し、Javaでもすぐに開発を始められるようにするために開発された。



### 2 それまでどうしていた？

それまでのSpring Frameworkでは、DBやサーバーの接続設定をXMLなどで細かく記述し、動かすまでに多くの準備が必要だった。

|         | 従来のSpring Framework | Spring Boot                     |
|---------|---------------------|---------------------------------|
| 設定方法    | XMLなどで細かく記述         | 規約に沿えば自動設定 (auto-configuration) |
| 起動までの準備 | 多い(サーバー構築も別途)       | 組み込みサーバーで即座に起動                  |
| 依存関係管理  | 個別に指定・バージョン調整       | starterでまとめて管理                  |
| 向いている場面 | 既存の大規模Springアプリ     | 新規のJavaアプリ・マイクロサービス             |

### 3 Spring Boot で何ができるようになった？



すぐに動くアプリを構築できる



設定の大部分を自動化できる



エンタープライズ規模でも安定動作

## ひとことメモ

Spring Bootは「Springの機能はそのままに、面倒な設定だけをなくす」という発想で作られたフレームワーク。

## 関連キーワード

関連する技術



### Java

JVM上で動作し、OSを問わず動かせる言語。

P.175

# ド ッ カ ー Docker

「私の PC では動きます」を終わらせた技術

## ひとことという

アプリとその実行に必要なものを「箱」に入れて、どこでも同じように動かす仕組み。

### 1 なぜ生まれた？

環境(OS・ライブラリ・設定など)の違いで、「ローカルでは動くのに、本番では動かない」が頻発していた。



### 2 それまでどうしていた？

仮想マシン(VM)を使って、OSごと分離していた。

|         | VM(仮想マシン)   | Docker      |
|---------|-------------|-------------|
| 分離の単位   | OSごと        | アプリ+必要なものだけ |
| 重さ・起動   | 重い・遅い       | 軽い・速い       |
| 再現性・配布  | 環境の用意が大変    | 同じ箱を渡せる     |
| 向いている場面 | OSごと分離したいとき | 開発・テスト・配布   |

### 3 Docker で何ができるようになった？



すぐに起動できる



同じ環境を共有できる



配布・デプロイが簡単

#### ひとことメモ

Docker は「環境ごとアプリを運ぶ」イメージ。  
必要なものを一つの箱にまとめるから、どこでも同じように動く。

## 関連キーワード

比較する技術



### VM(仮想マシン)

OSごと分離する仮想化技術。重いが強いつ分離ができる。

P.401

一緒に使う技術



### Docker Compose

複数のコンテナをまとめて定義・実行するツール。

P.321



### Container

Docker で作られるアプリ実行の単位。

P.309



### Image

コンテナを作るための設計図(テンプレート)。

P.305

次に読むなら



### Kubernetes

コンテナを大規模に自動で管理・運用するプラットフォーム。

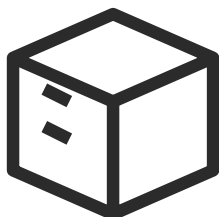
P.441

#### キーワードメモ

箱に入れるものは、アプリや設定、ライブラリなど。中身は自由に作れる。

# Image

コンテナを作るための「設計図」



インフラ  
コンテナ

## ひとことという

アプリと必要なものを 1 枚に固めた、コンテナのもとになる読み取り専用のテンプレート。

1

## なぜ生まれた？

同じ環境を何度でも・どこでも再現するために、状態を 1 つに固めておきたいから。



2

## それまでどうしていた？

手順書を見ながら、サーバーへ必要なものを手動でインストールしていた。

|       | 手動構築    | イメージ     |
|-------|---------|----------|
| 環境の再現 | 手順書頼み   | イメージで同じ  |
| 共有方法  | 手順書を渡す  | イメージごと渡す |
| 個人差   | 人によって違う | 必ず同じになる  |
| 管理    | バラバラ    | レジストリで管理 |

3

## イメージで何ができる？



層を重ねて作る



どこでも同じを再現



レジストリで共有

## 関連キーワード

一緒に使う技術



### Container

イメージから生まれた、動くアプリの箱。

P.309



### Docker

イメージを作り・動かす代表的なツール。

P.301

次に読むなら



### Docker Compose

複数のイメージをまとめて動かす設定。

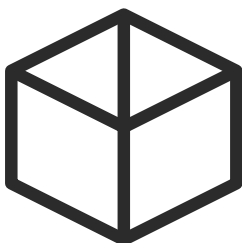
P.321

## キーワードメモ

イメージは層(レイヤー)の積み重ね。差分だけ足すので軽い。

# Container

イメージから生まれた「動くアプリの箱」



インフラ  
コンテナ

## ひとことという

イメージを実行して動き出した、アプリ 1 つ分の隔離された実行環境。

## 1 なぜ生まれた？

アプリを周りから隔離して、どこでも同じように、軽く動かしたかったから。



## 2 それまでどうしていた？

VM(仮想マシン)を立てて、OS ごと分離していた。

|       | VM(仮想マシン) | コンテナ   |
|-------|-----------|--------|
| 分離の単位 | OS ごと     | アプリ+依存 |
| 重さ    | 重い        | 軽い     |
| 起動    | 分単位       | 秒単位    |
| 使い捨て  | しにくい      | しやすい   |

## 3 コンテナで何ができる？



すぐ起動・破棄できる



同じイメージから量産



つないでシステムに

## 関連キーワード

比較する技術



### VM(仮想マシン)

OS ごと分離する、重いが強い仮想化。

P.401

一緒に使う技術



### Image

コンテナのもとになる設計図。

P.305



### Docker

コンテナを作り・動かす代表的なツール。

P.301

次に読むなら



### Kubernetes

大量のコンテナを自動で運用する仕組み。

P.441

## キーワードメモ

1 コンテナ=1 役割が基本。増やして並べて、システムを組む。

# ボリューム

コンテナが消えてもデータを残す「永続化の仕組み」



インフラ  
データ

## ひとことという

コンテナが削除されてもデータが消えないよう、ホスト側の保存場所をコンテナにつなぐ仕組み。

### 1 なぜ生まれた？

コンテナは「使い捨て」が前提なので、中にデータを書くとコンテナごと消えてしまうから。



### 2 それまでどうしていた？

コンテナ内のファイルシステムに直接データを書き込んでいた。

|          | コンテナ内保存 | Volume  |
|----------|---------|---------|
| コンテナ削除後  | 消える     | 残る      |
| 複数コンテナ共有 | できない    | できる     |
| バックアップ   | 手動・難しい  | ホスト側で管理 |
| DB との相性  | 悪い      | 良い      |

### 3 Volume で何ができる？



データを永続化



コンテナ間で共有



バックアップが楽

Volume は「コンテナの外の倉庫」。  
DB のデータや生成ファイルはここに入れるのが基本。

**313** 差分でわかる エンジニアのゆるい辞典

## 関連キーワード

---

Container / 100 ~ 1000

コンポーズ

# Compose

複数コンテナを「1 枚の設定」でまとめて動かす



インフラ  
ツール

ひとことというと

何個ものコンテナの構成を 1 つのファイルに書いて、まとめて起動・停止できるツール。

1 なぜ生まれた？

Web・DB・キャッシュなど複数のコンテナを、毎回手で起動するのが大変だったから。



2 それまでどうしていた？

docker run でコンテナを 1 個ずつ手動で起動していた。

|     | docker run | Compose |
|-----|------------|---------|
| 対象  | 1 コンテナずつ   | まとめて一括  |
| 設定  | コマンドに毎回    | ファイルに固定 |
| 再現性 | 手順に依存      | 誰でも同じ   |
| 起動  | 何度もコマンド    | up 一発   |

3 Compose で何ができる？



構成をファイル化



up で一括起動



依存関係を自動解決

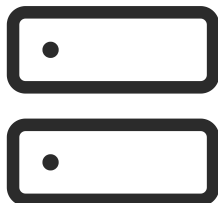
Compose は「構成の設計図」。開発環境を“ファイルごと”配れるのが強み。

**321** 差分でわかる エンジニアのゆるい辞典

**関連キーワード**

---

「1 台の中に、もう 1 台の PC」を作る技術



インフラ  
仮想化

ひとことという

物理マシンの上に、OS ごと丸ごとの“もう一台”を立ち上げて動かす仕組み。

1 なぜ生まれた？

高性能なサーバを 1 つの用途で遊ばせず、独立した複数の環境に分けて使いたかったから。



2 それまでどうしていた？

物理サーバーを 1 台に 1 用途で購入・設置していた。

|         | 物理サーバー   | VM(仮想マシン) |
|---------|----------|-----------|
| 1 台の使い方 | 1 用途のみ   | 複数に分割できる  |
| コスト     | 用途の数だけ購入 | 1 台で済む    |
| 柔軟性     | 増やすには購入  | すぐ追加できる   |
| 分離      | 物理的に完全に別 | OS レベルで分離 |

3 VM で何ができる？



別 OS をそのまま動かせる



環境を丸ごと保存できる



スナップショットで戻せる

関連キーワード

比較する技術



Docker

アプリを箱に入れて軽く動かす技術。

P.301

一緒に使う技術



Container

アプリ単位で分ける、軽い実行環境。

P.309

次に読むなら



Kubernetes

大量のコンテナを自動で運用する仕組み。

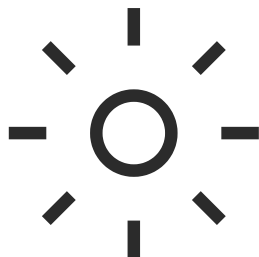
P.441

キーワードメモ

仮想化は「1 台を分ける」発想。分け方の細かさで VM とコンテナに分かれる。

# Kubernetes

大量のコンテナを「自動で」まわす司令塔



インフラ  
運用基盤

## ひとことという

たくさんのコンテナを、複数のサーバ上で自動で配置・維持・回復してくれる仕組み。

## 1 なぜ生まれた？

コンテナが増えすぎて、  
人手で配置・監視・再起動しきれなくなったから。



## 2 それまでどうしていた？

Compose や手動スクリプトで、1 台ずつ管理・監視していた。

|      | Docker Compose | Kubernetes |
|------|----------------|------------|
| 規模   | 1 台想定          | 多数のサーバ     |
| 障害時  | 手で対応           | 自動で復旧      |
| スケール | 手動             | 自動で増減      |
| 用途   | 開発中心           | 本番運用       |

## 3 Kubernetes で何ができる？



複数サーバに自動  
配置



落ちても自動で復  
旧



負荷に応じて増減

## 関連キーワード

比較する技術



### Docker Compose

1 台でまとめてコンテナを動かす設定。

P.321

一緒に使う技術



### Container

Kubernetes が配置・管理する最小単位。

P.309



### Docker

動かすコンテナ・イメージを作る土台。

P.301

次に読むなら



### Image

各コンテナのもとになる設計図。

P.305

## キーワードメモ

「宣言的」に理想状態を書く。ズレたら自動で直しに行く。

# DevOps

「開発」と「運用」の壁をなくす考え方



開発  
運用

## ひとことという

開発 (Dev) と運用 (Ops) を一体で動かし、高速かつ安定したリリースを継続的に実現する文化・考え方。

### 1 なぜ生まれた？

開発チームは「素早く変更する」、運用チームは「安定させる」という相反する目標を持ち、リリースのたびに摩擦・遅延・障害が繰り返されていたから。



### 2 それまでどうしていた？

Dev (開発) と Ops (運用) は別チームで、「作る人」と「動かす人」の間に高い壁があった。

|        | 従来の体制         | DevOps   |
|--------|---------------|----------|
| チーム構成  | Dev と Ops が分断 | 一体で動く    |
| リリース頻度 | 数か月に1回・手動     | 頻繁・自動化   |
| 障害対応   | 責任の押し付け合い     | 同じゴールで動く |
| 品質管理   | 後工程でまとめてテスト   | 常時テスト・監視 |

### 3 DevOps で何ができるようになった？



デプロイサイクル  
が速くなる



自動化でミスが減  
る



開発と運用が同じ  
目標で動ける

DevOps はツールより「文化」。CI/CD や IaC はその実現手段の一つ。  
SRE は信頼性に特化した DevOps の実践形。

**701** 差分でわかる エンジニアのゆるい辞典

**関連キーワード**

---

中田 英樹